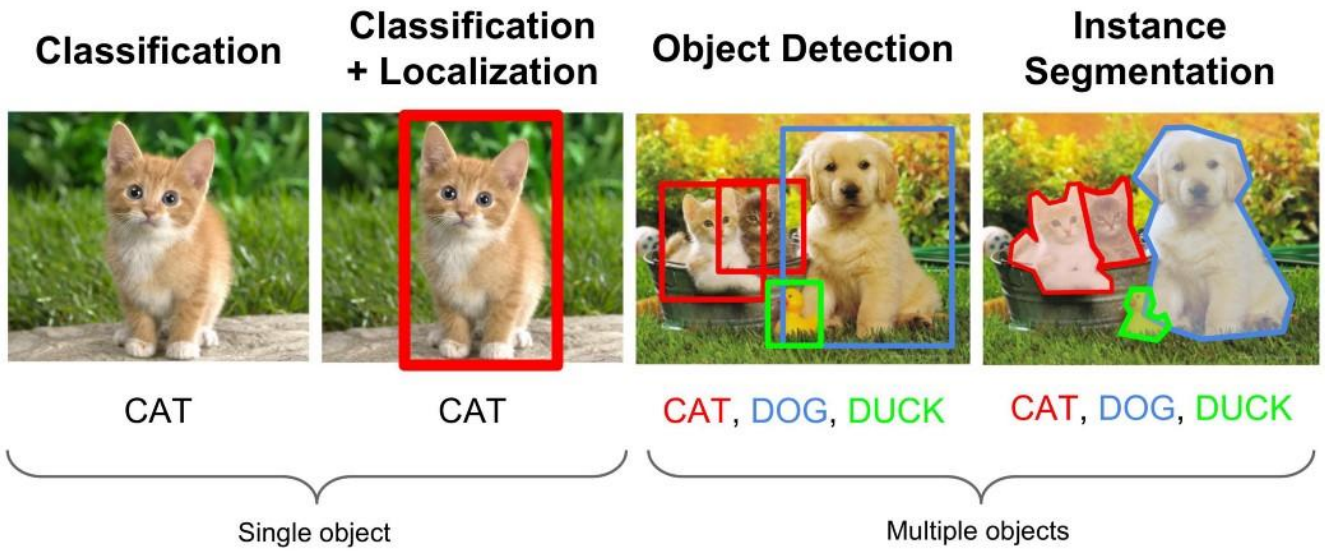
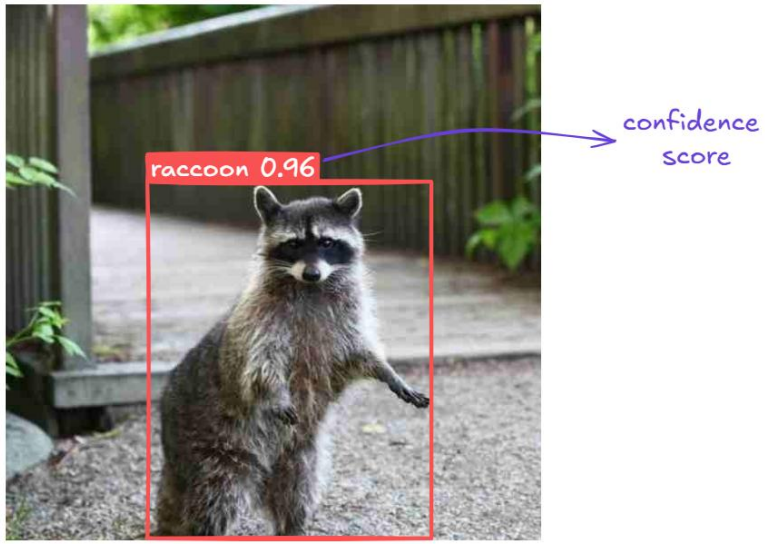
4. Applications

Object Detection

Object detection



- **What ?** → **Classification** → **Class**
  - Confidence score
- **Where ?** → **Localization** → **Bounding box**
  - IoU (Intersection over Union)



https://herbwood.tistory.com/2
https://blog.roboflow.com/object-detection-metrics/

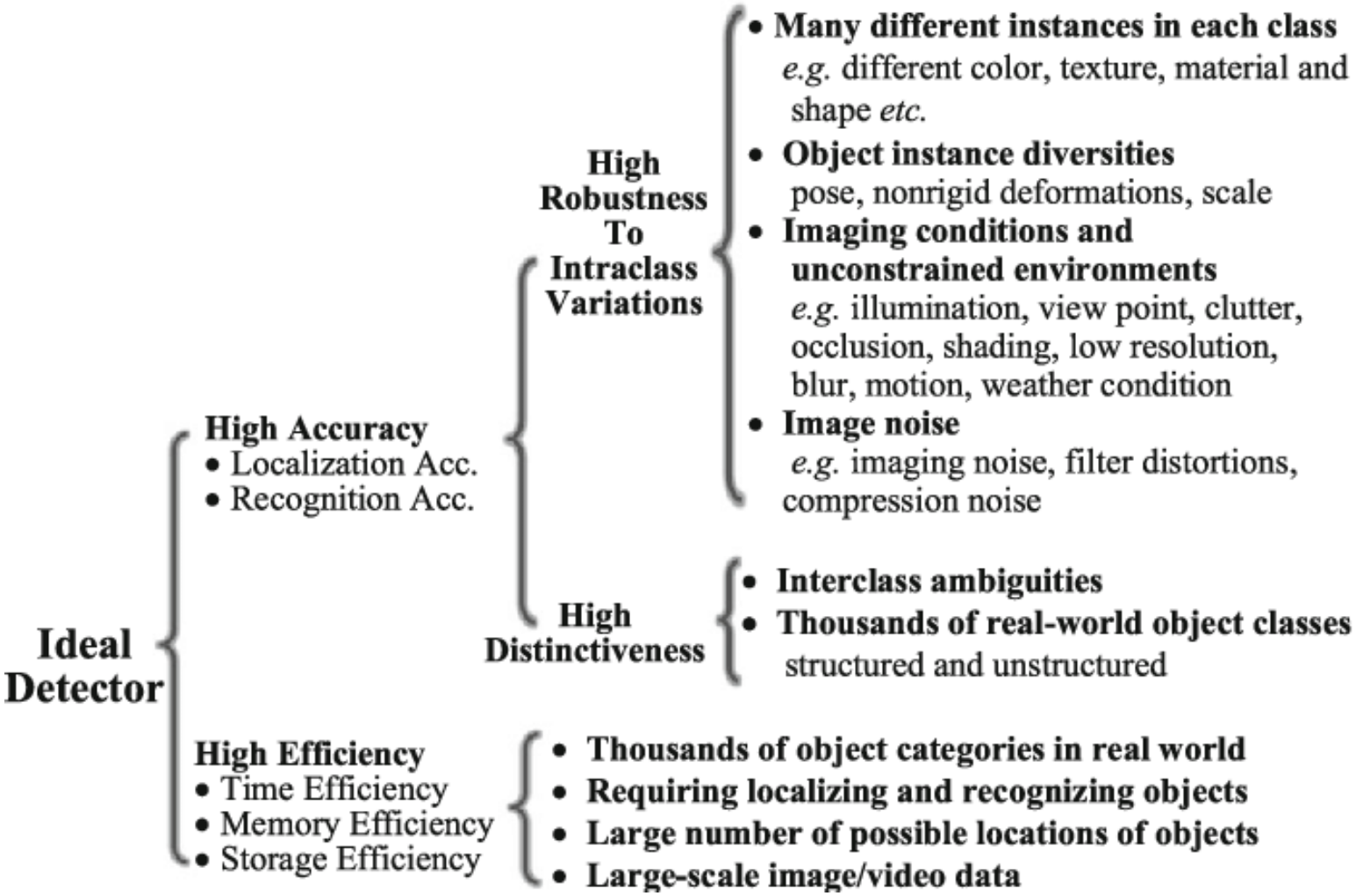
Applications



https://www.geeksforgeeks.org/computer-vision/real-world-applications-of-object-detection/

What makes object detection difficult?

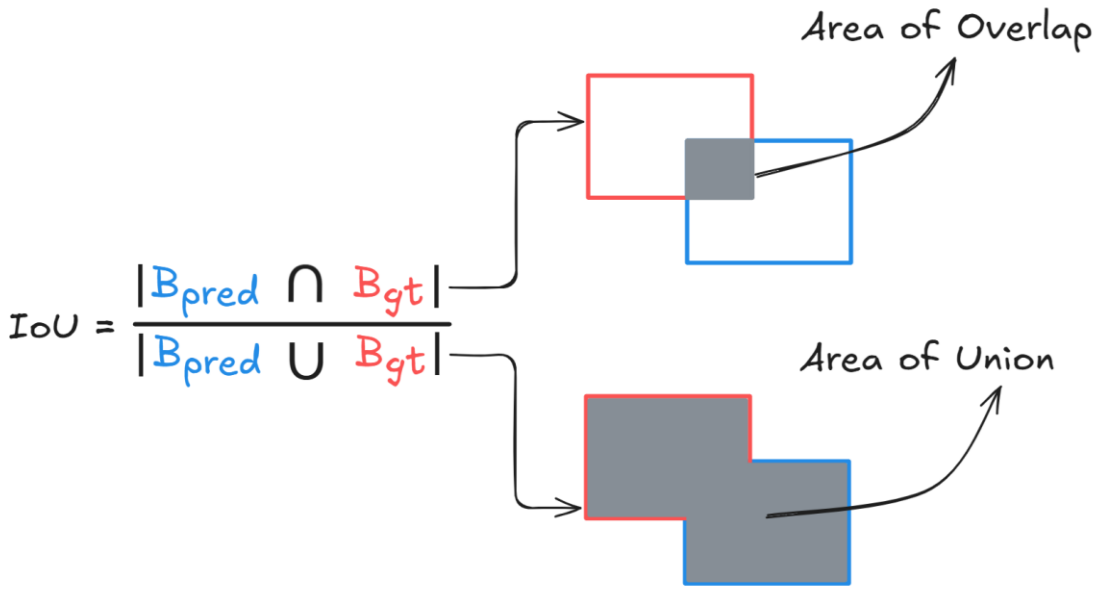
- **Scale change**
  - Close (big) or far (small)
- **Occlusion**
  - Partial
- **Pose change**
  - Rigid or non-rigid
  - Rotation
- **Background complexity**
  - Similar objects
- **Light condition**
  - Too dark or bright

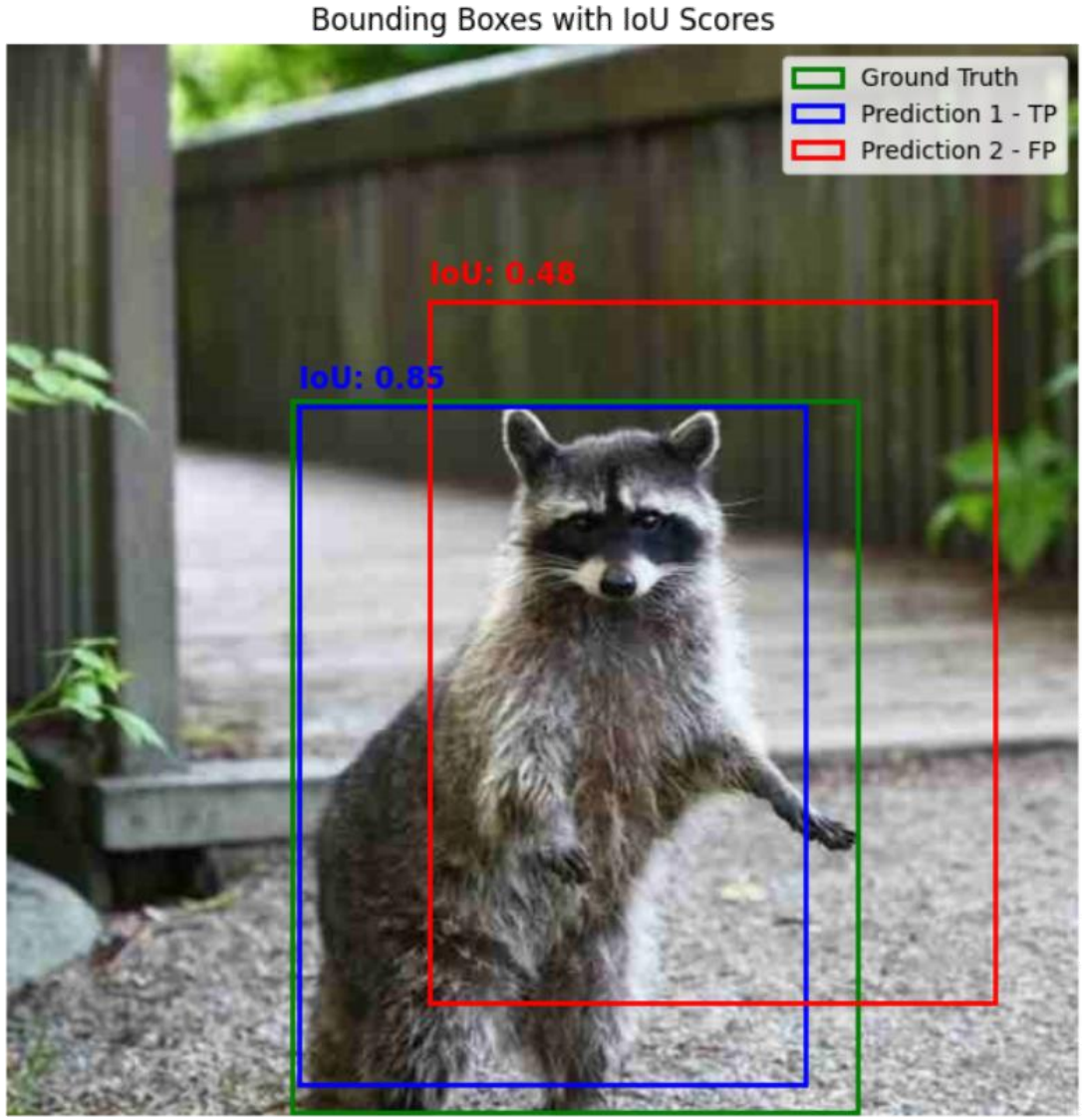


Deep Learning for Generic Object Detection: A Survey, 2019

Detection metric (1/4)

- **IoU (Intersection over Union)**





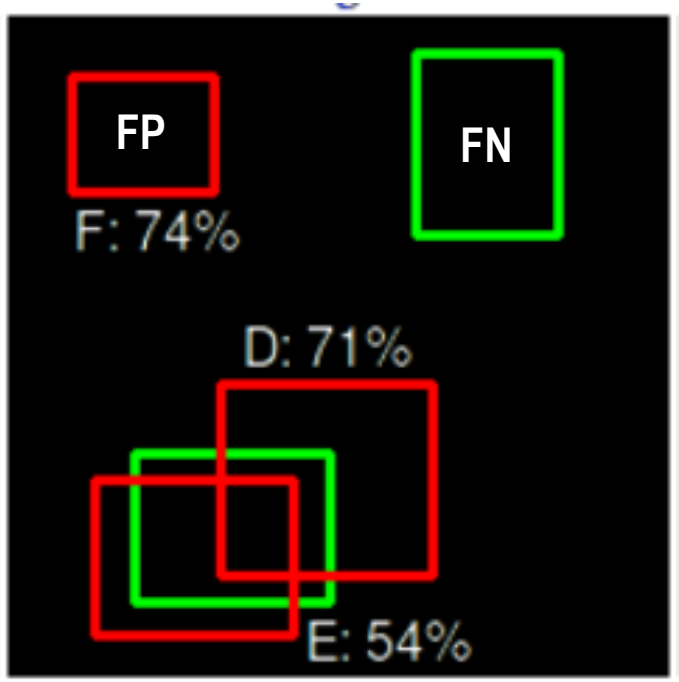
https://blog.roboflow.com/object-detection-metrics/

Detection metric (2/4)

- **Confusion matrix of object detection**
  - Confidence score < threshold : Prediction drop

- **Then, for the rest,**
  - TP (True Positive) : (Correct class) AND (IoU >= threshold) AND (Unique match)
  - FP (False Positive) : (False class) OR (IoU < threshold) OR (Rest of the unique match)
  - FN (False Negative) : Missing ground truth
  - TN (True Negative) : Meaningless in object detection

		Detection (Prediction)	
		True	False
Ground truth	True	TP	FN
	False	FP	TN



: Detection

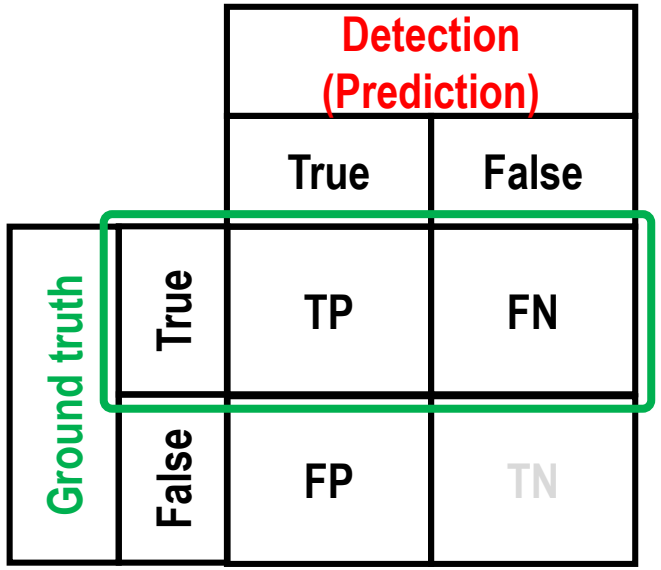
: Ground truth

- **Confidence score threshold : 50%**
- **IoU threshold : 50%**

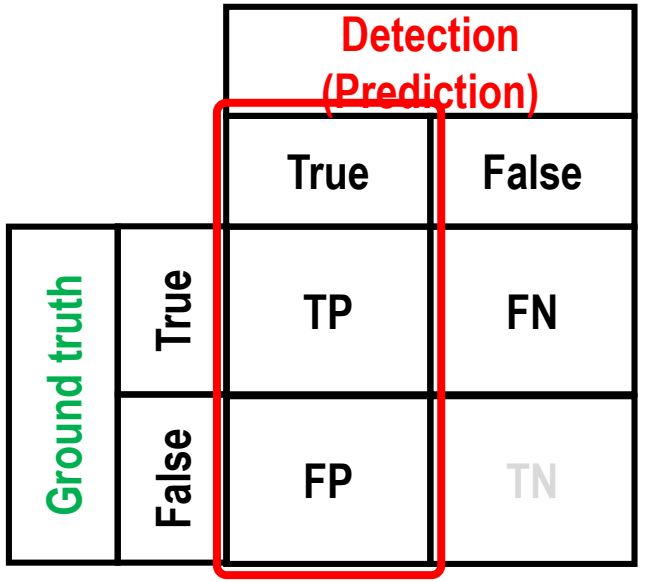
https://herbwood.tistory.com/2

Detection metric (3/4)

- **Confusion matrix**



$$\text{Recall} = \frac{TP}{TP + FN}$$

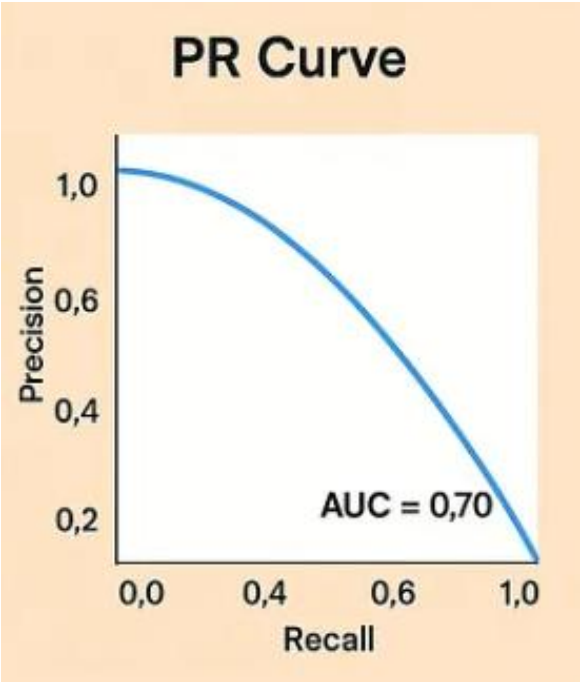


$$\text{Precision} = \frac{TP}{TP + FP}$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Detection metric (4/4)

- **Average Precision (AP) is the area under the Precision–Recall (PR) curve for a single class.**
  - Sorting predictions by confidence score (high → low)
  - Calculating TP/FP/FN at each confidence threshold
  - Drawing the Precision–Recall curve
  - Computing the integral of the PR curve (AUC)



https://blog.roboflow.com/object-detection-metrics/

- **Mean Average Precision (mAP) is simply the mean of AP across all classes**

$$mAP = \frac{1}{N} \sum_{c=1}^{N} AP_c$$

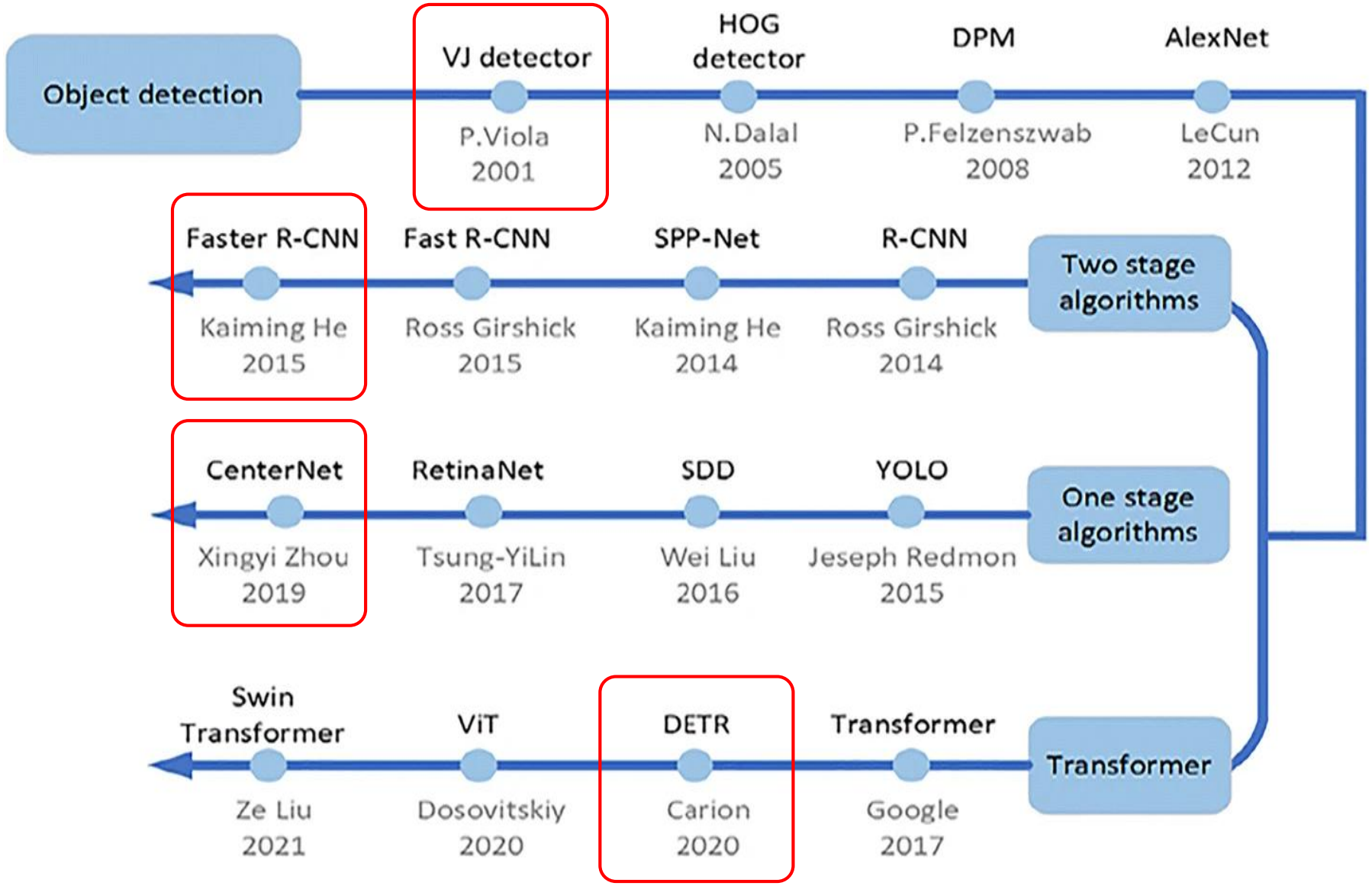
- **COCO Evaluation metric**
  - AP averaged over multiple IoU thresholds → AP@[.50:.05:.95]

$$AP = \frac{1}{10} \sum_{t=0.50}^{0.95} AP_t$$

  - AP by object size → AP@[small / medium / Large]

	Small	Medium	Large
Object Area Range	< 32^2 pixels	32^2 ~ 96^2 pixels	> 96^2 pixels

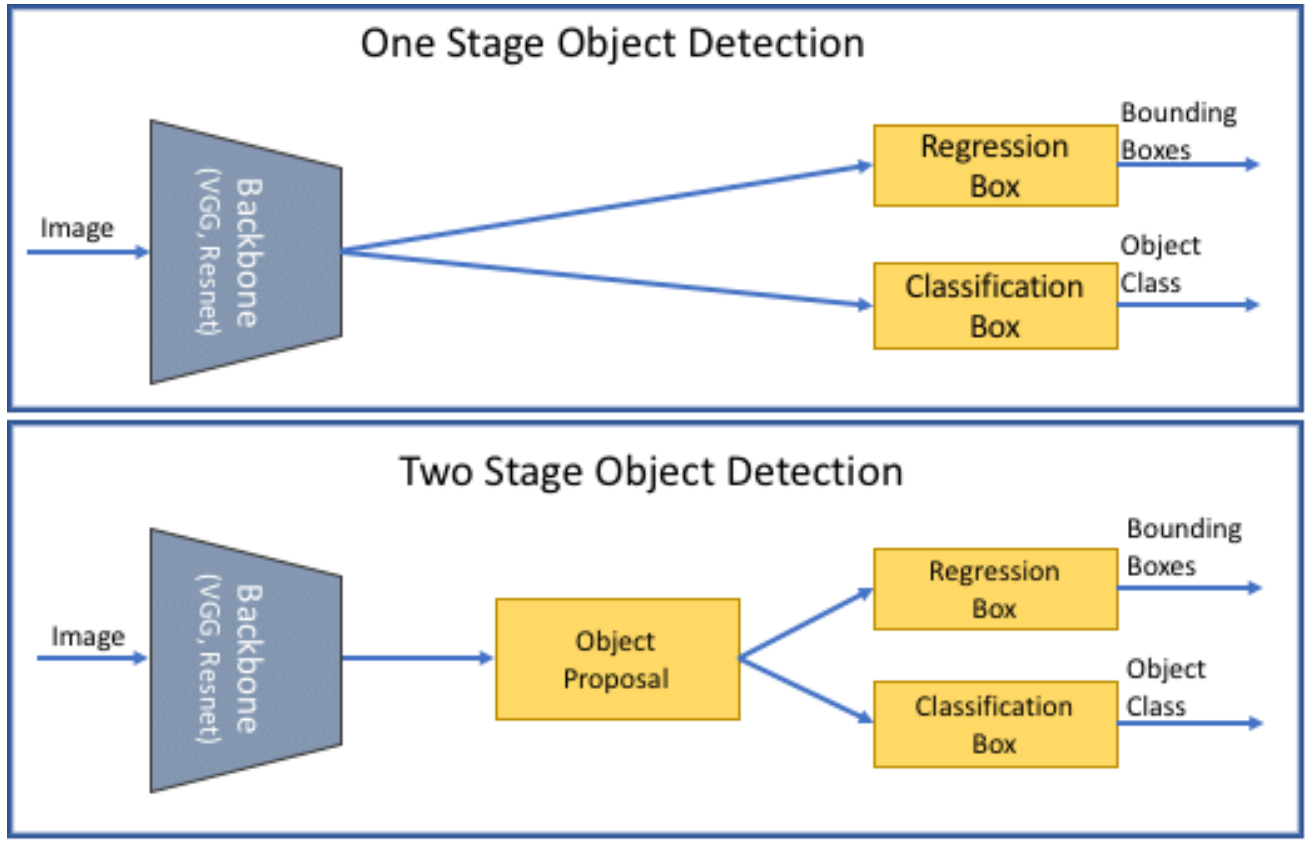
Evolution



A survey: object detection methods from CNN to transformer

One stage vs. Two stage detectors

- **One stage detectors**
  - focus on speed by directly prediction objects in a single pass
- **Two stage detectors**
  - focus on accuracy by using object (i.e., region) proposal

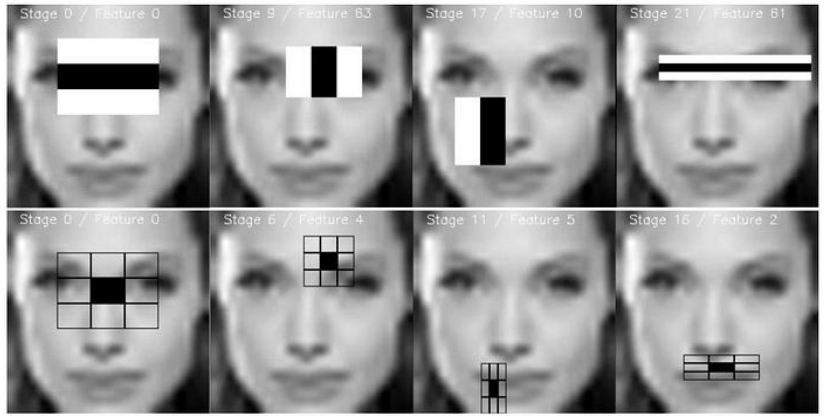


https://www.researchgate.net/figure/Two-stage-vs-one-stage-object-detection-models_fig3_353284602

[Viola-Jones face detection]

Rapid object detection using a boosted cascade of simple features, CVPR 2001

- 1) Selecting Haar-like features



- 2) Computing features using the Integral images

0	1	1	1
1	2	2	3
1	2	1	1
1	3	1	0

0	1	2	3
1	4	7	11
2	7	11	16
3	11	16	21

- 3) Running AdaBoost training to build a strong classifier

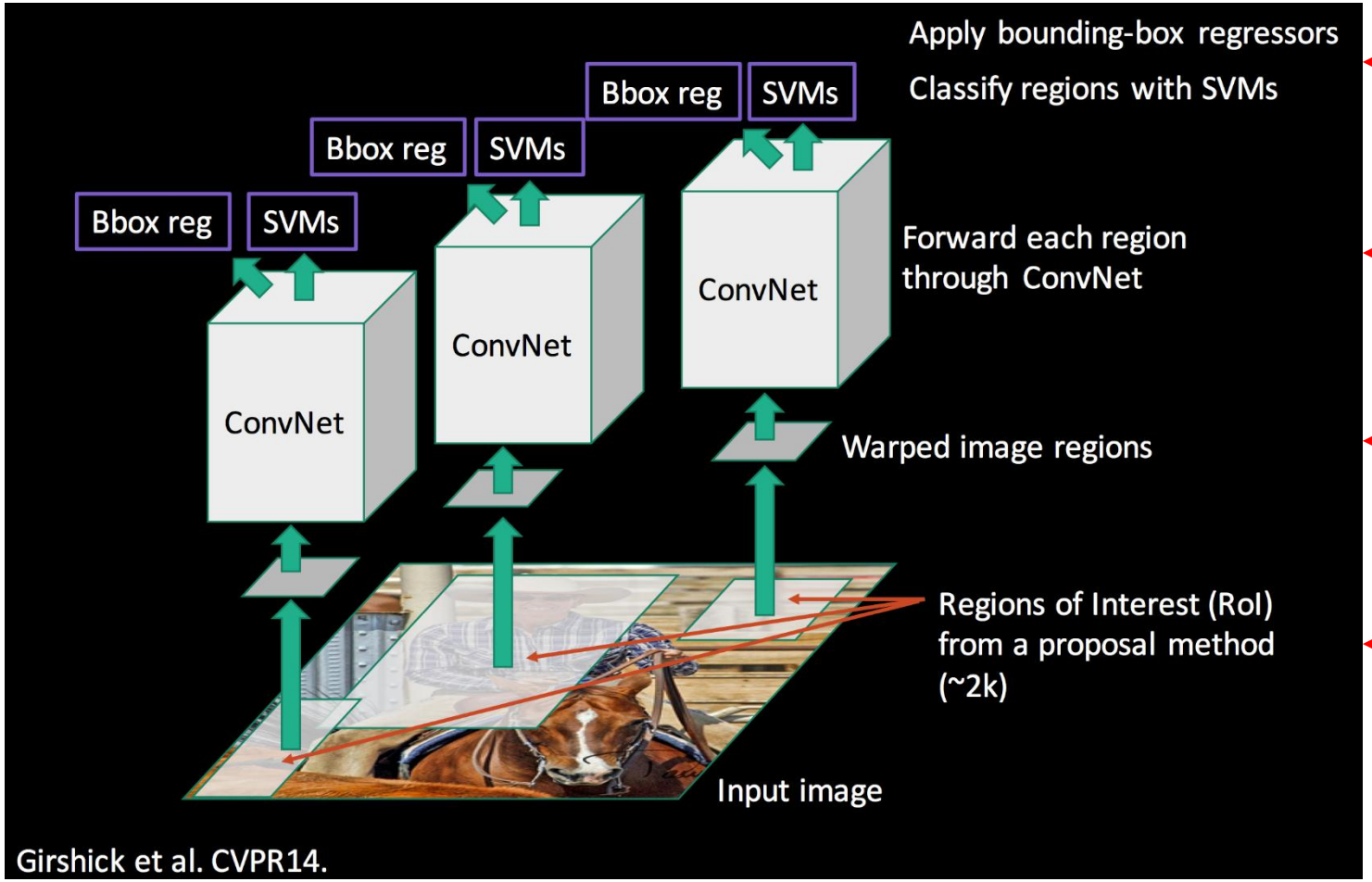


- 4) Creating classifier cascades
  - To quickly reject negative (i.e., non-face) regions

Rapid object detection using a boosted cascade of simple features, 2001
https://medium.com/@Andrew_D./computer-vision-viola-jones-object-detection-d2a609527b7c
https://medium.datadriveninvestor.com/understanding-adaboost-and-scikit-learns-algorithm-c8d8af5ace10

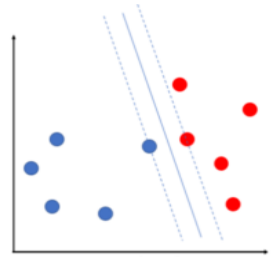
[R-CNN]

- **Architecture**



https://wikidocs.net/148633

- SVM binary classifier for each class
- Non-Maximum Suppression
- Bounding box regression



AlexNet → Fine tuning

227 x 227

Object proposal (e.g., **Selective search**)

- Initial segmentation
- Similarity computation
  - Color, texture, size, spatial
- Hierarchical grouping
- Multi-scale strategy
- Final output
  - ~2,000 regions / image

NMS

- **Non-Maximum Suppression**
  - Elimination of redundant boxes
  - Keep only the most accurate prediction for each object



Input: $B = \{b_1, .., b_N\}, S = \{s_1, .., s_N\}, Th$
B:list of detection boxes
S: corresponding detection scores
Th: NMS threshold

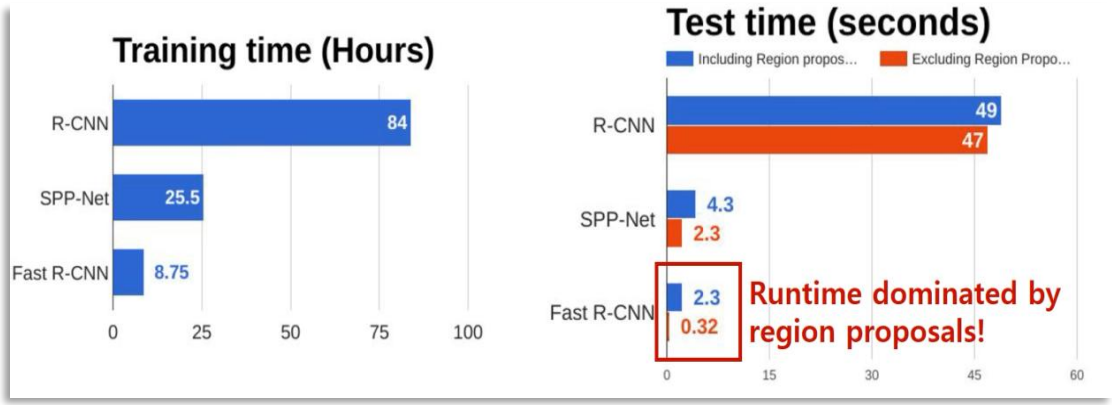
Begin:
$D \leftarrow \emptyset$
while $B \neq \emptyset$ **do**
$m \leftarrow argmax\ S$
$M \leftarrow b_m$
$D \leftarrow D \bigcup M, \quad B \leftarrow B - M$
for b_i *in* B **do**
if $iou(M, b_i) \geq Th$ **then**
$B \leftarrow B - b_i; \quad S \leftarrow S - s_i$
end
end
end
return D, S
end

Remove redundant boxes

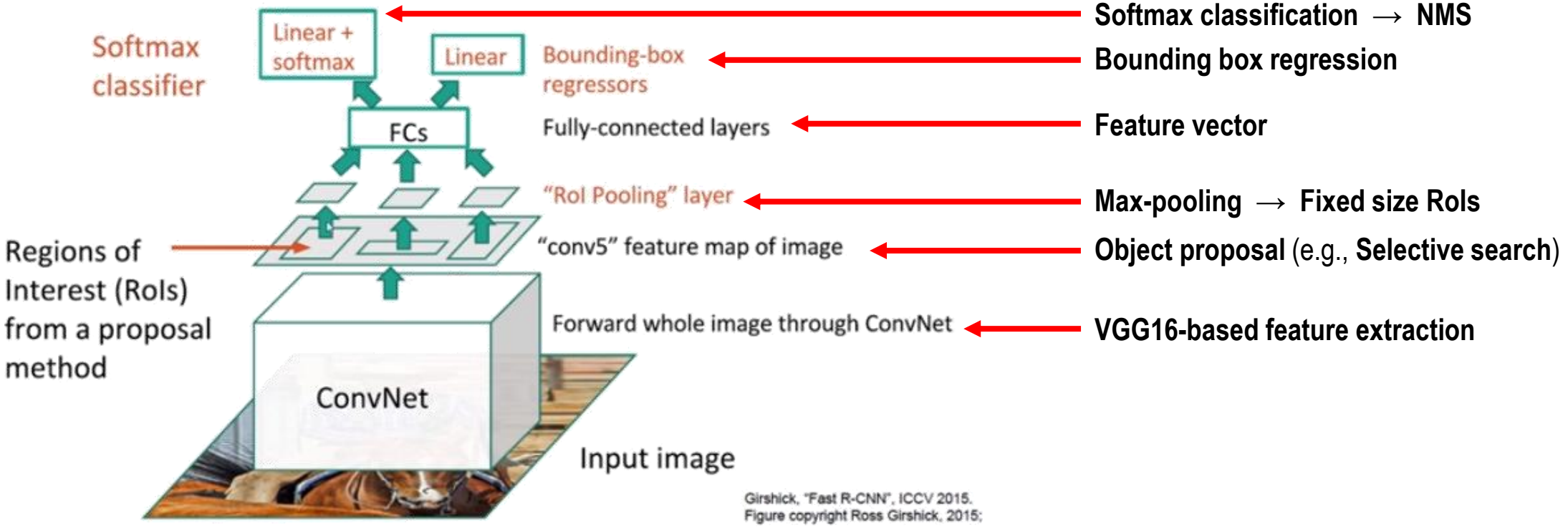
Graph Convolution and Self Attention Based Non-maximum Suppression, Advances in Neural Networks – ISNN 2019

[Fast R-CNN]

- **R-CNN drawbacks**
  - Requires warping each region proposal
  - Processes about 2000 region proposals independently through the CNN
  - Selective Search is a CPU-bound, non–GPU-friendly algorithm
  - No computation sharing across proposals



- **Architecture**



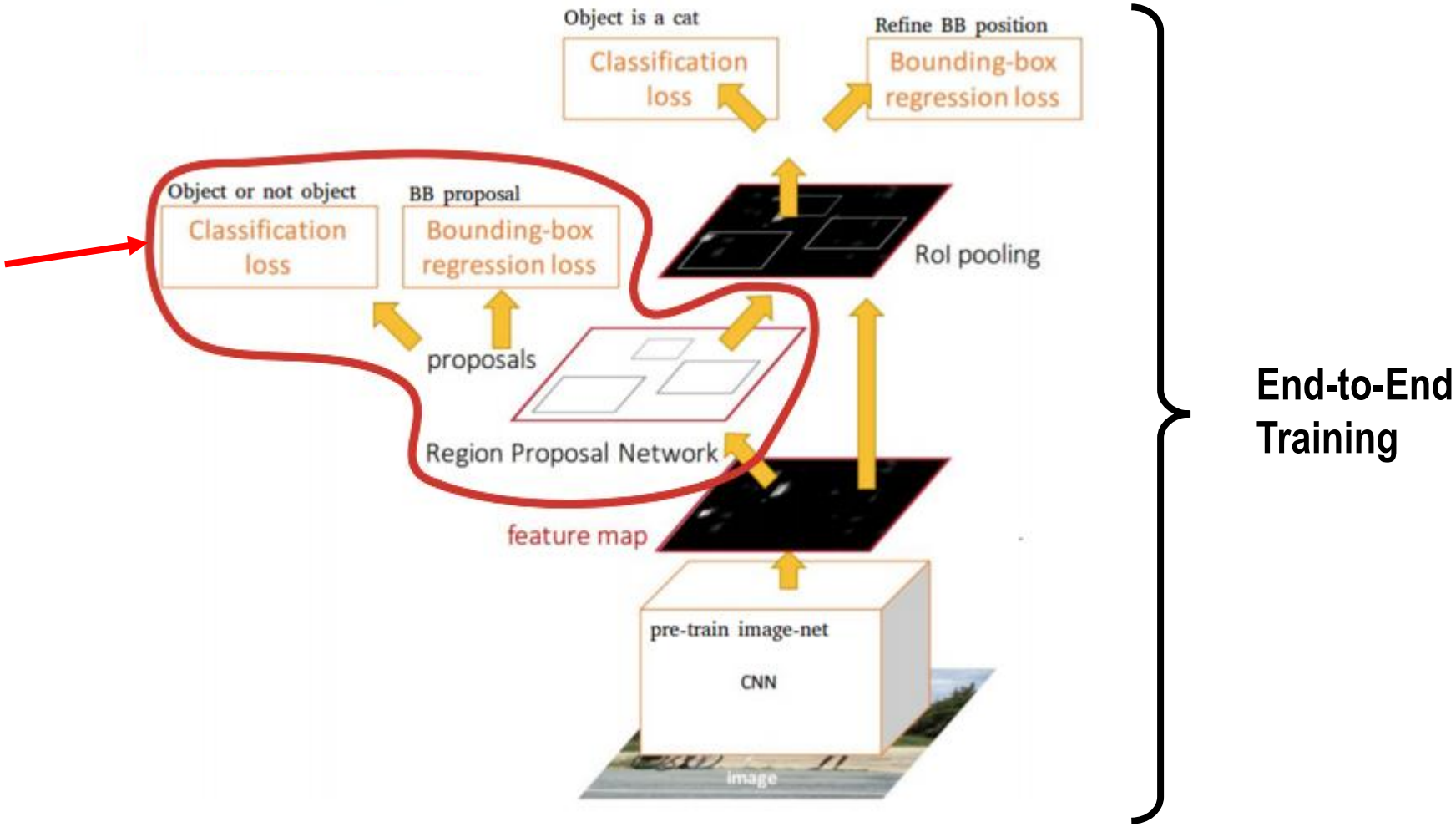
Girshick, "Fast R-CNN", ICCV 2015.
Figure copyright Ross Girshick, 2015;

https://wikidocs.net/148634

[Faster R-CNN]

- **Fast R-CNN drawbacks**
  - Selective Search (i.e., RPN) is still a CPU-bound, non–GPU-friendly algorithm (it takes 2 sec.)

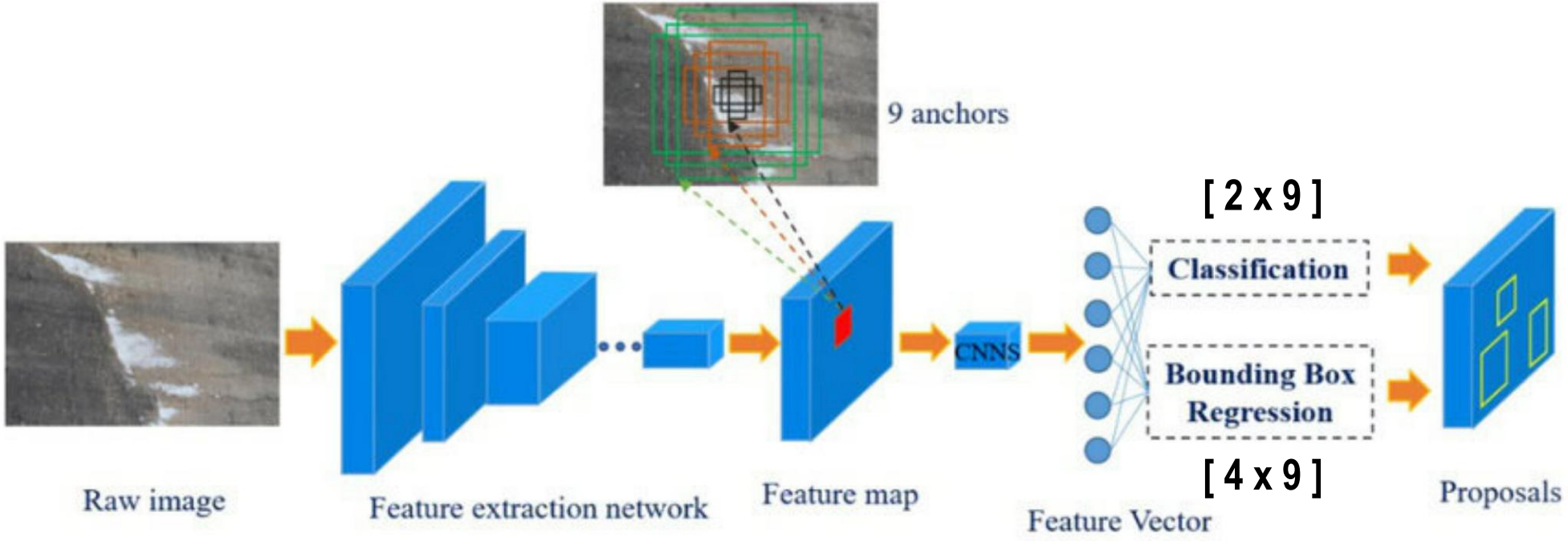
- **Architecture**



https://woochan-autobiography.tistory.com/921

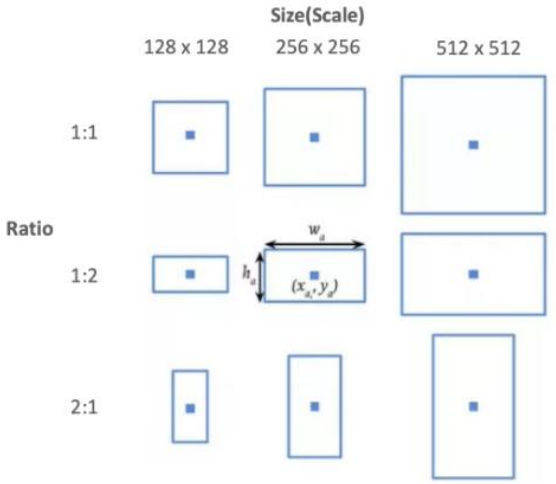
RPN (1/2)

- **RPN training**
  - Classification
    - Measure the overlaps (i.e., IoU) between each anchor and Ground truth
      - Positive : IoU > 0.7
      - Negative : IoU < 0.3
      - Ignore : Others
  - Bounding box regression
    - Offset regression for only positive anchors



RPN (2/2)

- **Anchor boxes**

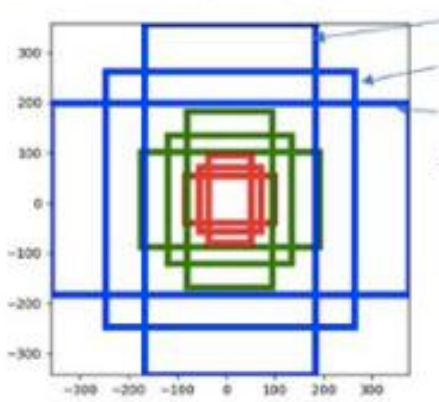


9 anchors
at each locations

Generate Anchors

Given:

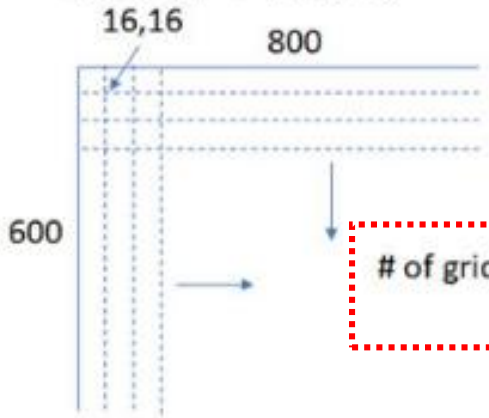
- Set of aspect ratios (0.5, 1, 2)
- Stride length (downscaling performed by resnet head: 16)
- Anchor Scales (8, 16, 32)



Aspect Ratio: 0.5
Aspect Ratio: 1
Aspect Ratio: 2

of anchors: 9

Add anchors to every grid location

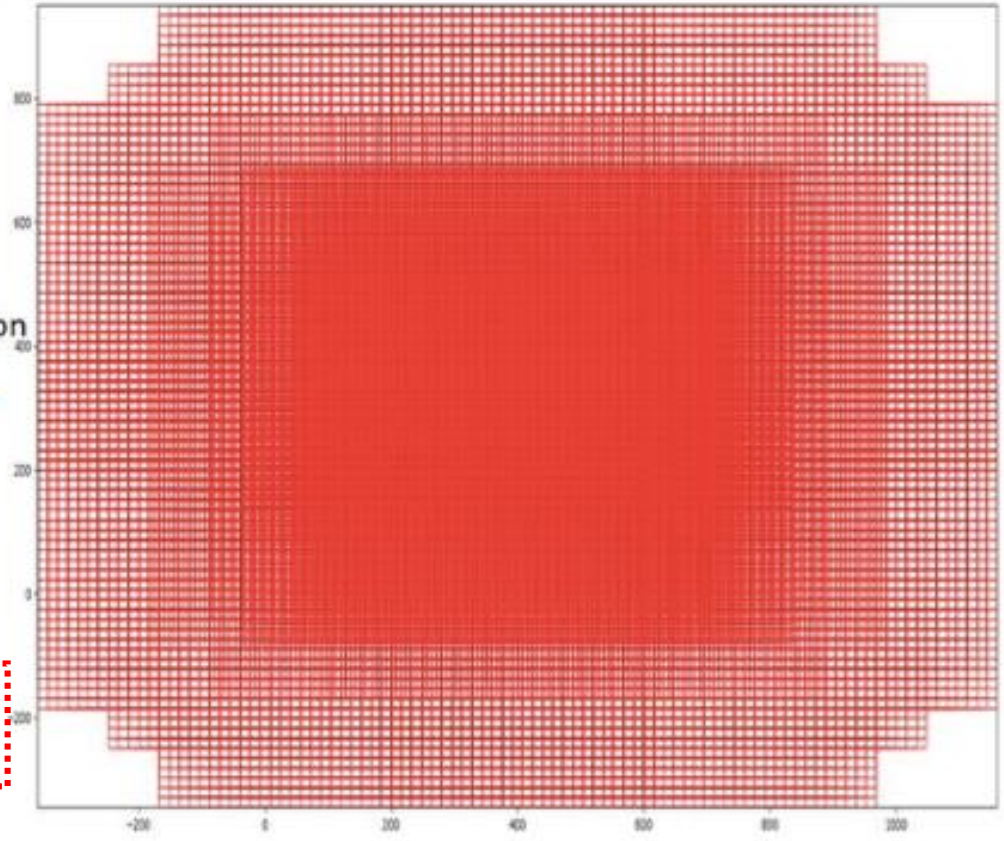


of grid locations: $\frac{800}{16} * \frac{600}{16} = 1900$

Create uniformly spaced grid with spacing = stride length

Total number of anchors: 1900*9 = 17100

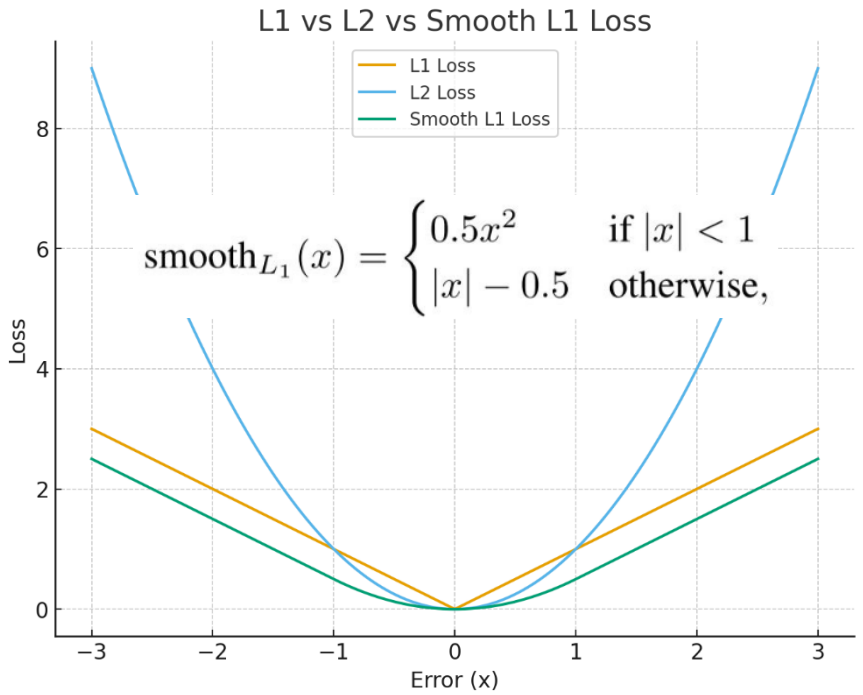
Some boxes lie outside the image boundary



https://woochan-autobiography.tistory.com/921

Loss

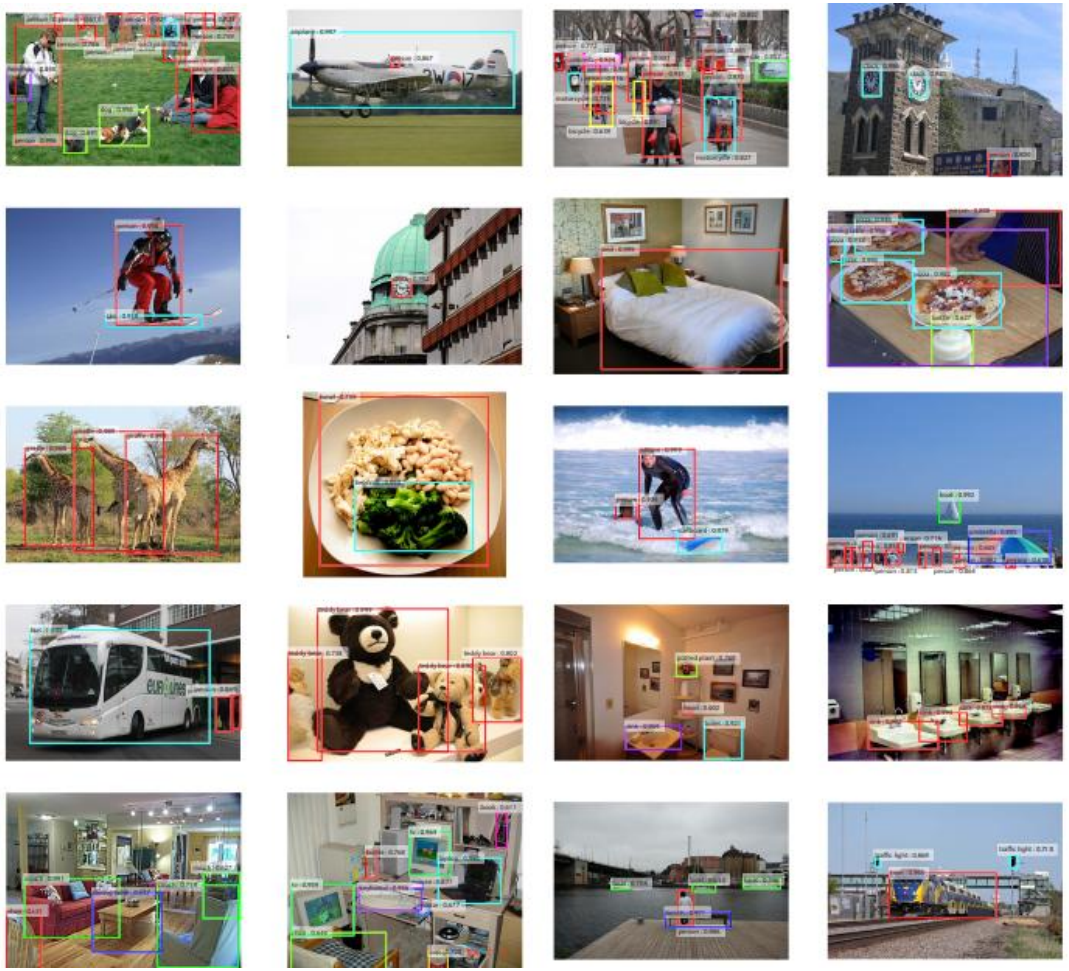
	Classification loss	Bounding box regression loss
RPN	Binary classification (Object vs. Background) using cross-entropy	Smooth L1 loss for anchor box coordinate refinement
Detection	Multi-class classification using cross-entropy	Smooth L1 loss for final bounding box adjustment

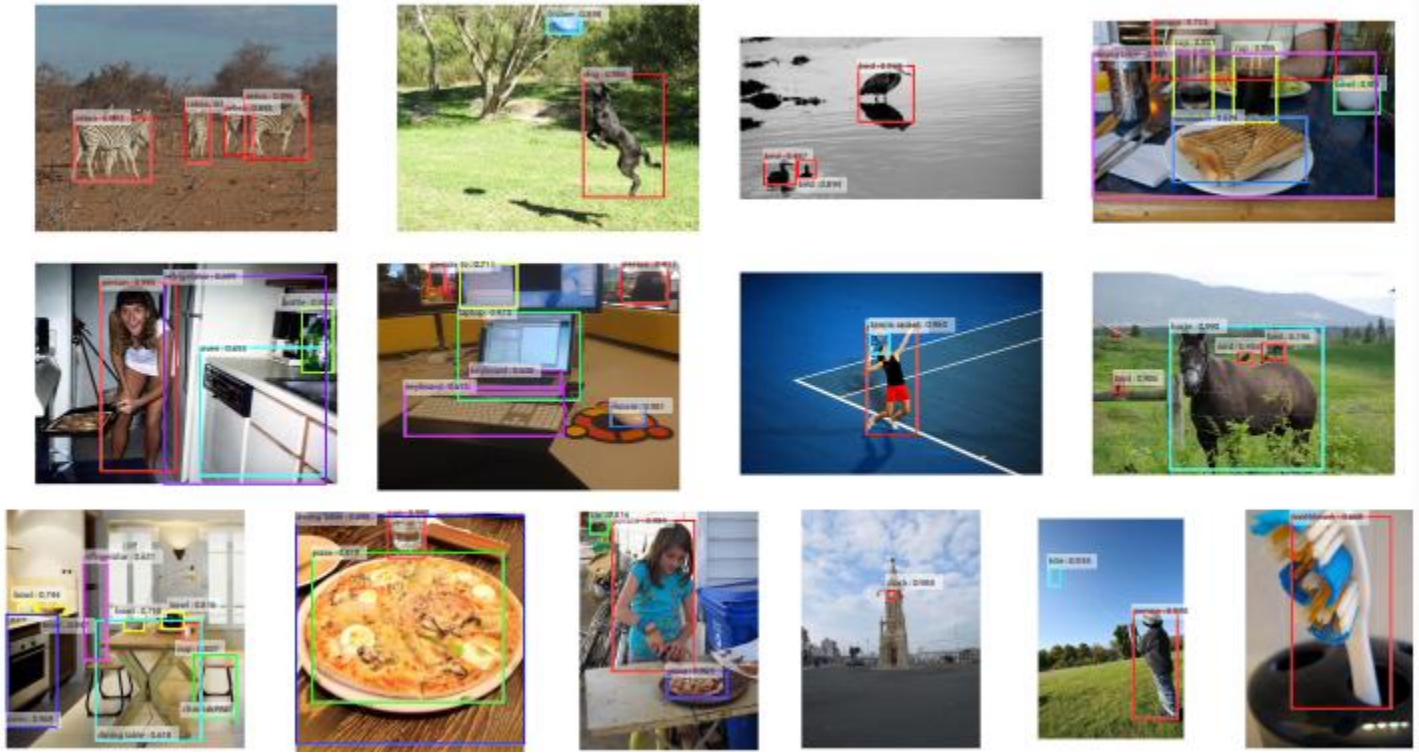


https://woochan-autobiography.tistory.com/921

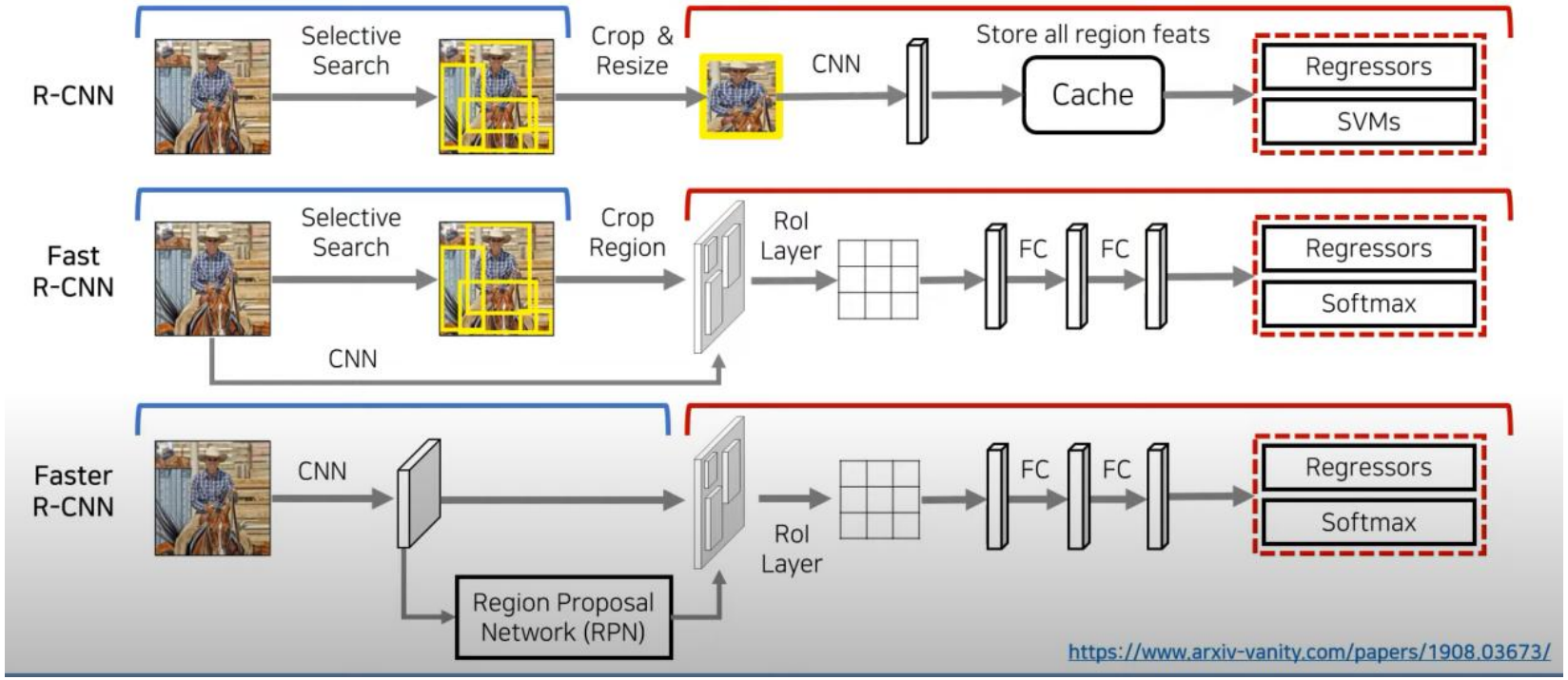
Experiments

- **Object detection on the MSSOCO dataset**





Summary on two stage detectors



https://wikidocs.net/148634

	Computation time [sec / image]	mAP (Pascal VOC 2007)
R-CNN	50	66.0
Fast R-CNN	2	66.9
Faster R-CNN	0.2	66.9

[CenterNet]

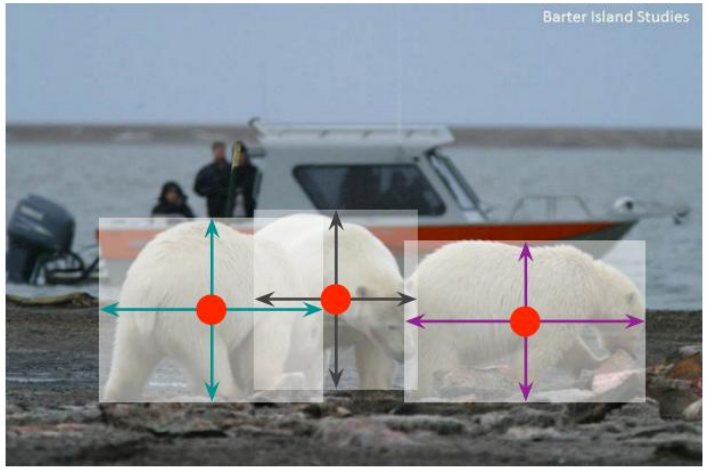
Objects as Points, 2019

- Why objects should be detected using bounding boxes first?

- **Motivation**
  - Traditional object detection approaches
    - Two-stage : Region proposal → Classification & regression
    - One-stage : Dense anchor-based prediction
  - Limitations
    - Large number of anchor boxes → Heavy computation
    - Extensive hyper-parameter tuning
    - Scale variation → Complex multi-level features (Feature Pyramid Network)

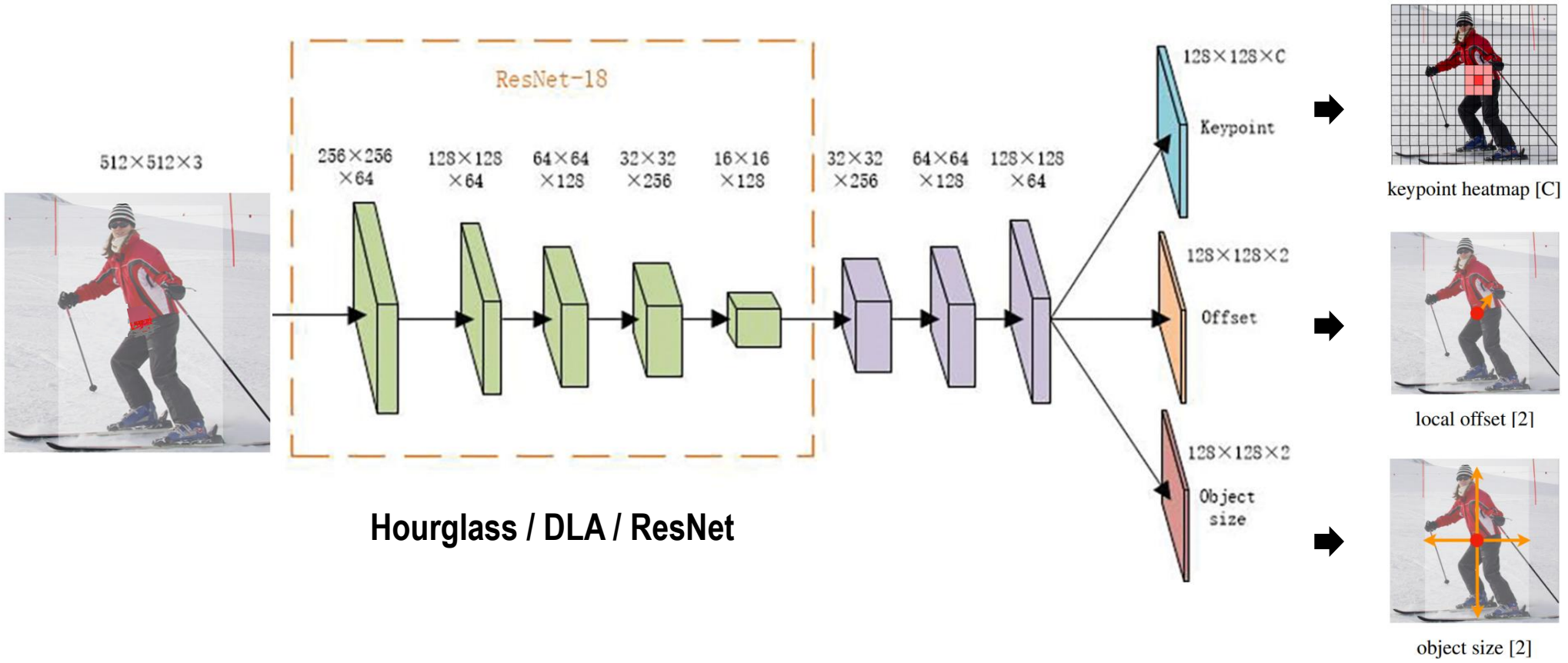
- **Goal**
  - Objects as points (i.e., anchor free object detection)
    - Represent each object by its center point
  - Predict
    - Center heatmap
    - Local offset
    - Object size (w, h)





Architecture

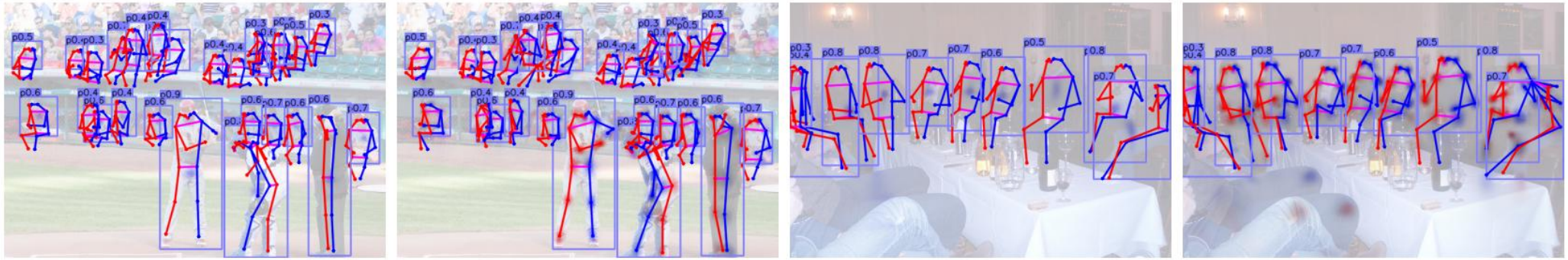
- **Keypoint heatmap**
  - Gaussian peak centered at object center → Modified Focal Loss (based on Gaussian kernel)
- **Local offset**
  - Corrects quantization error due to downsampling → L1 Loss
- **Object size**
  - Predicts width and height at the center → L1 Loss



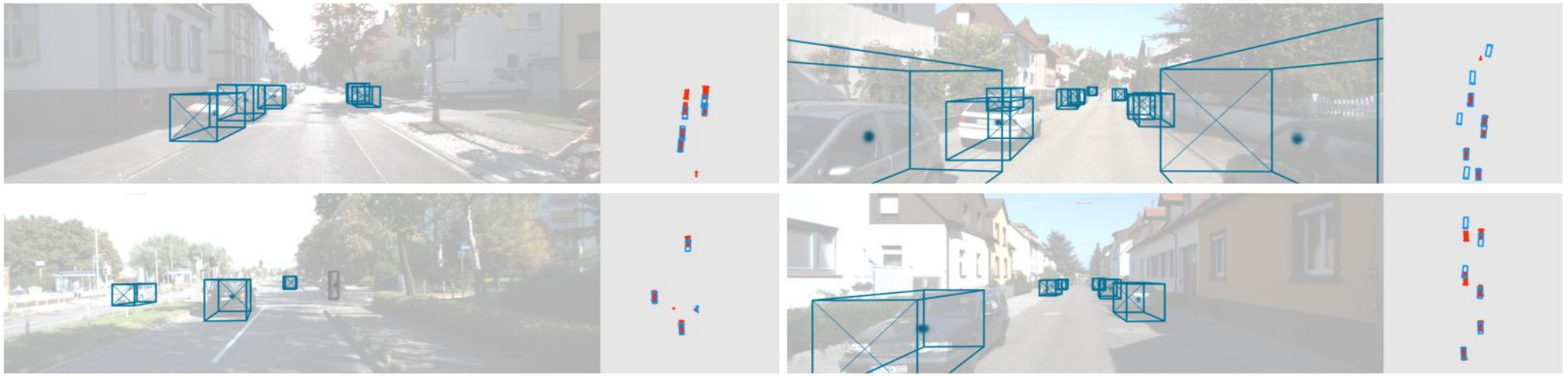
Applications (1/2)

Objects as Points, 2019

- Human pose estimation (on COCO validation)



- 3D bounding box estimation (on KITTI validation)



Applications (2/2)

- Cuboid detection

Experiments

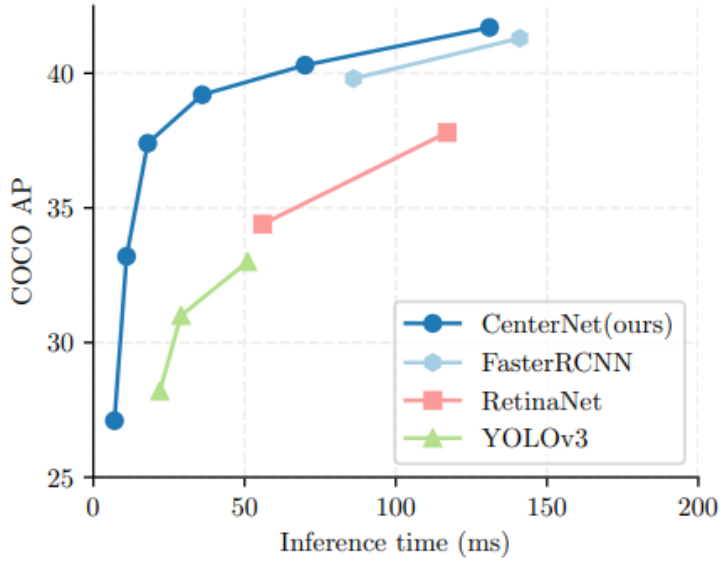


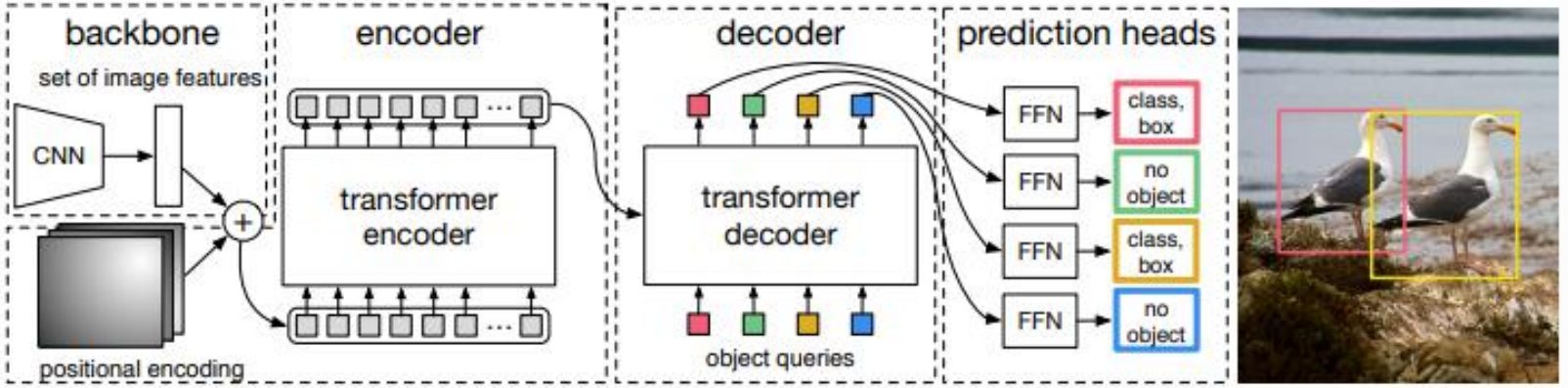
Figure 1: Speed-accuracy trade-off on COCO validation for real-time detectors. The proposed CenterNet outperforms a range of state-of-the-art algorithms.

	AP			AP_{50}			AP_{75}			Time (ms)			FPS		
	N.A.	F	MS	N.A.	F	MS	N.A.	F	MS	N.A.	F	MS	N.A.	F	MS
Hourglass-104	**40.3**	**42.2**	**45.1**	**59.1**	**61.1**	**63.5**	**44.0**	**46.0**	**49.3**	71	129	672	14	7.8	1.4
DLA-34	37.4	39.2	41.7	55.1	57.0	60.1	40.8	42.7	44.9	19	36	248	52	28	4
ResNet-101	34.6	36.2	39.3	53.0	54.8	58.5	36.9	38.7	42.0	22	40	259	45	25	4
ResNet-18	28.1	30.0	33.2	44.9	47.5	51.5	29.6	31.6	35.1	**7**	**14**	**81**	**142**	**71**	**12**

Table 1: Speed / accuracy trade off for different networks on COCO validation set. We show results without test augmentation (N.A.), flip testing (F), and multi-scale augmentation (MS).

[DETR]

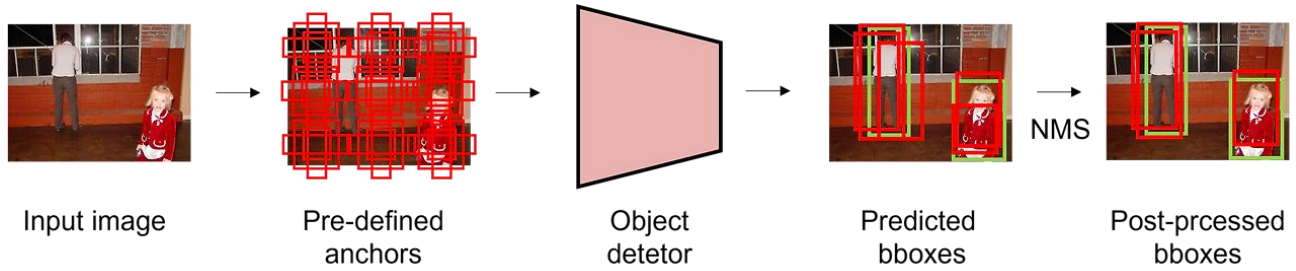
Abstract. We present a new method that views object detection as a direct set prediction problem. Our approach streamlines the detection pipeline, effectively removing the need for many hand-designed components like a non-maximum suppression procedure or anchor generation that explicitly encode our prior knowledge about the task. The main



- **Set** : a collection of unique, unordered elements
- **NMS (Non-Maximum Suppression)** : algorithm that removes duplicate bounding boxes for the same object and keeps only the one with the highest confidence score.

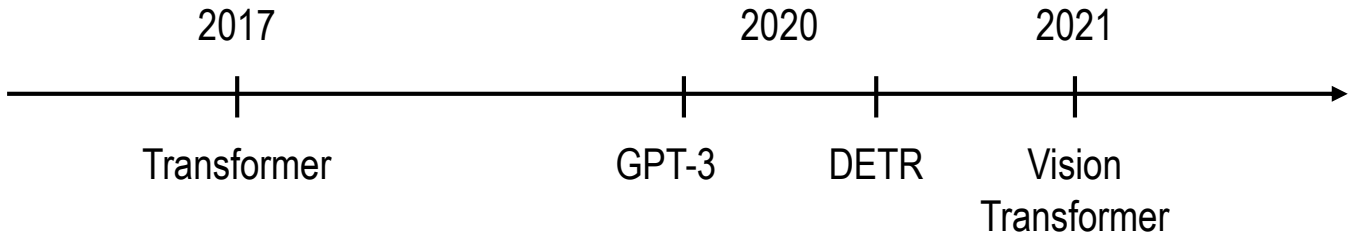
Overview

- **Main problems of conventional detectors**
  - Hand-crafted anchors
  - Heavy reliance on NMS (i.e., post-processing)
  - Complex multi-stage components (e.g., RPN, FPN)



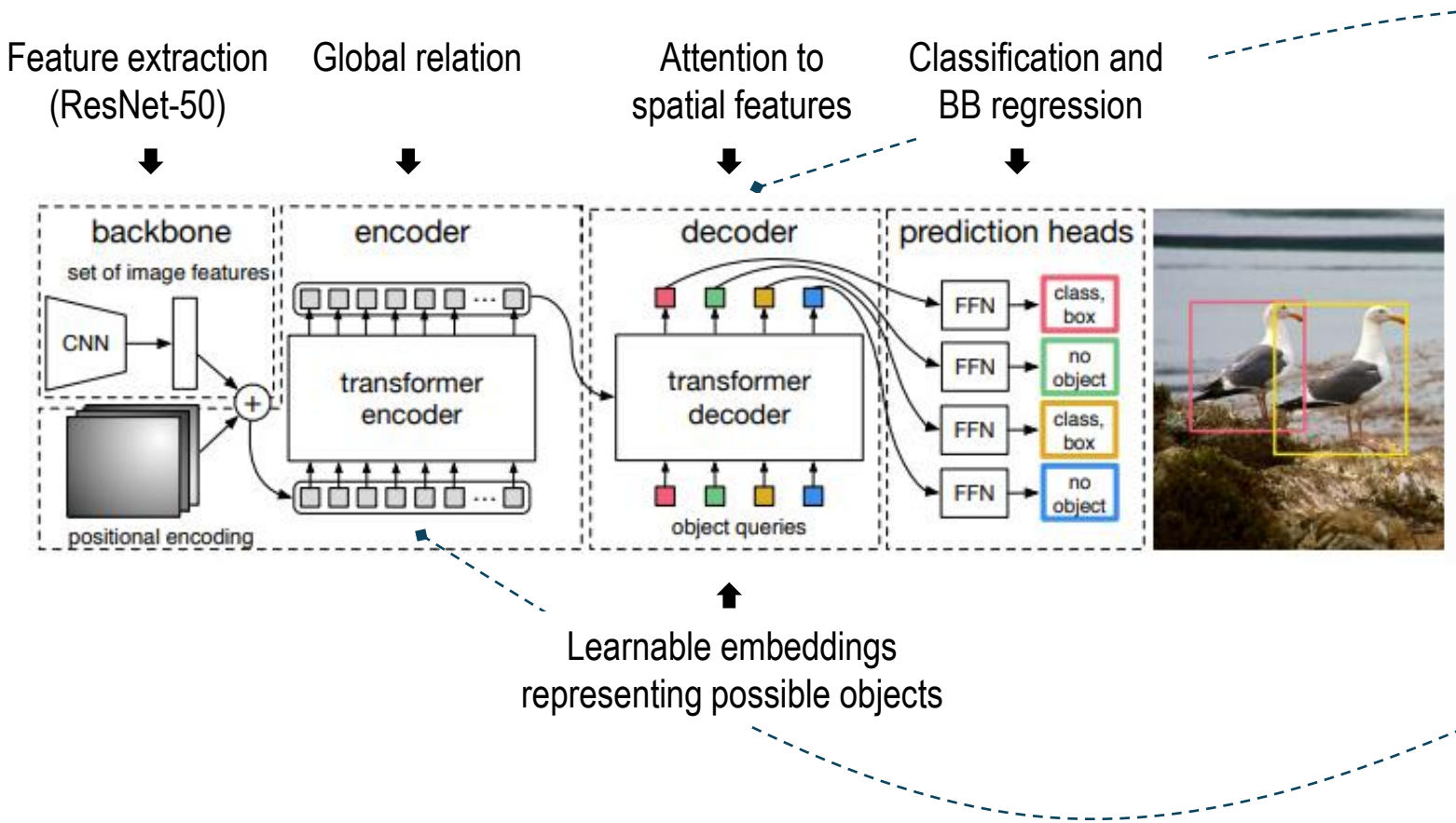
- **Huge success of the Transformer**
  - Capability of utilizing attention mechanism
  - Question:

    "Can we remove all handcrafted components and **make it truly end-to-end based on Transformer**?"



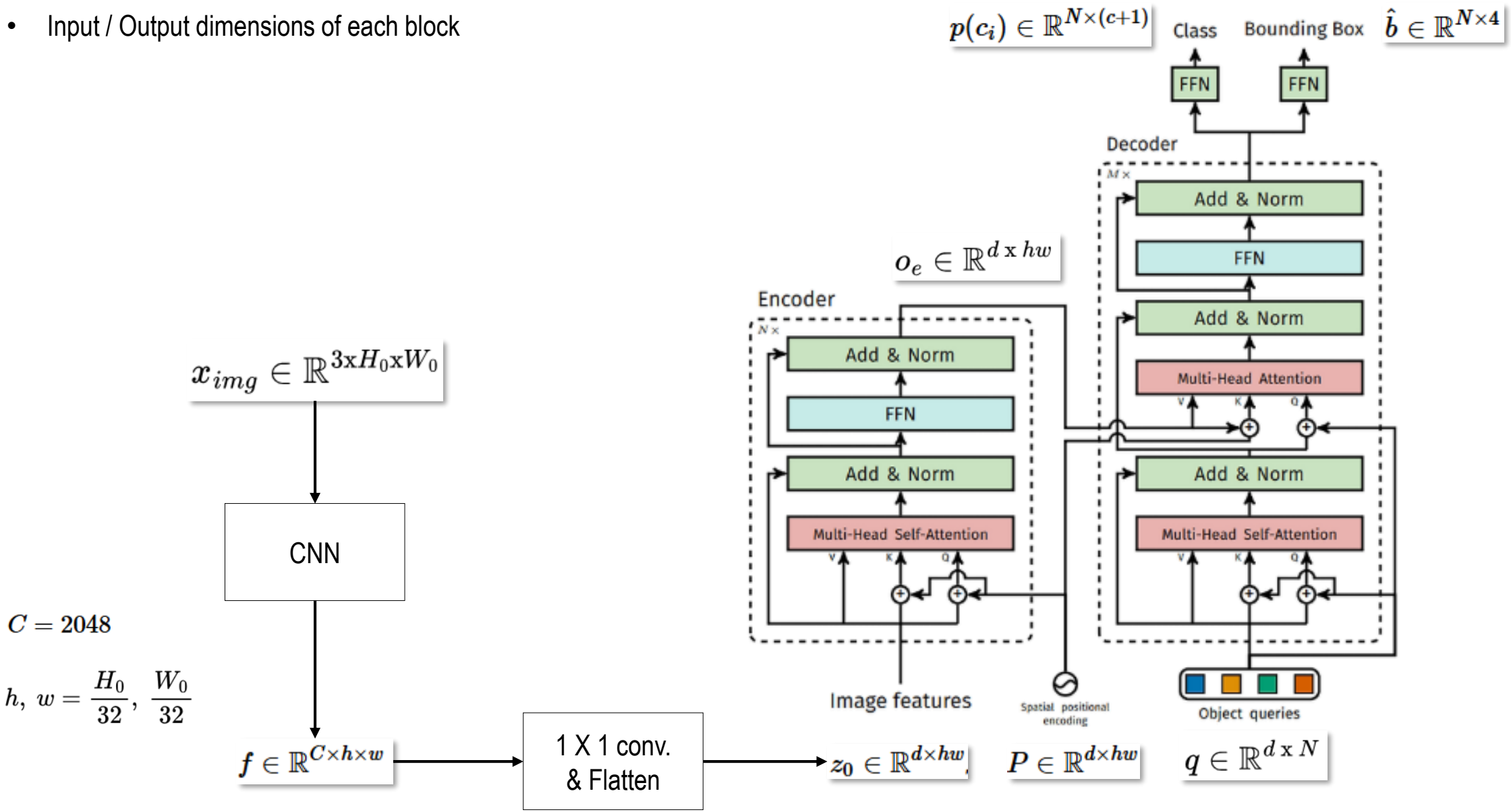
Architecture (1/2)

- **Core components**
  - Transformer Encoder-Decoder models global relations in image features
  - Set prediction with fixed output slots → one unique prediction per object
  - Bipartite matching → Hungarian algorithm ensures 1:1 optimal label prediction pairing



Architecture (2/2)

- Input / Output dimensions of each block

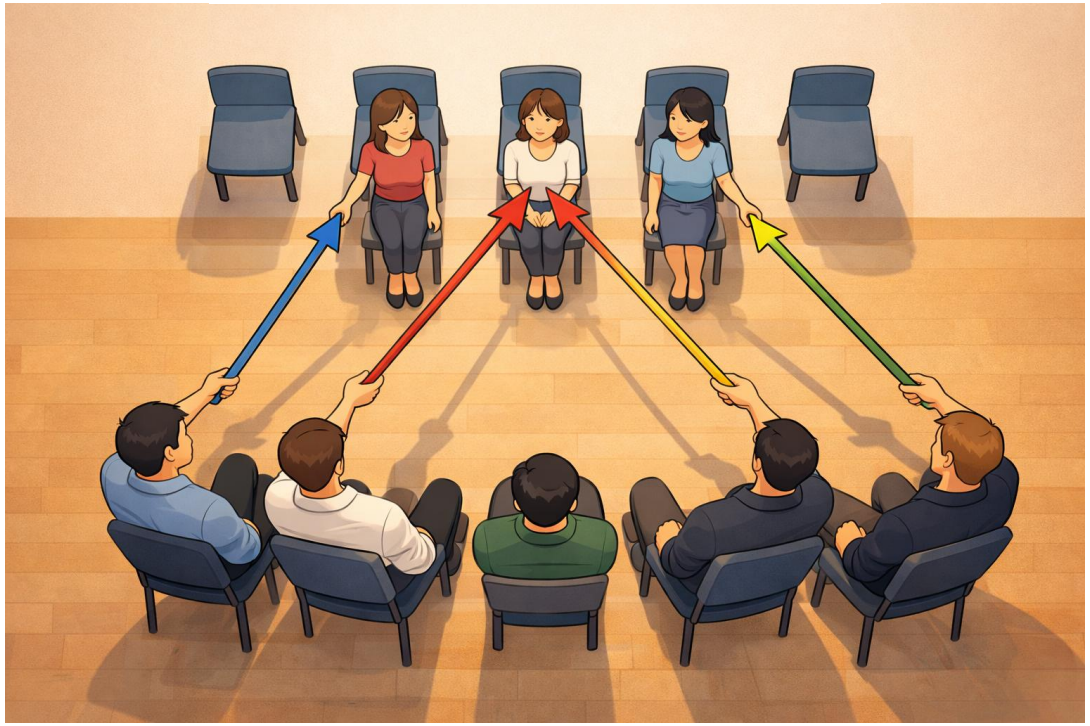


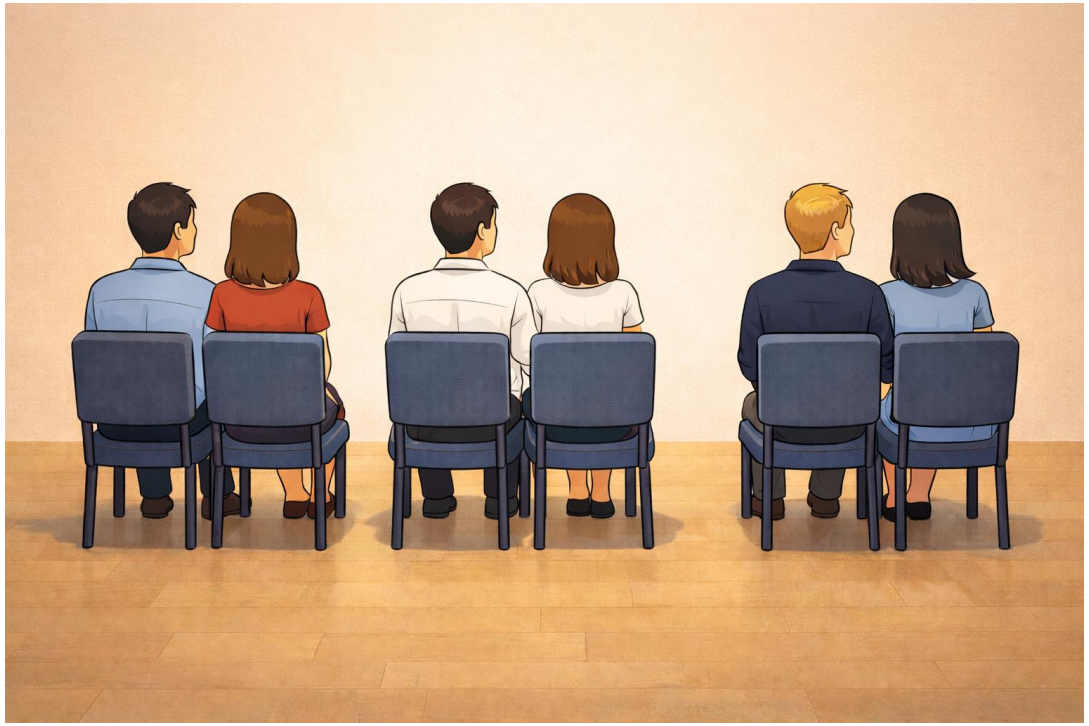
Loss (1/2)

Optimal matching

- Class matching
- Bounding box closeness

→ **Minimizing loss**





Loss (2/2)

- **Object detection set prediction loss**
  - Find optimal matching (i.e., bipartite matching → permutation)

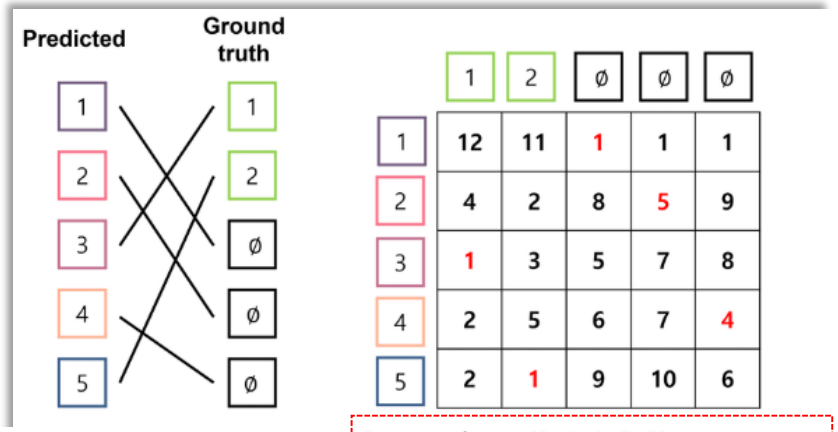
$$\hat{\sigma} = \arg\min_{\sigma \in \mathfrak{S}_N} \sum_i^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}), \quad \text{where} \quad \mathcal{L}_{match} = \underbrace{-\mathbb{1}_{\{c_i \neq \varnothing\}} \hat{p}_{\sigma(i)}(c_i)}_{①} + \mathbb{1}_{\{c_i \neq \varnothing\}} \mathcal{L}_{box}(b_i, \hat{b}_{\sigma(i)})$$

GIoU (Generalized IoU)

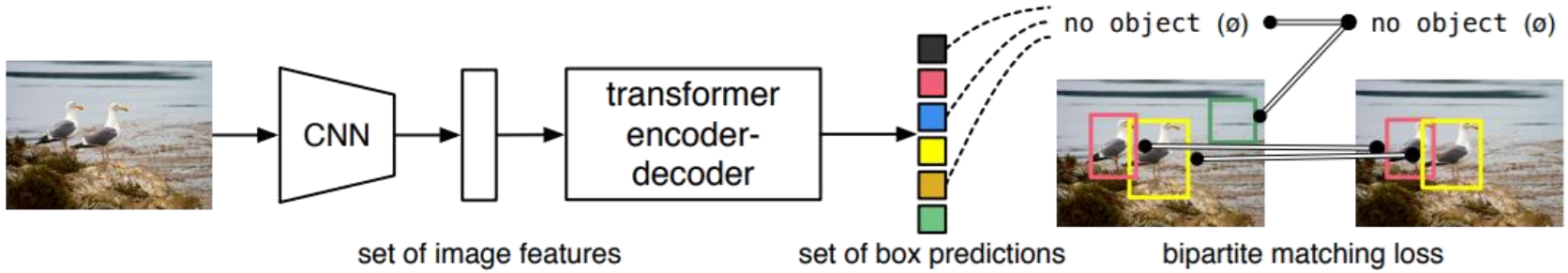
$$\lambda_{\text{iou}} \underbrace{\mathcal{L}_{\text{iou}}(b_i, \hat{b}_{\sigma(i)})}_{②} + \lambda_{\text{L1}} \underbrace{||b_i - \hat{b}_{\sigma(i)}||_1}_{③}$$

  - Compute Hungarian loss

$$\mathcal{L}_{\text{Hungarian}}(y, \hat{y}) = \sum_{i=1}^{N} \left[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{1}_{\{c_i \neq \varnothing\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}}(i)) \right]$$

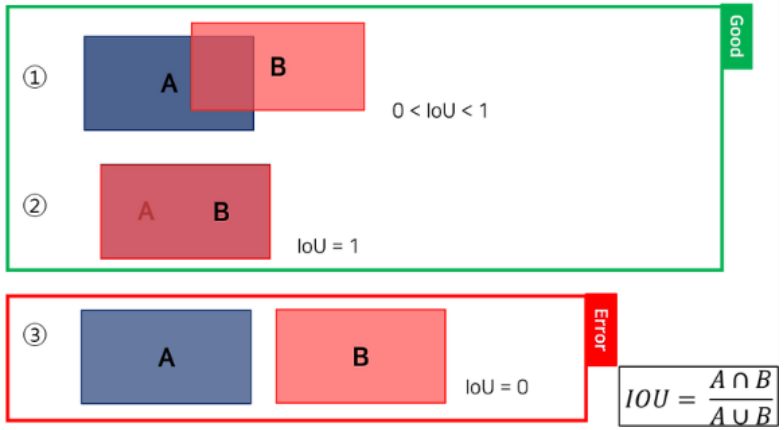


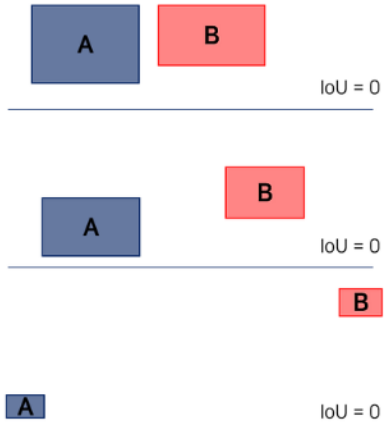
NMS is not needed !



IoU vs. GIoU

- **GIoU (Generalized IoU)**
  - Problems of IoU



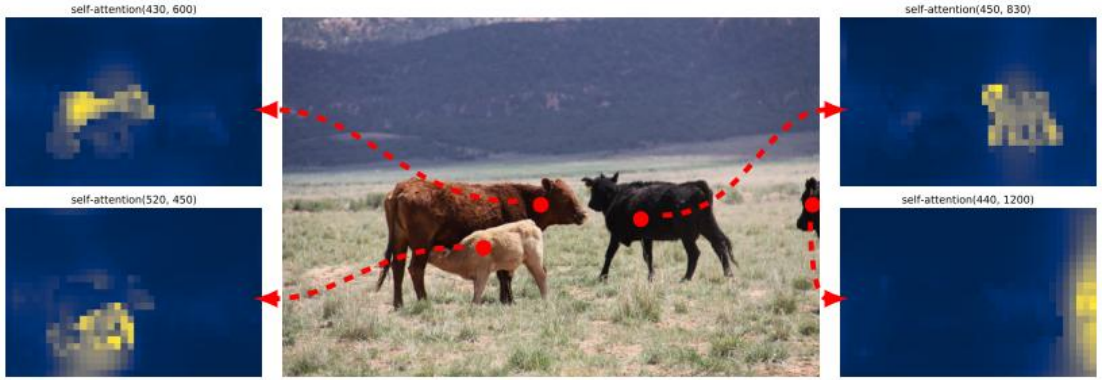


  - Loss in terms of GIoU
    - $GIoU = IoU - \frac{C \setminus (A \cup B)}{|C|}$
    - 

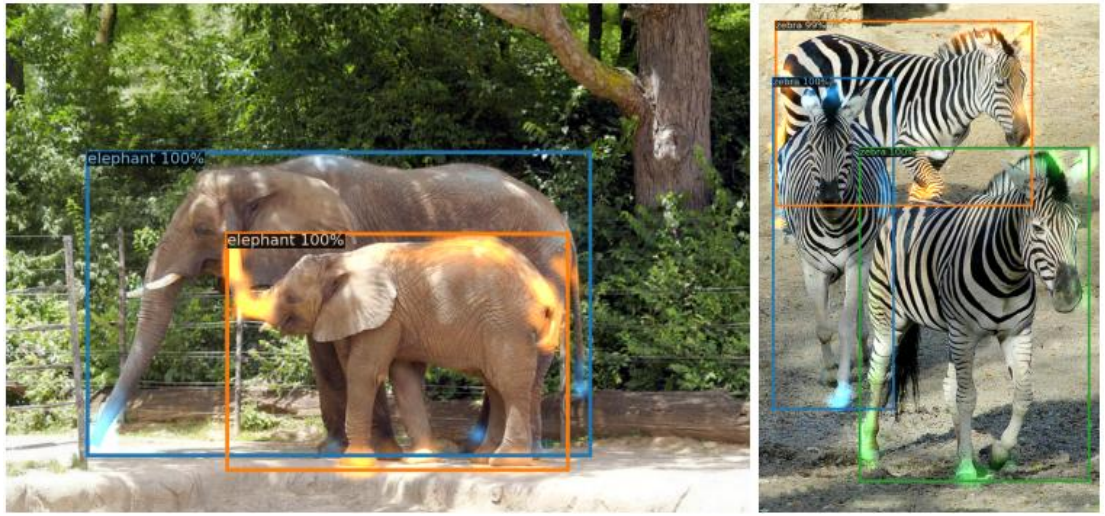
    - $-1 < GIoU < 1 \ (0 < IoU < 1)$
    - $Loss_{GIoU} = 1 - GIoU, \ 0 < Loss_{GIoU} < 2$

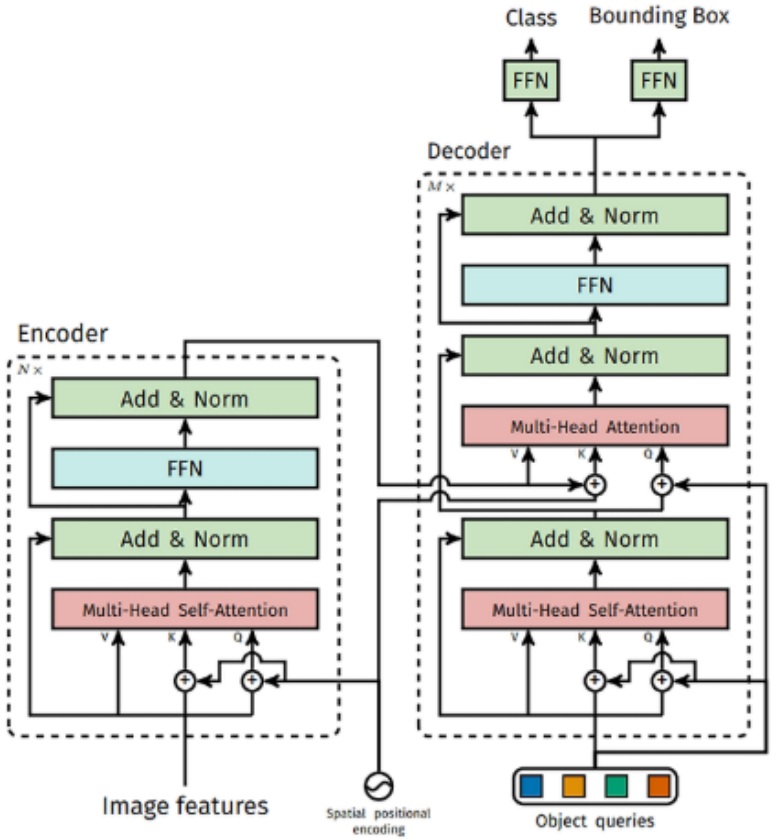
Experiment (1/2)

- **Encoder self-attention** for a set of reference points
  - Encoder can separate individual instances



- **Decoder attention** for every predicted object
  - Decoder typically attends to object extremities, such as legs and heads





Experiment (2/2)

- Panoptic segmentation

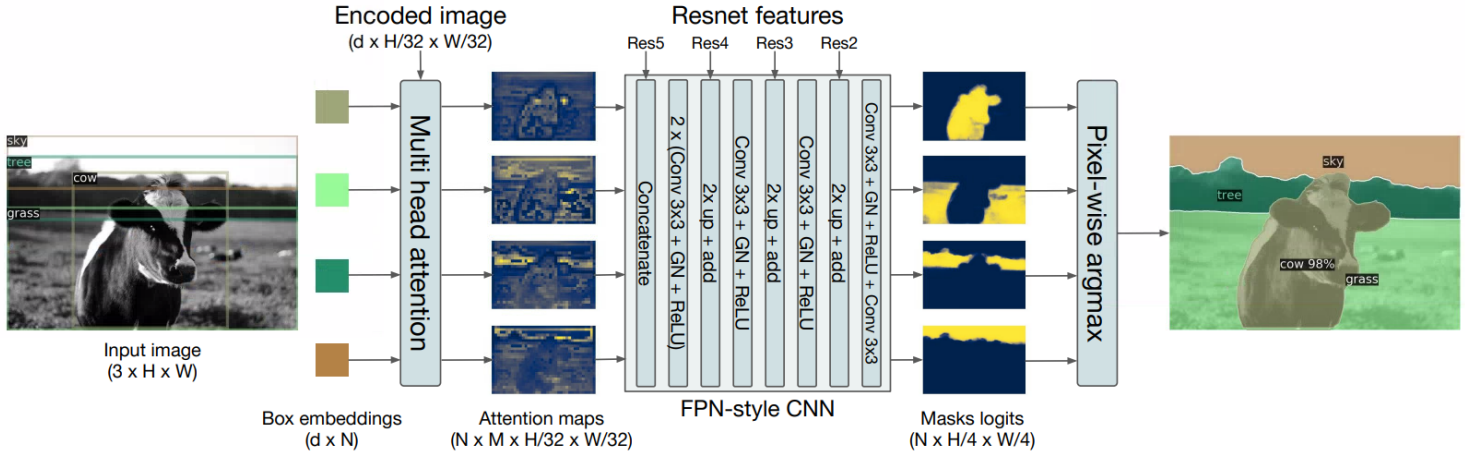


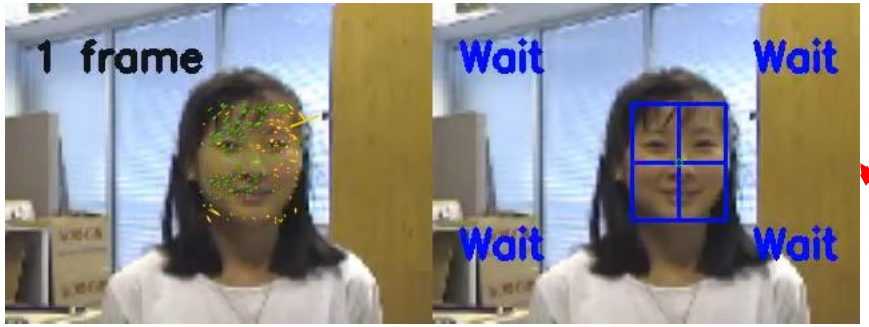
Fig. 8: Illustration of the panoptic head. A binary mask is generated in parallel for each detected object, then the masks are merged using pixel-wise argmax.



Fig. 9: Qualitative results for panoptic segmentation generated by DETR-R101. DETR produces aligned mask predictions in a unified manner for things and stuff.

Tracking

Challenges in tracking







- **Occlusion** – Hide and seek
  - When one object blocks another totally or partially
- **Disappear and reappear** – Unexpected revisit
  - Leaving the camera FOV for a moment, missed detection
- **Similar objects** – Twins, …, N-tuples
  - Soccer players, similar boxes
- **Fast motion** – Motion blur
  - Baseballs, cars
- **Scale variation** – Giant or dust
  - Zoom in and out
- **Camera motion** – Motion sickness
  - Pan-tilt CCTV, drone, smart phone camera
- **Noise and illumination change** – Ambiguous shape
  - Backlight, rain, fog
- **Shape shifting** – Abrupt appearance changes
  - Putting on a hat, opening an umbrella
- **Real-time constraints** – No time to think
  - Require 30~60 fps in applications

Tracking metric (1/3)

- **MOTA (Multiple Object Tracking Accuracy)**
  - Aggregates three types of tracking errors into a single score
    - FN : Missed detections
    - FP : False detections
    - IDsw : ID switches (i.e., incorrect identity assignments)
  - Limitations
    - Aggregates different error types into a single value
    - Can become negative for poor tracking

$$\text{MOTA} = 1 - \frac{\text{FN} + \text{FP} + \text{IDsw}}{\text{GT}}$$

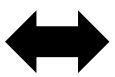
- **IDF1 (Identity F1-Score)**
  - Measures how well the tracker maintains consistent identities over time
    - IDTP: True Positive with correct identity
    - IDFP: False Positive identity assignment
    - IDFN: Missed identity (i.e., lost track)
  - Limitations
    - Less sensitive to bounding-box localization or detection errors
    - Focuses primarily on ID consistency

$$\text{IDF1} = \frac{2 \cdot \text{IDP} \cdot \text{IDR}}{\text{IDP} + \text{IDR}} = \frac{2 \cdot \text{IDTP}}{2 \cdot \text{IDTP} + \text{IDFP} + \text{IDFN}}$$

$$\text{IDP} = \frac{\text{IDTP}}{\text{IDTP} + \text{IDFP}}, \quad \text{IDR} = \frac{\text{IDTP}}{\text{IDTP} + \text{IDFN}}$$

Tracking metric (2/3)

- **HOTA (Higher Order Tracking Accuracy)**
  - Evaluates tracking performance in three balanced components
    - Detection Accuracy (DetA) – detection accuracy
      - F1-score
    - Association Accuracy (AssA) – identity consistency
      - AssA = Correctly associated detections / Total ground truth objects
    - Localization Accuracy (LocA) – bounding box precision
      - mean IoU or mAP
  - Limitations
    - Computation is more complex
    - Less intuitive than classical metrics



$$\mathrm{HOTA} = \sqrt{\mathrm{DetA} \times \mathrm{AssA}}$$

Tracking metric (2/3)

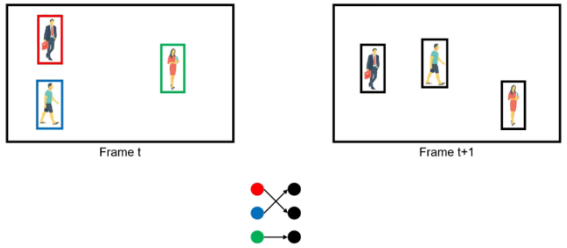
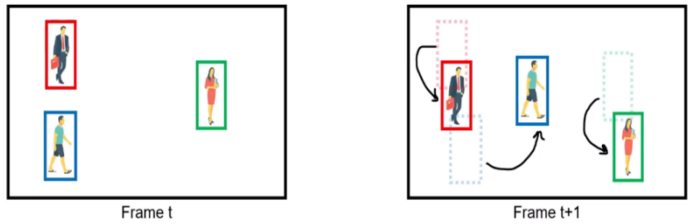
- Summary

	MOTA	IDF1	HOTA
What It Measures	Overall error rate (FN, FP, IDsw)	Identity consistency	Detection + Association + Localization
Strength	Simple, historical standard	Best ID metric for modern tracking	Most balanced & insightful
Weakness	Can be negative; weak identity evaluation	Ignores detection localization quality	More complex to compute
Formula	$1 - \frac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDsw}}{\mathrm{GT}}$	$\frac{2 \cdot \mathrm{IDTP}}{2 \cdot \mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}$	$\sqrt{\mathrm{DetA} \times \mathrm{AssA}}$

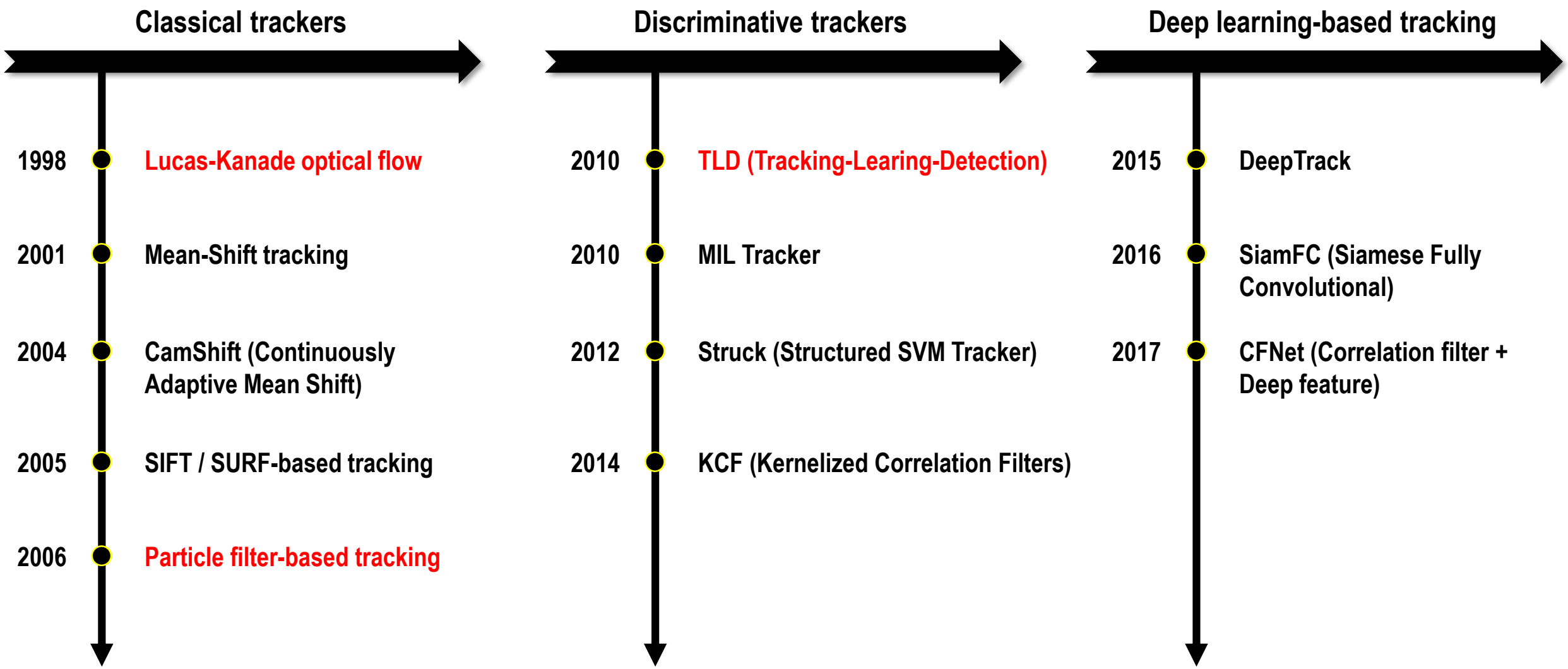
Tracking methodology

- **MOT (Multiple Object Tracking)**
  - Aims to detect and track multiple objects over time in video streams
- **Two major methodologies**

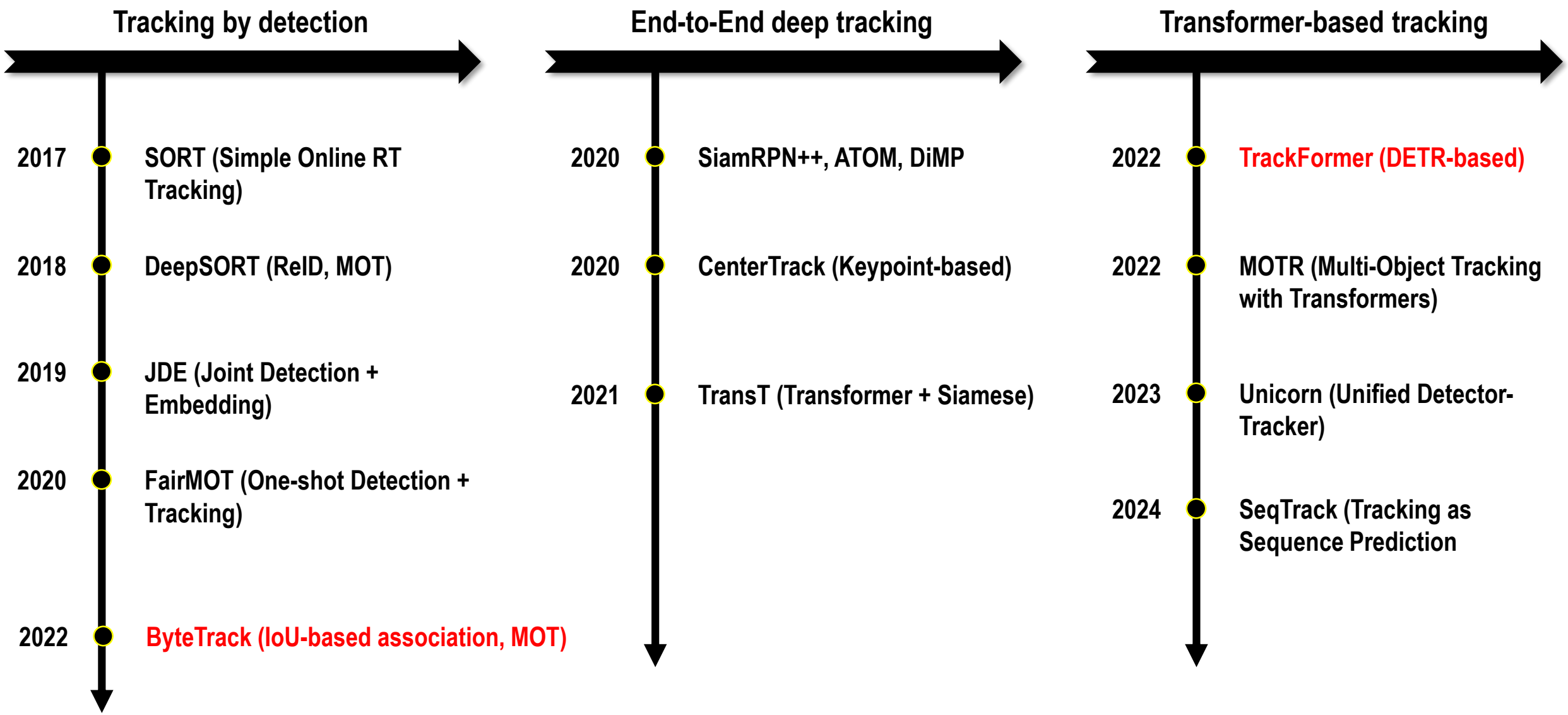
https://www.youtube.com/watch?v=q1OaqRj-Pmo&t=246s

Feature	Tracking-by-Detection	Tracking-by-Regression
Architecture	2-stage 	1-stage 
Workflow	Detector + Association (First detect, then associate)	Regression Network (From previous box, then regress next box)
Identity maintenance	Matching across frames	Propagation over frames
Influence of detection error	Catastrophic	Less critical
Occlusion recovery	Re-identification	Prediction
Limitations	• Association failures happen when - occlusion - appearance changes - missing detection	• Accumulate drift over time • Hard to handle appearance change suddenly • Limited by regression model's motion prior

Evolution of object tracking (1/2)



Evolution of object tracking (2/2)

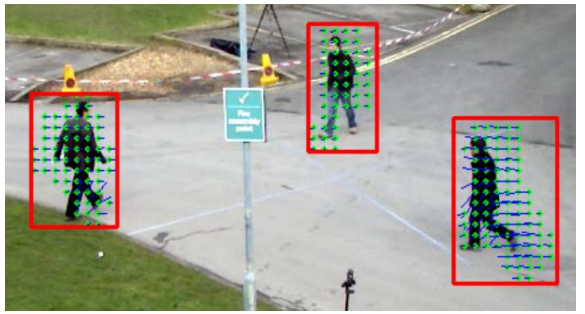


Lucas-Kanade optical flow

- What is the **optical flow**?
  - Apparent motion of pixels between two consecutive frames
- Fundamental **assumptions**
  - Brightness constancy
    $I(x, y, t) = I(x+u, y+v, t+1)$, where I denotes image array.
  - Small motion
    Motion between consecutive frames must be small enough to allow a first-order Taylor expansion.
  - Spatial Coherence
    Pixels within a local window share the same motion vector (u, v)
- Optical flow **constraint equation**
  - First-order Taylor Expansion
    $I(x+u, y+v, t+1) \approx I + I_x u + I_y v + I_t = I(x, y, t)$, where I_x indicates a partial derivative about x.
- **Lucas-Kanade formulation**
  - For n points within an window,

$$\begin{bmatrix} I_x(p_1) \\ I_x(p_2) \\ \vdots \\ I_x(p_n) \end{bmatrix} u + \begin{bmatrix} I_y(p_1) \\ I_y(p_2) \\ \vdots \\ I_y(p_n) \end{bmatrix} v = - \begin{bmatrix} I_t(p_1) \\ I_t(p_2) \\ \vdots \\ I_t(p_n) \end{bmatrix} \Rightarrow A \cdot \begin{bmatrix} u \\ v \end{bmatrix} = -b \Rightarrow (A^T A) \begin{bmatrix} u \\ v \end{bmatrix} = -A^T b \Rightarrow \begin{bmatrix} u \\ v \end{bmatrix} = -(A^T A)^{-1} A^T b$$

- **Limitations**
  - Fails under large motion (needs pyramids)
  - Sensitive to illumination changes
  - Cannot handle textureless regions
  - Struggles with occlusion



https://gaussian37.github.io/vision-concept-optical_flow/

Particle filter

- **What is the Particle filter**?
  - Sequential Monte-Carlo method that estimates the posterior distribution of a state using a set of random samples (particles)
  - Used when the system is **non-linear** and **non-Gaussian**, where Kalman filters fail
- **Bayesian filtering**
  - Prediction : $p(x_t|z_{1:t-1}) = \int p(x_t|x_{t-1})\, p(x_{t-1}|z_{1:t-1})\, dx$
  - Update : $p(x_t|z_{1:t}) \propto p(z_t|x_t)\, p(x_t|z_{1:t-1})$

x : state,
z : measurement

- **Core algorithm**

Step 1: Initialization

Generate N particles $x_t^{(i)}$ from prior distribution.

Step 2: Prediction

Use system model:

$$x_t^{(i)} \sim p(x_t|x_{t-1}^{(i)})$$

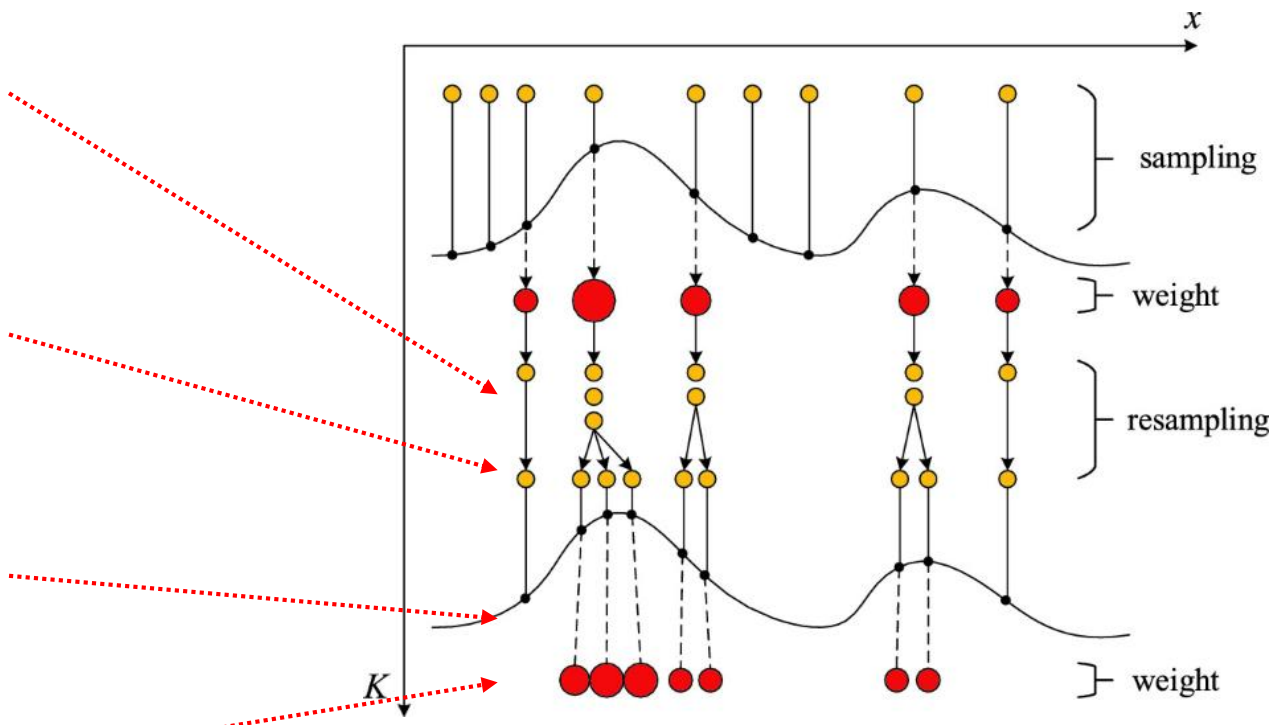
Step 3: Weighting

Based on measurement likelihood:

$$w_t^{(i)} = p(z_t|x_t^{(i)})$$

Step 4: Normalization

$$\tilde{w}_t^{(i)} = \frac{w_t^{(i)}}{\sum_j w_t^{(j)}}$$



https://link.springer.com/chapter/10.1007/978-3-031-04245-4_39

[TLD]

- **Goal** : Enabling long-term, robust object tracking by combining tracking, learning, and detection in a unified framework
  - Track locally and efficiently
  - Learn new appearances online
  - Detect the object globally when tracking fails

- **Tracking**
  - Short-term, frame-to-frame object tracking
  - Uses "Median Flow," a robust optical-flow-based tracker
  - Fast and accurate for small displacements
- **Detection**
  - Sliding-window detector over the entire image
  - Uses an ensemble of random fern classifiers
  - Re-detects the target when the tracker fails or drifts
- **Learning**
  - Updates the detector online
  - Generates positive and negative training samples
  - Suppresses false positives and adapts to new object appearances

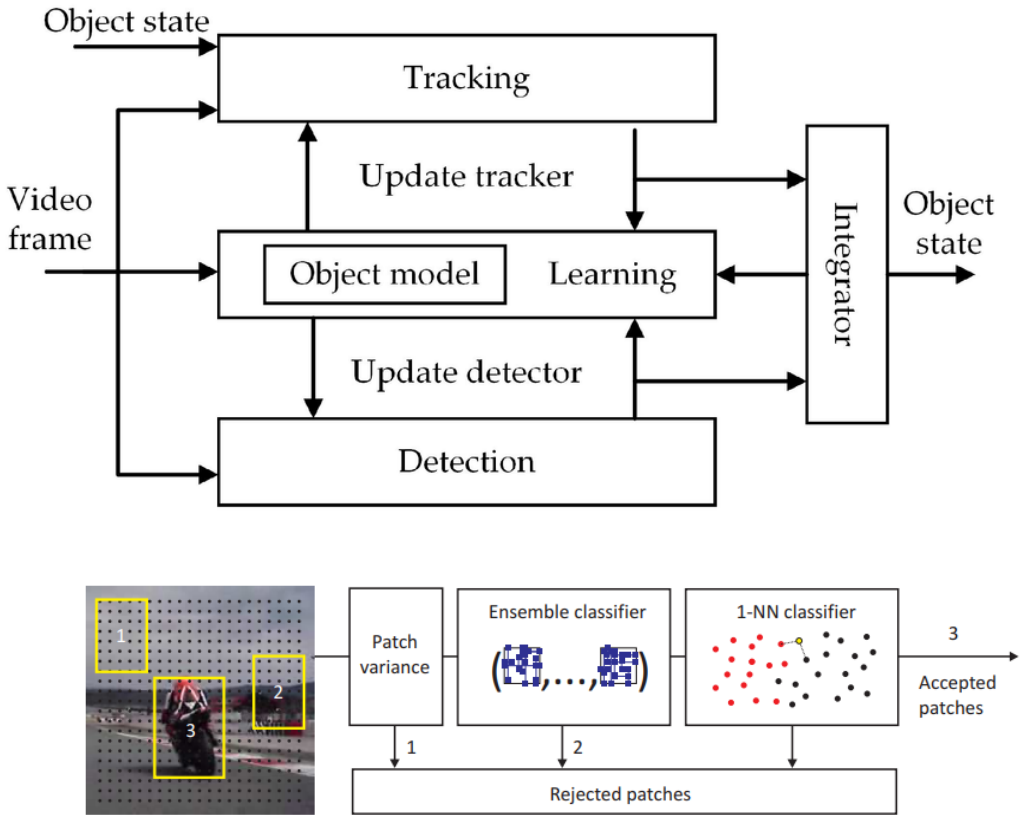


Fig. 9. Block diagram of the object detector.

- Demo video : https://youtu.be/1GhNXHCQGsM

[ByteTrack]

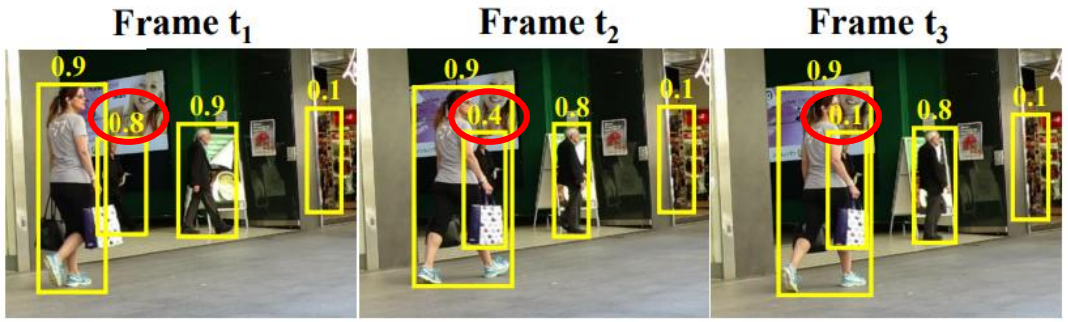
ByteTrack: Multi-Object Tracking by Association Every Detection Box, ECCV 2022

- **Motivation**
  - Limitations of the traditional MOT frameworks following Tracking by Detection
    - Reliance on high-confidence detections only
    - Low confidence detections are often discarded
  - As a result,
    - Occlusion, motion blur, far-distance objects → low detection confidence
    - Causes fragmented tracks & ID switches

- **Key idea —** Two-level association
  - Stage 1 : High-score detection association
    - IoU-based matching between existing tracks and high-confidence detections
      → Create stable identity continuity
  - Stage 2 : Low-score detection association
    - Remaining active tracks try to match low-confidence detections
      → Rescue objects during occlusion or distance blur

" Low confidence detections are **not noise**, they are **temporal evidence**. "

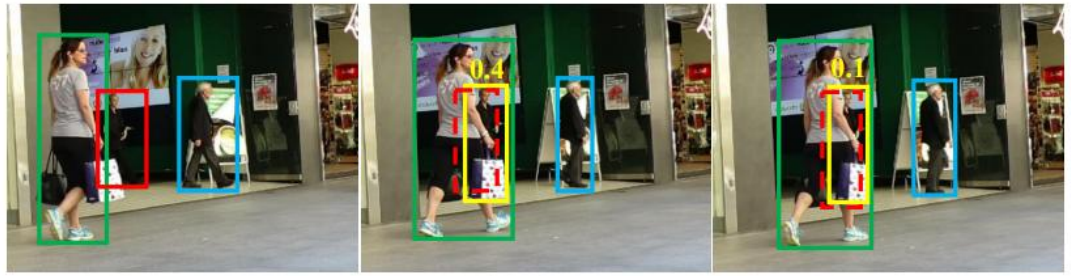
Overall algorithm



(a) detection boxes



(b) tracklets by associating high score detection boxes



(c) tracklets by associating every detection box

Algorithm 1: Pseudo-code of BYTE.

Input: A video sequence V; object detector Det; detection score threshold τ
Output: Tracks $\mathcal{T}$ of the video

Initialization: $\mathcal{T} \leftarrow \emptyset$
for *frame* f_k *in* V **do**
  /* Figure 2(a) */
  /* predict detection boxes & scores */
  $\mathcal{D}_k \leftarrow \text{Det}(f_k)$
  $\mathcal{D}_{high} \leftarrow \emptyset$
  $\mathcal{D}_{low} \leftarrow \emptyset$
  for d *in* $\mathcal{D}_k$ **do**
    if $d.score > \tau$ **then**
      $\mathcal{D}_{high} \leftarrow \mathcal{D}_{high} \cup \{d\}$
    end
    else
      $\mathcal{D}_{low} \leftarrow \mathcal{D}_{low} \cup \{d\}$
    end
  end
  /* predict new locations of tracks */
  for t *in* $\mathcal{T}$ **do**
    $t \leftarrow \text{KalmanFilter}(t)$
  end
  /* Figure 2(b) */
  /* first association */
  Associate $\mathcal{T}$ and $\mathcal{D}_{high}$ using Similarity#1
  $\mathcal{D}_{remain} \leftarrow$ remaining object boxes from $\mathcal{D}_{high}$
  $\mathcal{T}_{remain} \leftarrow$ remaining tracks from $\mathcal{T}$
  /* Figure 2(c) */
  /* second association */
  Associate $\mathcal{T}_{remain}$ and $\mathcal{D}_{low}$ using similarity#2
  $\mathcal{T}_{re-remain} \leftarrow$ remaining tracks from $\mathcal{T}_{remain}$
  /* delete unmatched tracks */
  $\mathcal{T} \leftarrow \mathcal{T} \setminus \mathcal{T}_{re-remain}$
  /* initialize new tracks */
  for d *in* $\mathcal{D}_{remain}$ **do**
    $\mathcal{T} \leftarrow \mathcal{T} \cup \{d\}$
  end
end
Return: $\mathcal{T}$

Algorithm (1/4)

- **Detection + score-based split**
- **Track position predictions**

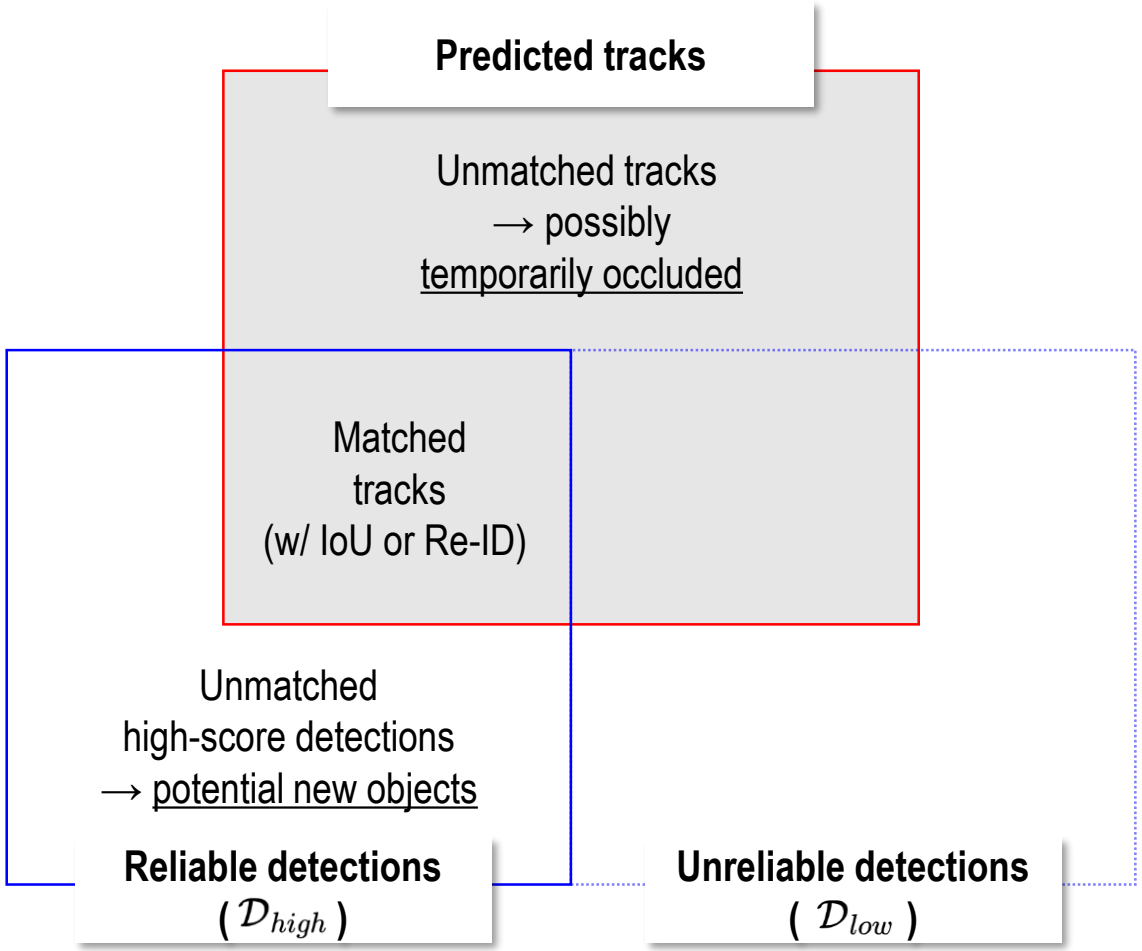
- Run detector and generate
  - Detection boxes
  - Confidence scores
- Initialize
  - High-score detection set
  - Low-score detection set
- For every detection, split it by confidence score
  - Higher than threshold
  - Lower than threshold
- Track position predictions
  - Predict track locations at the current frame by motion dynamics, i.e., Kalman filter

```
/* Figure 2(a) */
/* predict detection boxes & scores */
D_k ← Det(f_k)
D_high ← ∅
D_low ← ∅
for d in D_k do
    if d.score > τ then
        D_high ← D_high ∪ {d}
    end
    else
        D_low ← D_low ∪ {d}
    end
end

/* predict new locations of tracks */
for t in T do
    t ← KalmanFilter(t)
end
```

Algorithm (2/4)

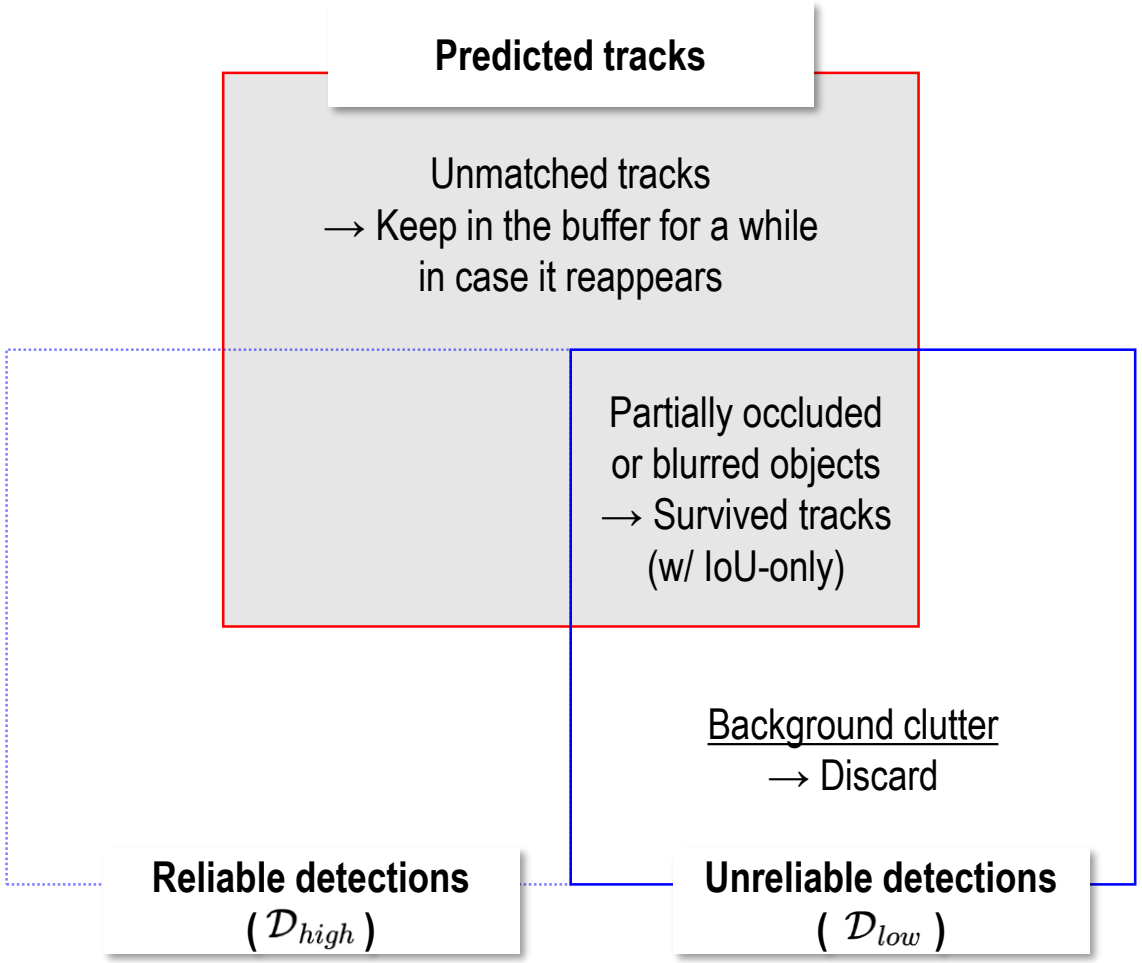
- **First association**
  - Match tracks with high score detections




```
/* Figure 2(b) */
/* first association */
```
Associate $\mathcal{T}$ and $\mathcal{D}_{high}$ using `Similarity#1`
$\mathcal{D}_{remain} \leftarrow$ remaining object boxes from $\mathcal{D}_{high}$
$\mathcal{T}_{remain} \leftarrow$ remaining tracks from $\mathcal{T}$

Algorithm (3/4)

- **Second association**
  - Save tracks using low score detections



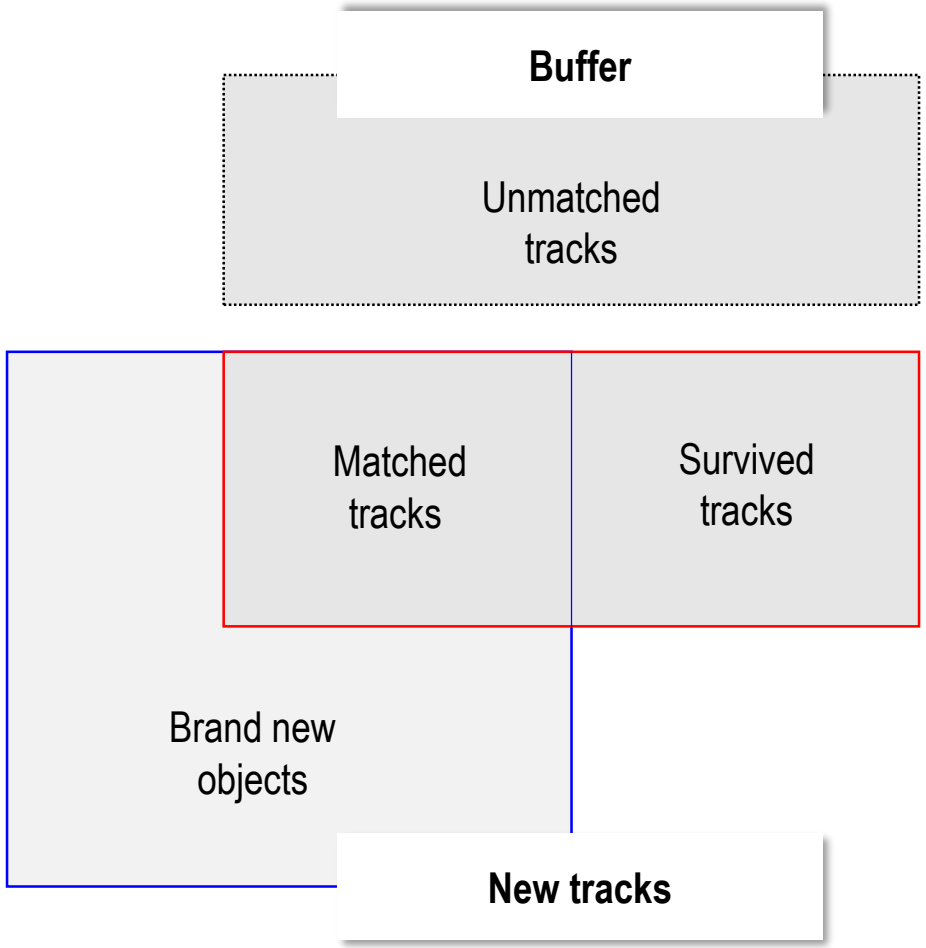

```
/* Figure 2(c) */
/* second association */
Associate T_remain and D_low using similarity#2
T_re-remain ← remaining tracks from T_remain

/* delete unmatched tracks */
T ← T \ T_re-remain

/* initialize new tracks */
for d in D_remain do
    T ← T ∪ {d}
end
```

Algorithm (4/4)

- **Add new objects into track**
- **Keep tracks that still cannot be matched**




```
/* Figure 2(c) */
/* second association */
Associate T_remain and D_low using similarity#2
T_re-remain ← remaining tracks from T_remain

/* delete unmatched tracks */
T ← T \ T_re-remain

/* initialize new tracks */
for d in D_remain do
  | T ← T ∪ {d}
end
```

Experiments

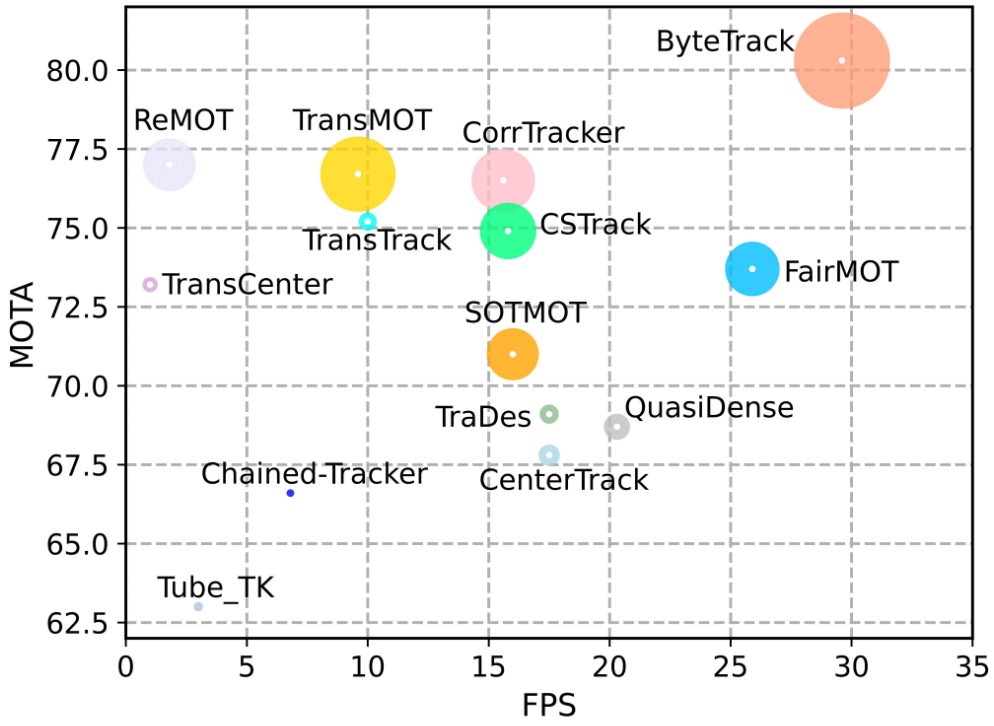
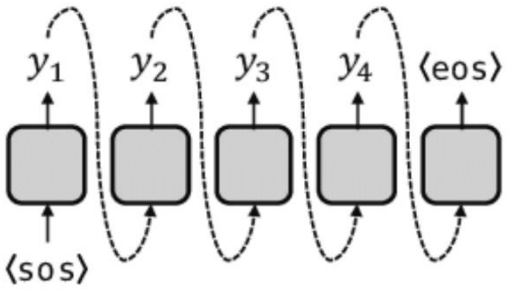


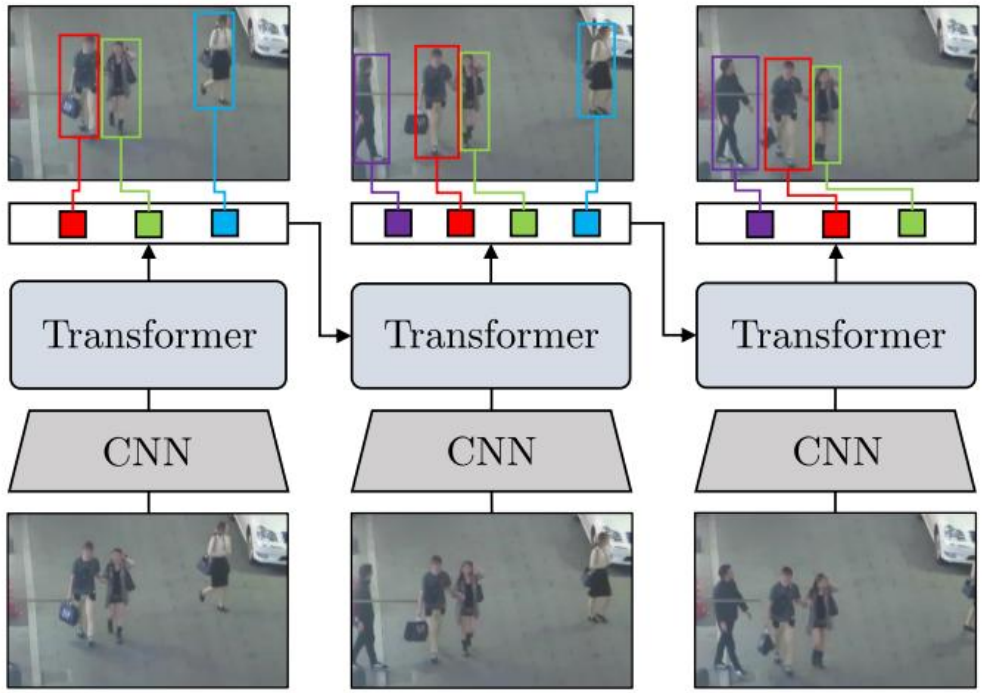
Figure 1. MOTA-IDF1-FPS comparisons of different trackers on the test set of MOT17. The horizontal axis is FPS (running speed), the vertical axis is MOTA, and the radius of circle is IDF1. Our ByteTrack achieves 80.3 MOTA, 77.3 IDF1 on MOT17 test set with 30 FPS running speed, outperforming all previous trackers. Details are given in Table 4.

[TrackFormer]

- **Motivation**
  - Tracking-by-detection decouples detection and identity assignment
  - Hand-crafted association rules (IoU, motion model)
  - Identity switches during heavy occlusion
- **Key idea**
  - Tracking into a sequence prediction task based on Transformer (i.e., Autoregressive manner)
    → Similar to DETR but extended to time dimension
  - End-to-end joint detection and tracking-by-attention



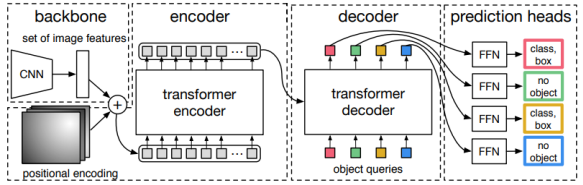
Autoregressive schema

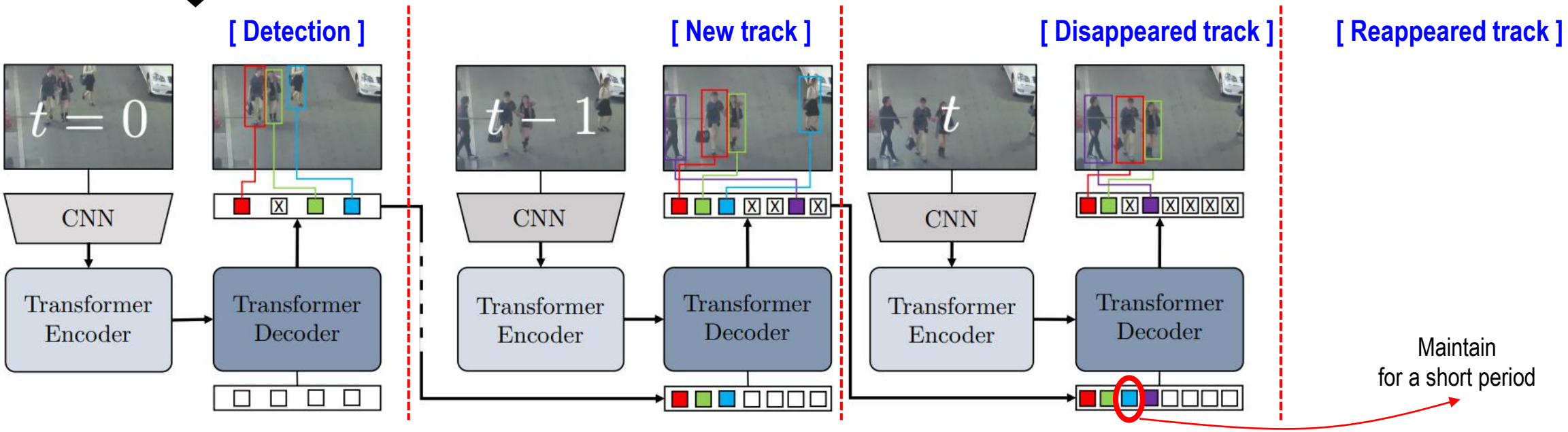


Architecture (1/2)

- TrackFormer is built on DETR (DEtection TRansformer)
- Transformer Encoder → Feature aggregation
- Transformer Decoder → Query-based object prediction
  - Detection queries : Find new objects in the frame
  - Track queries : Track previously detected objects

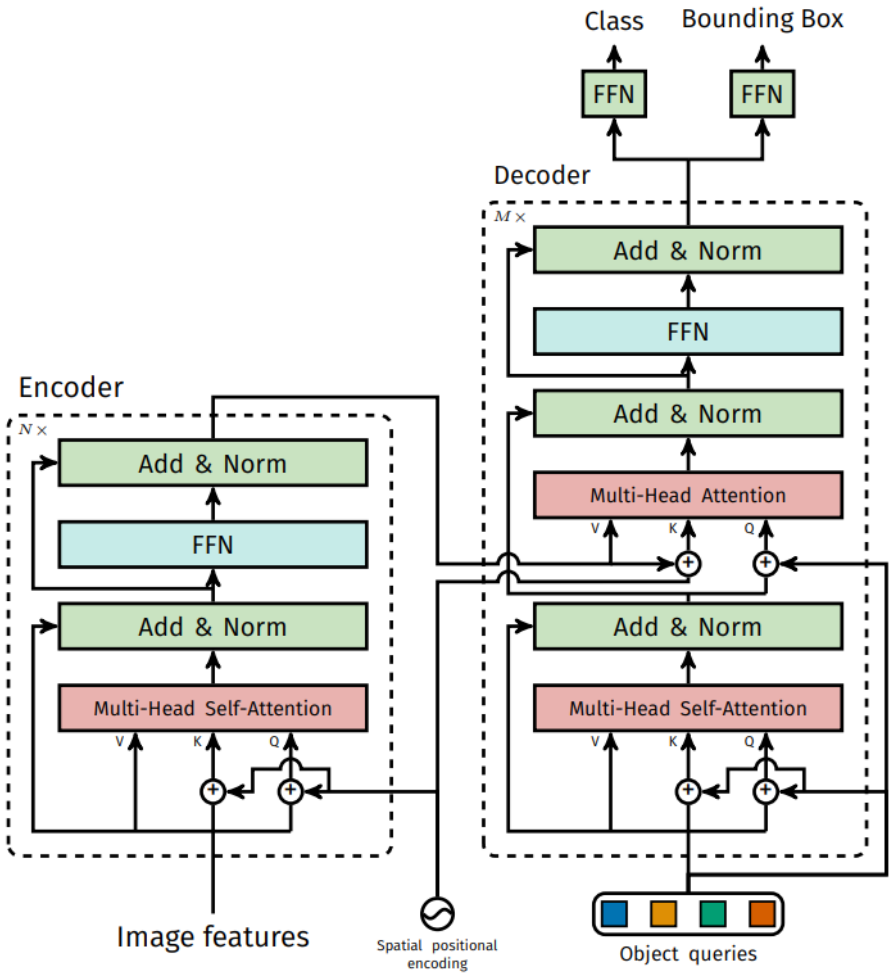
- Track queries → Persistent query embedding
  - Represents object identity
  - Attends to next-frame features through attention
  - Updates its location (bounding box) and confidence
  - Disappears if track is lost



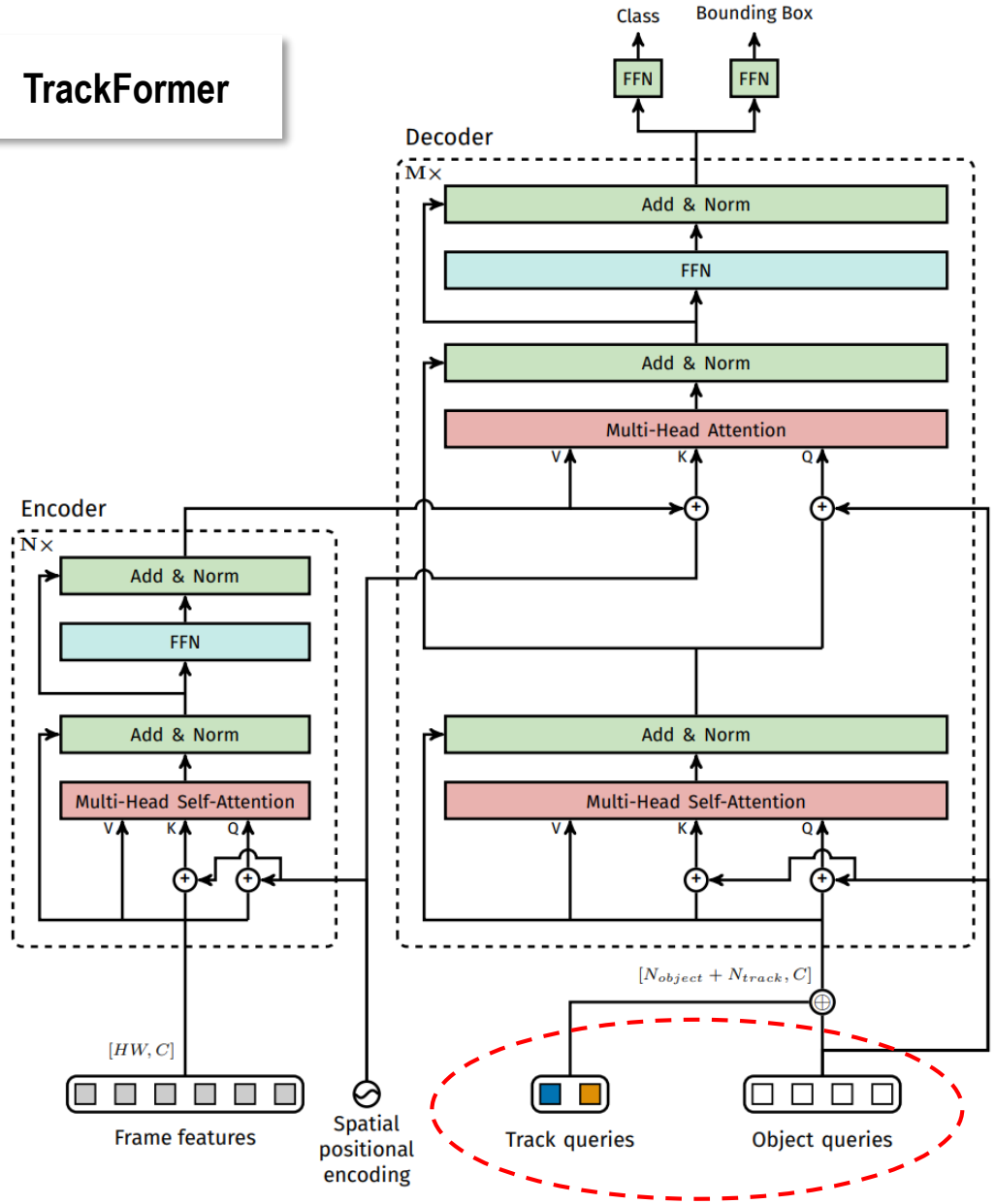


Architecture (2/2)

DETR



TrackFormer



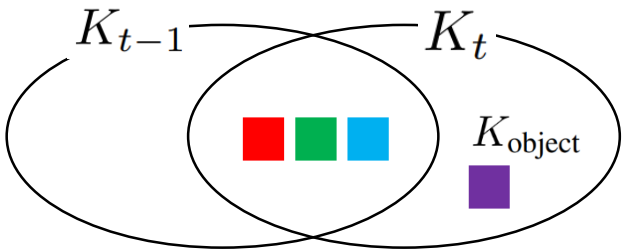
Training (1/3)

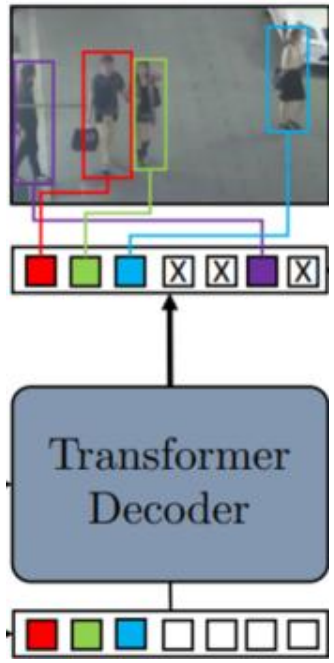
- **Bipartite matching**
  - Find the mapping from ground truth objects y_i to the joint set of object and track query predictions
    - Via track identity or costs based on bounding box similarity and object class

$K_t \cap K_{t-1}$: Match by track identity k.

$K_{t-1} \setminus K_t$: Match with background class.

$K_t \setminus K_{t-1}$: Match by minimum cost mapping.





- Minimum cost mapping : Hungarian matching for new tracks

$$\hat{\sigma} = \arg\min_{\sigma} \sum_{k_i \in K_{\text{object}}} \mathcal{C}_{match}(y_i, \hat{y}_{\sigma(i)})$$

$$\mathcal{C}_{match}(y_i, \hat{y}_{\sigma(i)}) = \underbrace{-\lambda_{\text{cls}} \hat{p}_{\sigma(i)}(c_i)}_{①} + \mathcal{C}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})$$

$$\mathcal{C}_{\text{box}}(b_i, \hat{b}_{\sigma(i)}) = \underbrace{\lambda_{\ell_1} ||b_i - \hat{b}_{\sigma(i)}||_1}_{②} + \underbrace{\lambda_{\text{iou}} \mathcal{C}_{\text{iou}}(b_i, \hat{b}_{\sigma(i)})}_{③}$$

$\hat{y}_j \rightarrow j = \pi(i)$

Training (2/3)

- **Set prediction loss**
  - Enforces a one-to-one mapping between Track/Detection queries and ground-truth objects per frame
  - Each ground-truth object must be matched to exactly one query output
  - Matching step (Applied only during training)
    - Finds optimal one-to-one assignment between Ground-truth objects and Query outputs (Track + Detection queries)
    - Matching cost is computed using:
      - Classification cost / Bounding box L1 cost / GIoU cost
  - Loss Computation After Matching
    - Matched queries
      - Classification loss → correct class
      - Regression loss → correct bounding box
    - Unmatched queries
      - Treated as “no-object” background
      - Only classification loss applied



$$N = N_{\text{object}} + N_{\text{track}}$$

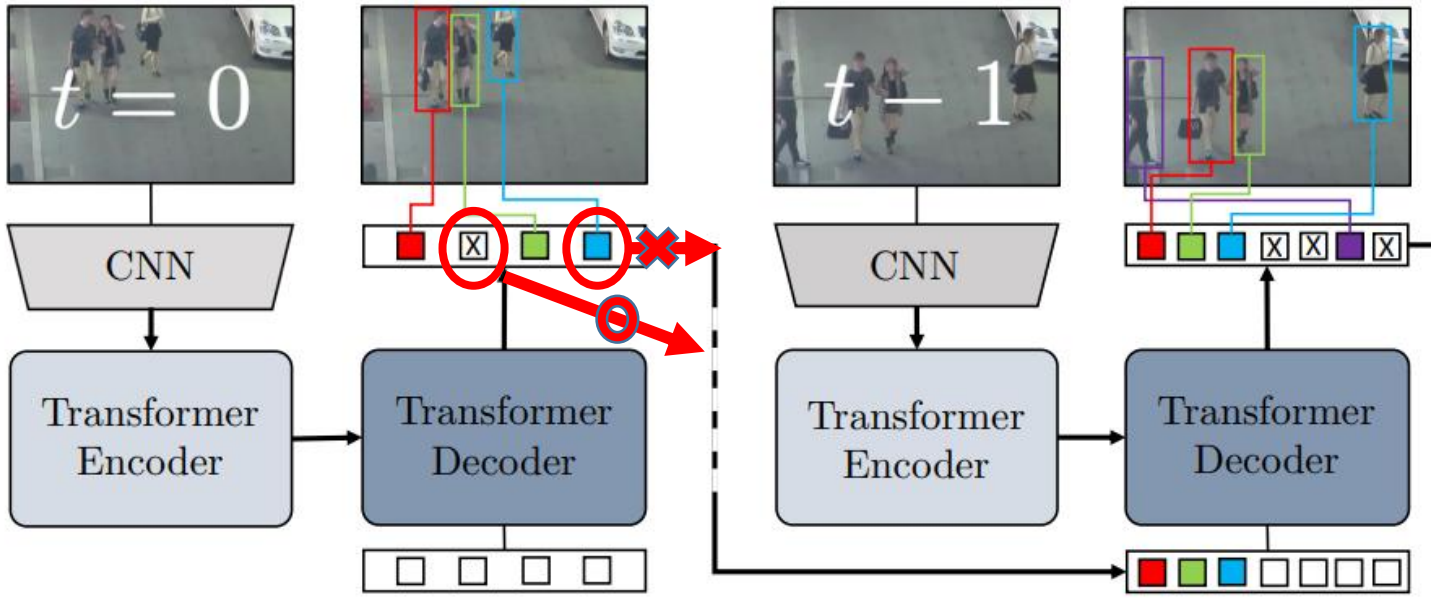
$$\mathcal{L}_{\text{MOT}}(y, \hat{y}, \pi) = \sum_{i=1}^{N} \mathcal{L}_{\text{query}}(y, \hat{y}_i, \pi)$$

$$\mathcal{L}_{\text{query}} = \begin{cases} -\lambda_{\text{cls}} \log \hat{p}_i(c_{\pi=i}) + \mathcal{L}_{\text{box}}(b_{\pi=i}, \hat{b}_i), & \text{if } i \in \pi \\ -\lambda_{\text{cls}} \log \hat{p}_i(0), & \text{if } i \notin \pi. \end{cases}$$

ground truth object matched with prediction i by $y_{\pi=i}$

Training (3/3)

- **Track augmentation**
  - Enrich the set of potential track queries during the training
  - The frame t-1 is sampled from a range of frames around frame t (i.e., skipping frames)
    → simulation of camera motion and low frame rates
  - Sample false negatives (e.g., not sending blue track to the next decoder)
    → encourage the model to have a better ability to detect new objects
  - Add false positives (e.g., sending background track to the next decoder)
    → to be robust against false positive mistakes



Inference

- **Track queries** from previous frame
  → Predict updated tracks
- **Detection** queries
  → Identify new objects
- **Unified output**
  → Combined active tracks + new detections
- **Dead tracks** are removed based on tracking confidence
  but, attention-based short-term re-identification works for short-term occlusion (based on patience window)
  → Vulnerable to re-entry scenario

Experiments

	Method	Data	FPS ↑	MOTA ↑	IDF1 ↑	MT ↑	ML ↓	FP ↓	FN ↓	ID Sw. ↓
	MOT17 [30] - Public									
Offline	jCC [22]	–	–	51.2	54.5	493	872	25937	247822	1802
Offline	FWT [19]	–	–	51.3	47.6	505	830	24101	247921	2648
Offline	eHAF [46]	–	–	51.8	54.7	551	893	33212	236772	1834
Offline	TT [65]	–	–	54.9	63.1	575	897	20236	233295	1088
Offline	MPNTrack [6]	M+C	–	58.8	61.7	679	788	17413	213594	1185
Offline	Lif_T [20]	M+C	–	60.5	65.6	637	791	14966	206619	1189
Online	FAMNet [10]	–	–	52.0	48.7	450	787	14138	253616	3072
Online	Tracktor++ [4]	M+C	1.3	56.3	55.1	498	831	**8866**	235449	1987
Online	GSM [29]	M+C	–	56.4	57.8	523	813	14379	230174	**1485**
Online	CenterTrack [69]	–	17.7	60.5	55.7	580	777	11599	208577	2540
Online	TMOH [47]	–	–	62.1	**62.8**	633	739	10951	201195	**1897**
Online	**TrackFormer**	–	7.4	**62.3**	57.6	**688**	**638**	16591	**192123**	4018
	MOT17 [30] - Private									
Online	TubeTK [32]	JTA	–	63.0	58.6	735	468	27060	177483	4137
Online	GSDT [55]	6M	–	73.2	66.5	981	411	26397	120666	3891
Online	FairMOT [66]	CH+6M	–	73.7	72.3	1017	408	27507	117477	3303
Online	PermaTrack [50]	CH+PD	–	73.8	68.9	1032	405	28998	115104	3699
Online	GRTU [54]	CH+6M	–	75.5	76.9	1158	495	27813	108690	1572
Online	TLR [53]	CH+6M	–	76.5	73.6	1122	300	29808	99510	3369
Online	CTracker [36]	–	–	66.6	57.4	759	570	22284	160491	5529
Online	CenterTrack [69]	CH	17.7	67.8	64.7	816	579	**18498**	160332	3039
Online	QuasiDense [33]	–	–	68.7	66.3	957	516	26589	146643	3378
Online	TraDeS [57]	CH	–	69.1	63.9	858	507	20892	150060	3555
Online	**TrackFormer**	CH	7.4	**74.1**	**68.0**	**1113**	**246**	34602	**108777**	**2829**
	MOT20 [13] - Private									
Online	FairMOT [66]	CH+6M	–	61.8	67.3	855	94	103440	88901	5243
Online	GSDT [55]	6M	–	67.1	67.5	660	164	31913	135409	3131
Online	SOTMOT [68]	CH+6M	–	68.6	71.4	806	120	57064	101154	4209
Online	ReMOT [60]	CH+6M	–	77.4	73.1	846	123	28351	86659	1789
Online	**TrackFormer**	CH	7.4	68.6	65.7	666	181	20348	140373	1532

Segmentation

Segmentation

- **Pixel-wise classification**



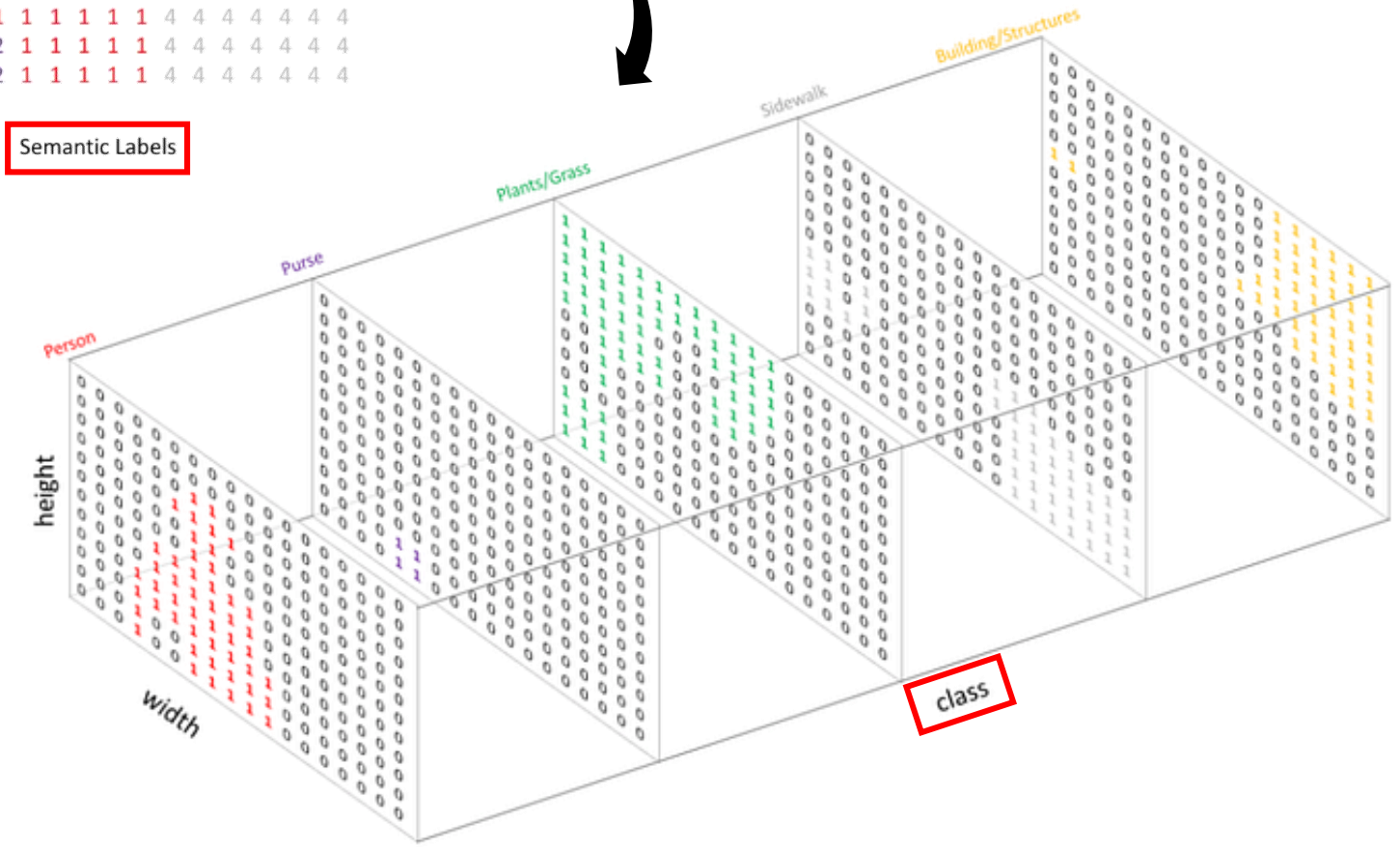
Input

segmented

1: Person
2: Purse
3: Plants/Grass
4: Sidewalk
5: Building/Structures

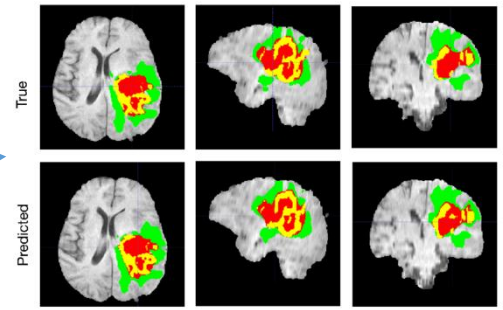
```
3 3 3 3 3 3 3 3 3 3 3 3 5 5 5 5 5 5
3 3 3 3 3 3 3 3 3 3 3 3 5 5 5 5 5 5
3 3 3 3 3 3 1 1 3 3 3 3 5 5 5 5 5 5
3 3 3 3 3 1 1 1 1 3 3 3 5 5 5 5 5 5
3 3 3 3 3 3 1 1 3 3 3 5 5 5 5 5 5 5
5 5 3 3 3 3 1 1 3 3 5 5 5 5 5 5 5 5
4 4 3 4 1 1 1 1 1 1 4 4 4 5 5 5 5 5
4 4 3 4 1 1 1 1 1 1 4 4 4 4 4 5 5 5
4 4 4 1 1 1 1 1 1 1 1 4 4 4 4 4 4 4
3 3 3 1 1 1 1 1 1 1 1 4 4 4 4 4 4 4
3 3 3 1 2 2 1 1 1 1 1 4 4 4 4 4 4 4
3 3 3 1 2 2 1 1 1 1 1 4 4 4 4 4 4 4
```

Semantic Labels

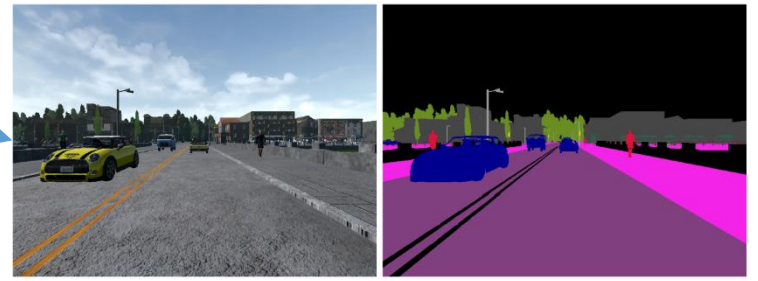


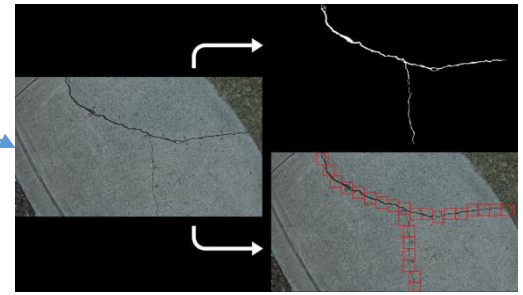
Why segmentation?

- **Precision matters**
  - Many real-world tasks require exact shapes and boundaries, not rough boxes
  - Example: medical tumor segmentation, robotic grasp point extraction
    → Detection ≈ location only, Segmentation = accurate object contour
- **Distinguishing Multiple Objects of the Same Class**
  - Need to separate each instance individually
  - Example: three people close together → overlapping shapes
    → Bounding boxes can overlap and fail to separate object identities
- **Understanding the Whole Scene**
  - Not only objects, but roads, sky, sidewalk, terrain also matter
  - Example: autonomous driving: drivable vs non-drivable areas
- **Pixel-Level Decision Making**
  - Tiny defects or anomalies may be missed by bounding boxes
  - Example: manufacturing inspection, surface crack detection
    → Pixel-wise classification enables fine-grained quality assurance
- **Foundation for Future Vision Systems**
  - Key enabler for AR background removal, image editing
  - Example: Bokeh (i.e., background blur in image capture)
  - Trend → More general, more interactive segmentation models









Metric

- **IoU (Intersection over Union)**
  - Measures overlap between prediction and ground truth for each class

$$IoU_c = \frac{|P_c \cap G_c|}{|P_c \cup G_c|}$$

- **mIoU (Mean IoU)** → The most widely used metric for semantic segmentation benchmarks !
  - Average IoU across all classes

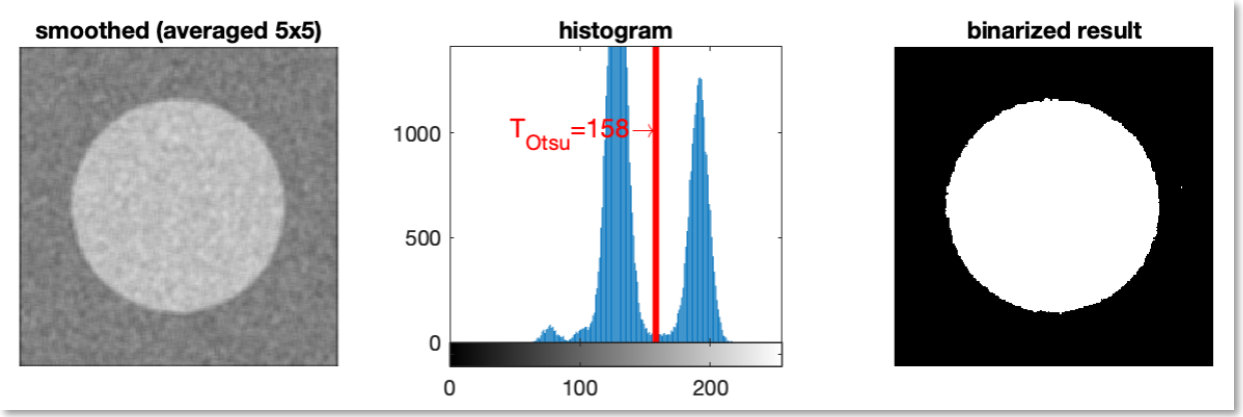
$$mIoU = \frac{1}{C} \sum_{c=1}^{C} \frac{|P_c \cap G_c|}{|P_c \cup G_c|}$$

- **FWIoU (Frequency Weighted IoU)**
  - Applying class frequency as a weighting factor
  - Mitigates imbalance by weighting classes based on their occurrence frequency

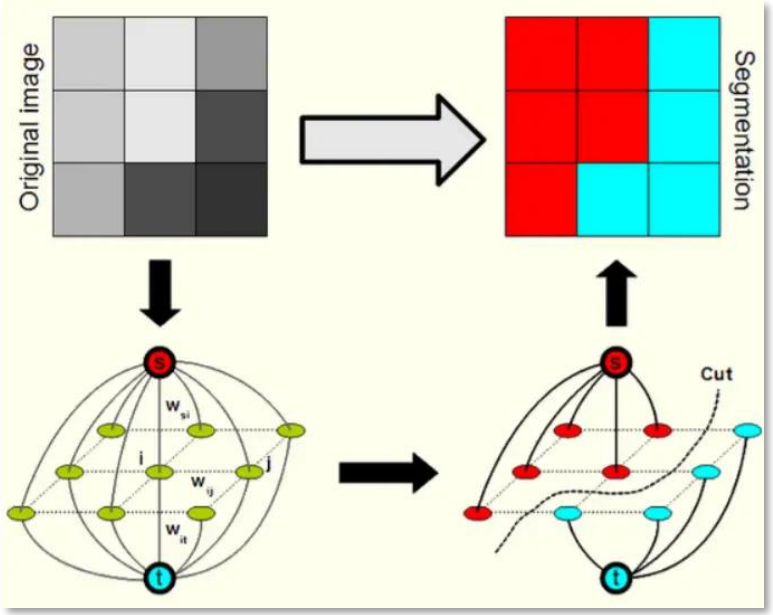
$$FWIoU = \sum_{c} \frac{n_c}{N} IoU_c$$

Classical segmentation

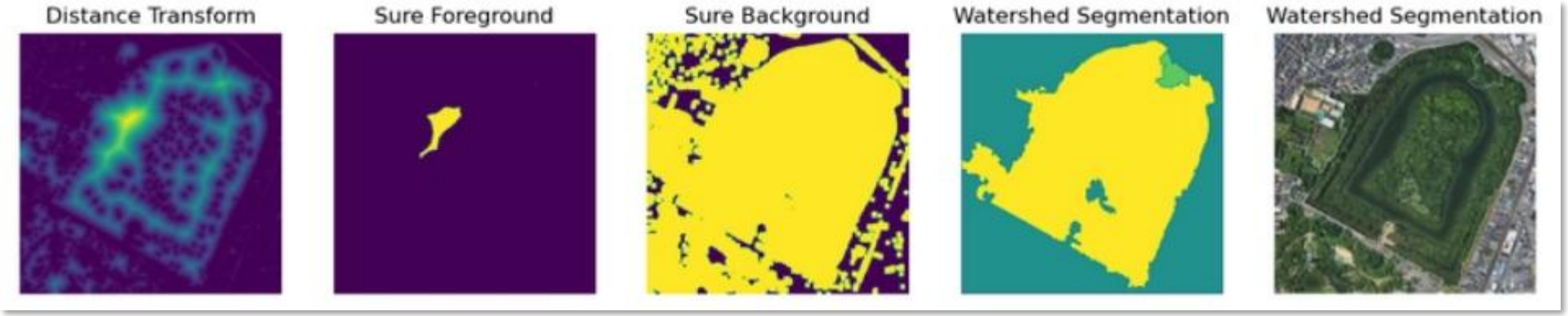
- Thresholding / Otsu
- Clustering-based methods (K-means clustering)
- Graph-based methods (GrabCut)
- Edge-based method (Canny edge + watershed)



http://moodle.mmclab.eu/mod/page/view.php?id=59&lang=en



https://machinelearningknowledge.ai/foreground-extraction-using-grabcut-algorithm-in-python-opencv-with-example/

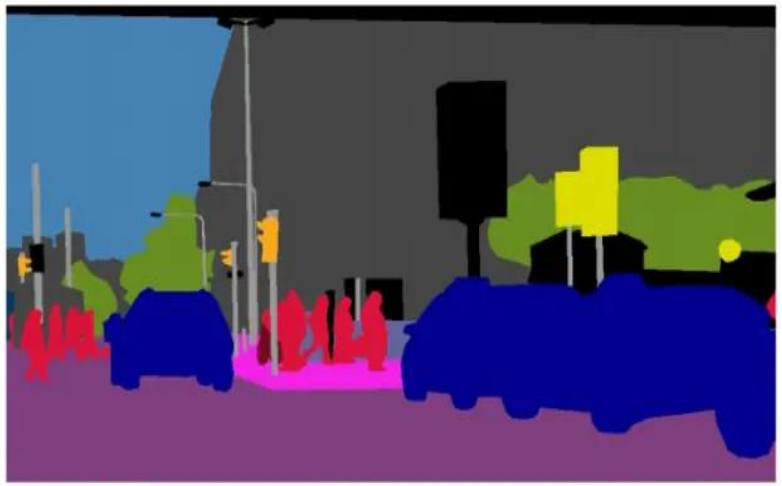


https://dev.to/jarvissan22/python-cv2-image-segmentation-canny-edges-watershed-and-k-means-methods-18l0

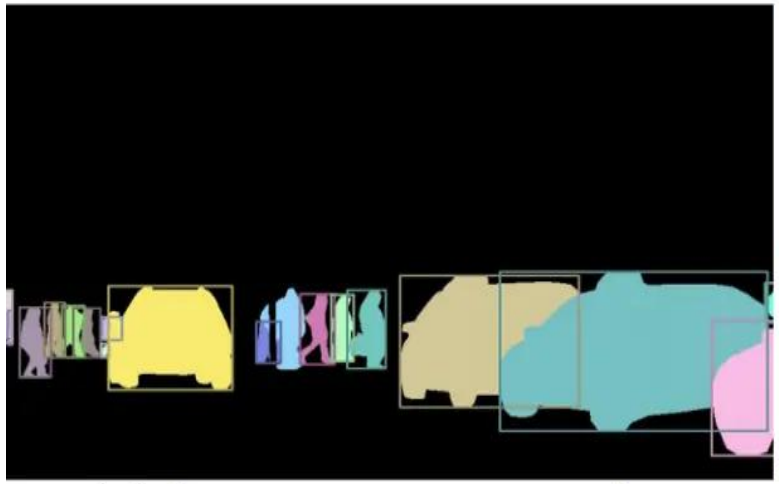
Segmentation category



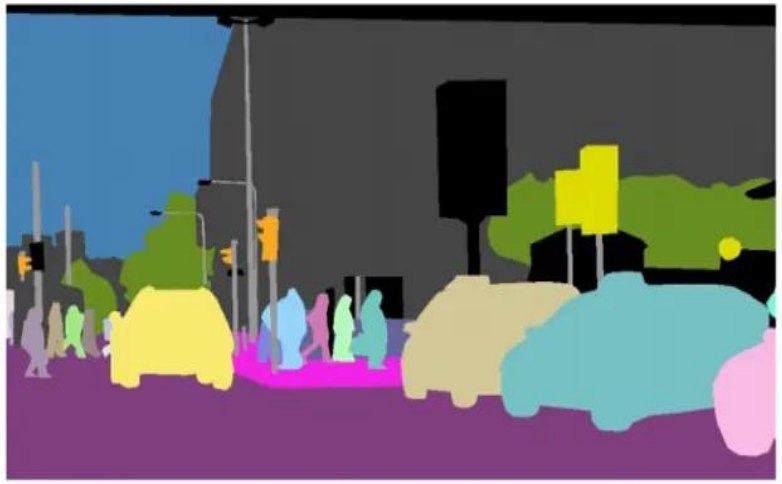
(a) image



(b) semantic segmentation



(c) instance segmentation



(d) panoptic segmentation

➡ **Things & Stuff**

[FCN]

- **Motivation**
  - Classic CNNs use fully-connected (FC) layers → collapse spatial resolution
  - For segmentation,
    - Predict a class label for every pixel
      → Preserve spatial structure + dense prediction required
- **Key idea**
  - Network remains fully convolutional
    - Remove Fully-connected layers
      → Replace with 1×1 convolution
  - 2D Feature Map → Segmentation Map
    - Final output shape = input spatial size (H×W)
    - Up-sampling method
      - Initialization with fixed bilinear interpolation
      - Learned up-sampling (deconvolution)

First end-to-end, **fully convolutional**
deep learning architecture
for **semantic segmentation !**

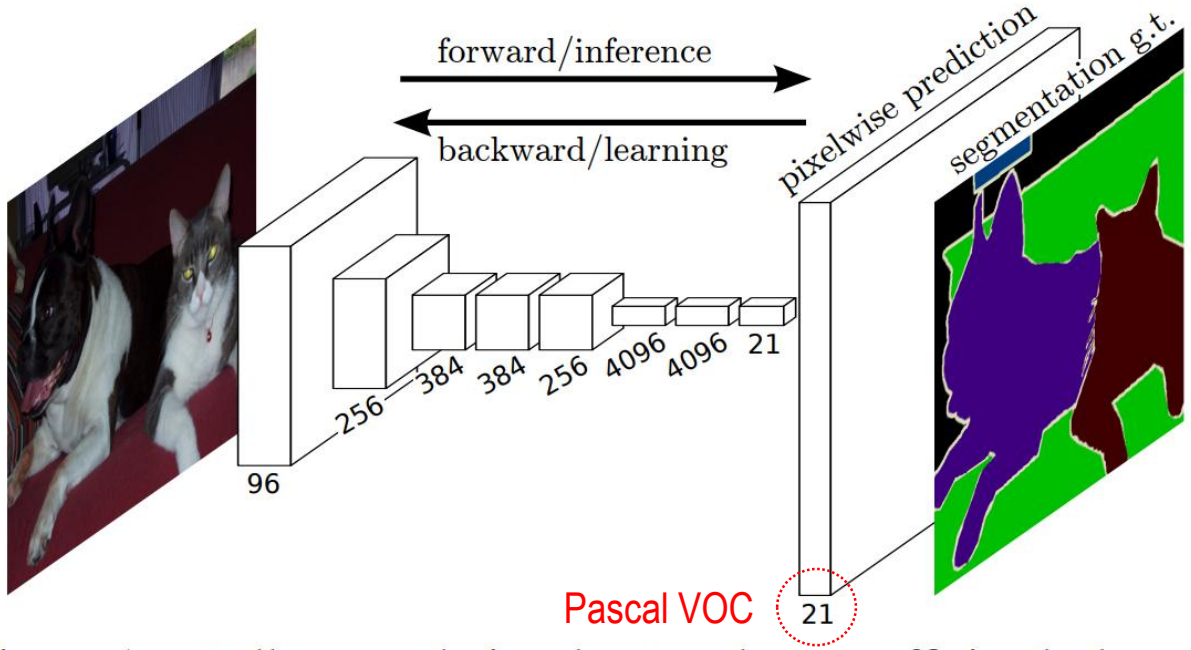


Figure 1. Fully convolutional networks can efficiently learn to make dense predictions for per-pixel tasks like semantic segmentation.

Skip fusion

- **Recover spatial details** which are lost in downsampling
- Combine **deep semantic + shallow spatial information**

Fusion Level	Source	Purpose
FCN-32s	Deepest layer only	Coarsest output
FCN-16s	+ Pool4	More spatial detail
FCN-8s	+ Pool3	Best boundary accuracy

Upsampling + multi-scale skip fusion improves segmentation quality !

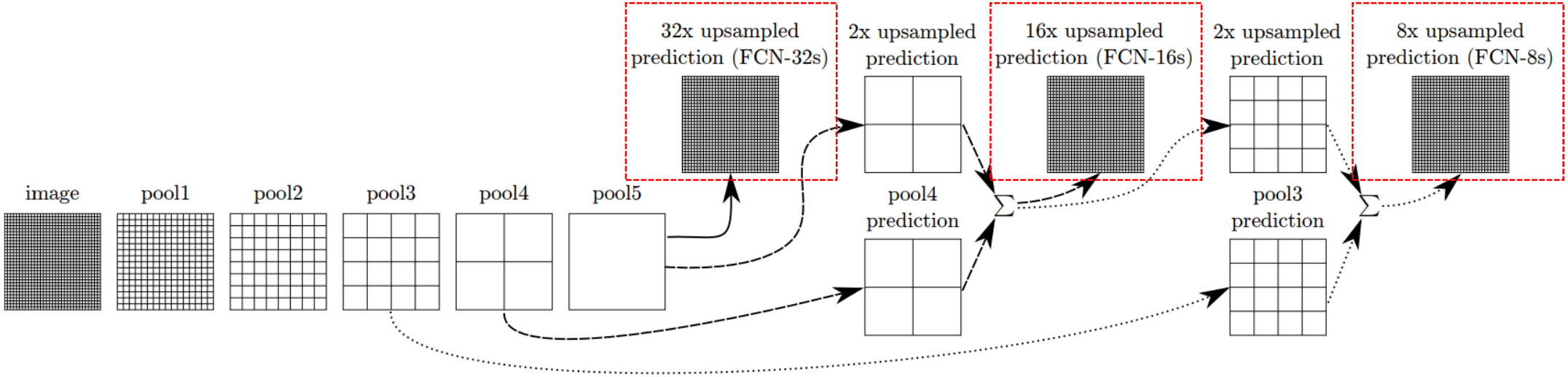
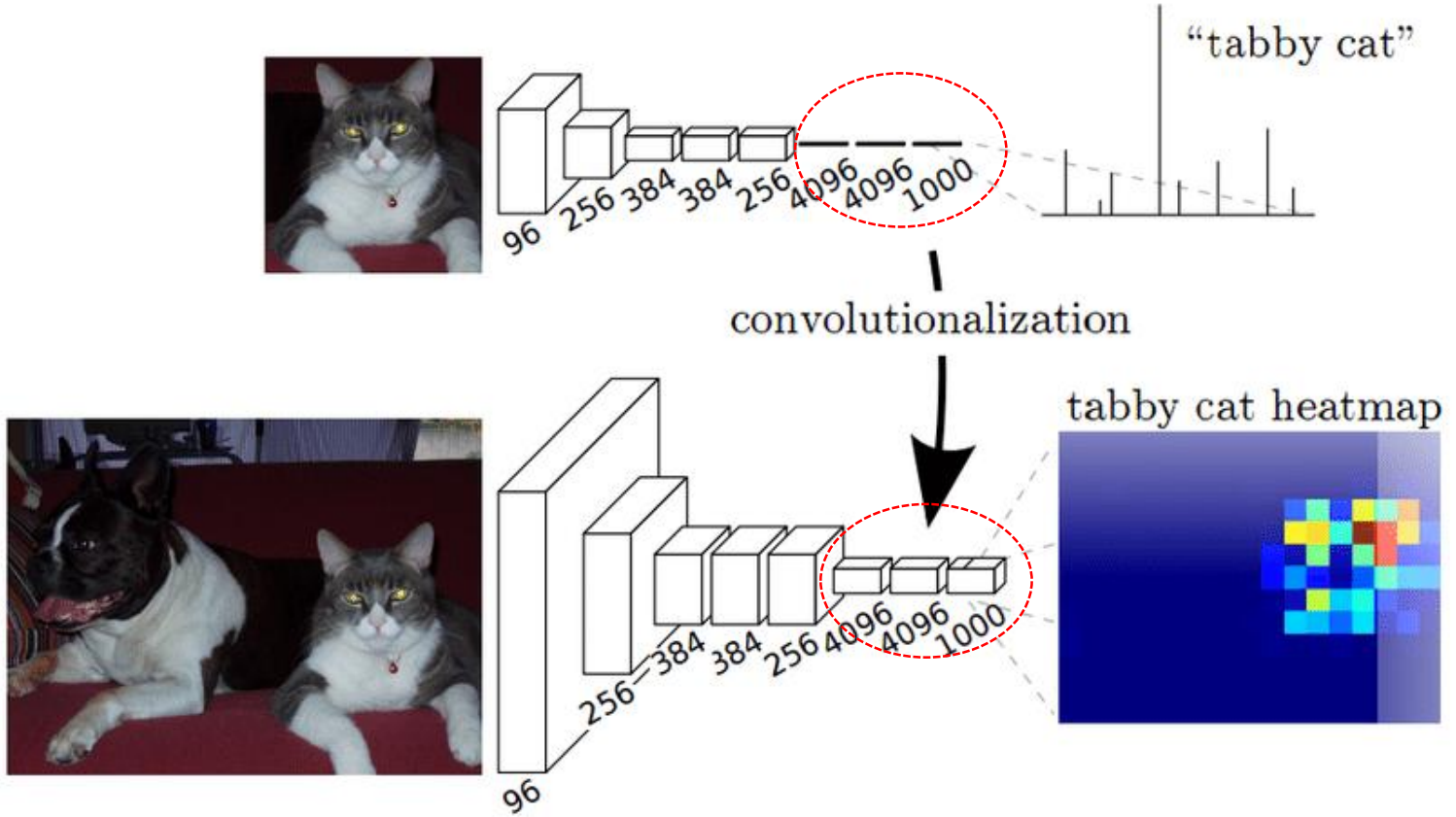


Figure 3. Our DAG nets learn to combine coarse, high layer information with fine, low layer information. Layers are shown as grids that reveal relative spatial coarseness. Only pooling and prediction layers are shown; intermediate convolution layers (including our converted fully connected layers) are omitted. Solid line (FCN-32s): Our single-stream net, described in Section 4.1, upsamples stride 32 predictions back to pixels in a single step. Dashed line (FCN-16s): Combining predictions from both the final layer and the `pool4` layer, at stride 16, lets our net predict finer details, while retaining high-level semantic information. Dotted line (FCN-8s): Additional predictions from `pool3`, at stride 8, provide further precision.

Training

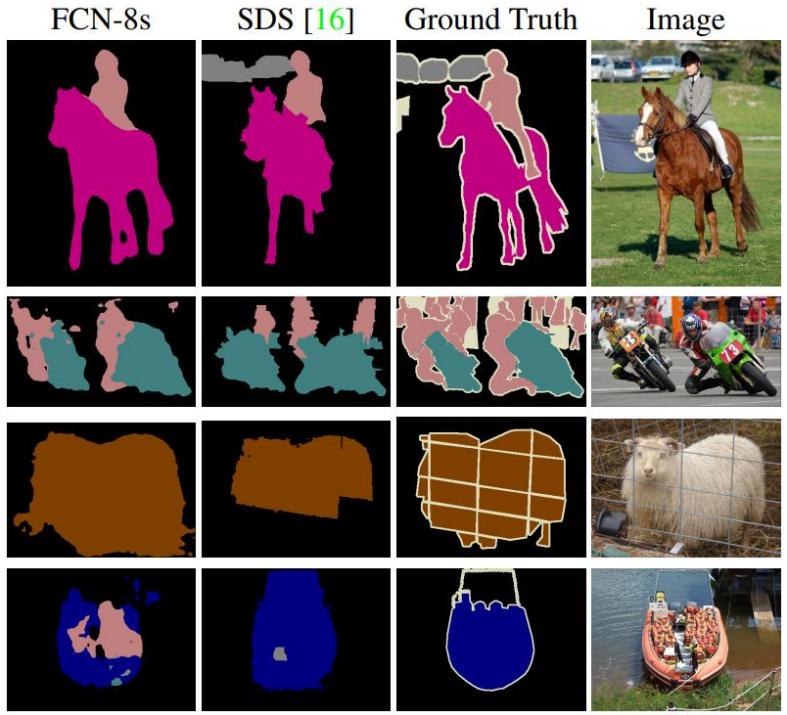
- **Pixel-wise Softmax + Cross Entropy**
  - Loss is summed/averaged across all pixels
- **End-to-end training via backpropagation**



Summary

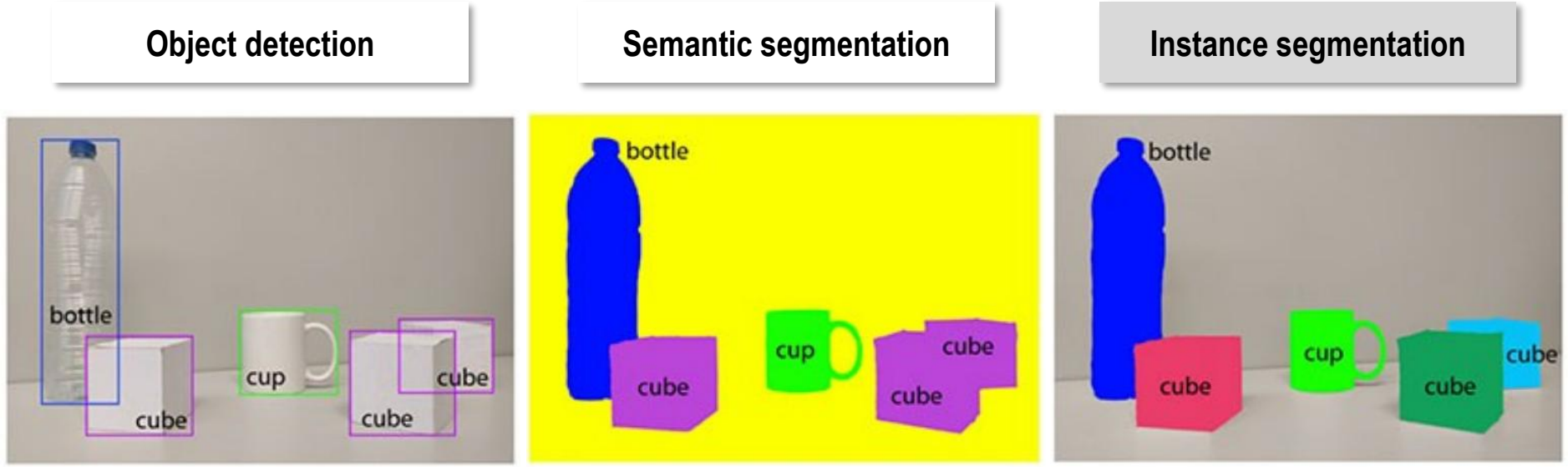
- **Strengths & Weakness**

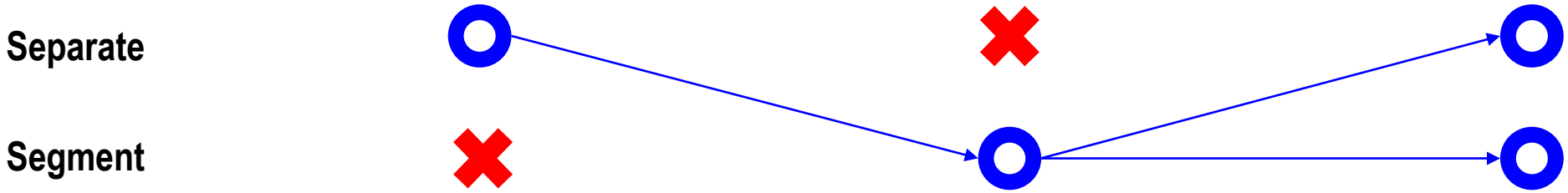
Strengths	Weaknesses
First deep learning model for segmentation	Upsampled masks may look coarse
Efficient inference (fast)	Limited edge sharpness
Can reuse pre-trained backbones	Not ideal for complex boundaries
Simple architecture	No object separation (semantic only)



[Mask R-CNN]

- **Motivation**
  - Extending detection (i.e., Faster R-CNN) to the **instance segmentation**



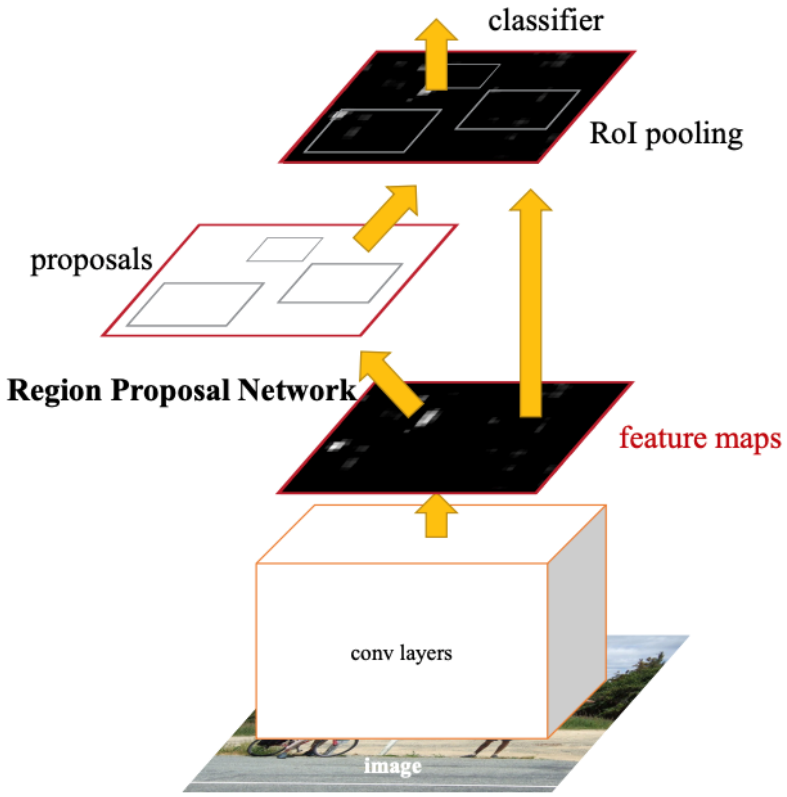


→ Let's segment on each bounding box !

https://www.linkedin.com/pulse/quick-understanding-instance-segmentation-vs-semantic-rohan-chikorde

Overview

- **Key idea**
  - Build on Faster R-CNN, then add a parallel mask prediction branch
    → Multi-task model : Detection (class & bonding box) + Segmentation simultaneously



Faster R-CNN

Faster R-CNN with FCN on RoIs

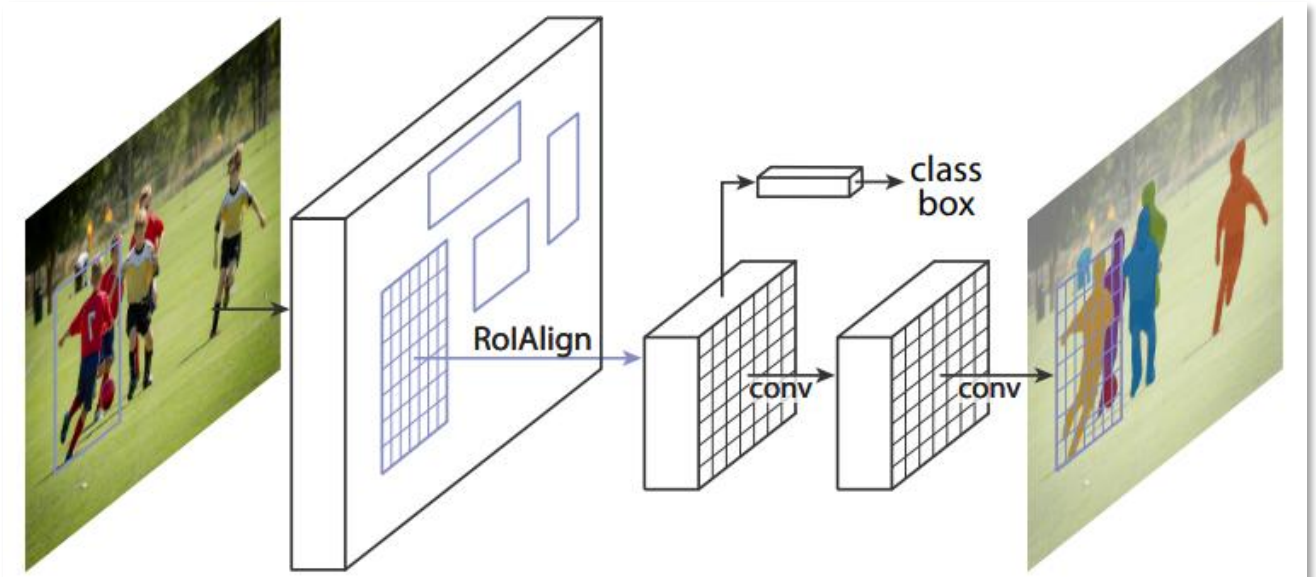
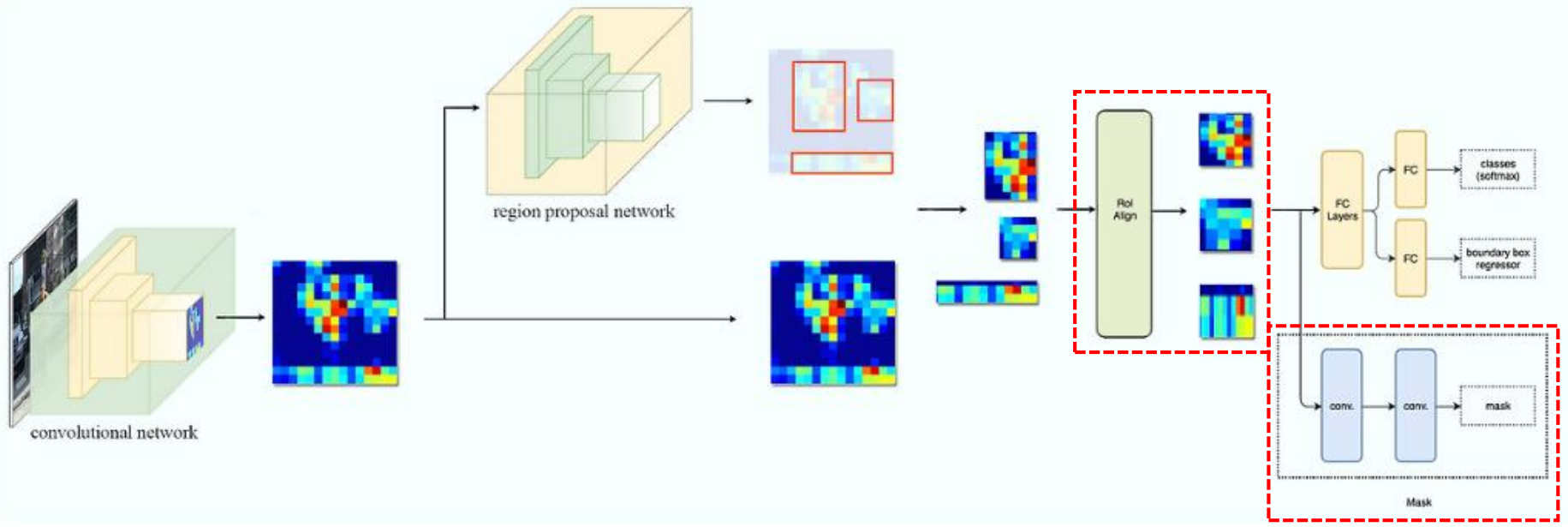


Figure 1. The **Mask R-CNN** framework for instance segmentation.

Architecture

- **Feature extraction** — Multi-scale feature maps for better object-size handling
- **RPN** (Region Proposal Network) — Generates candidate RoIs (regions of interest)
- **ROI Align** — Extract fixed-size region features **without misalignment**
- **Multi-task heads**

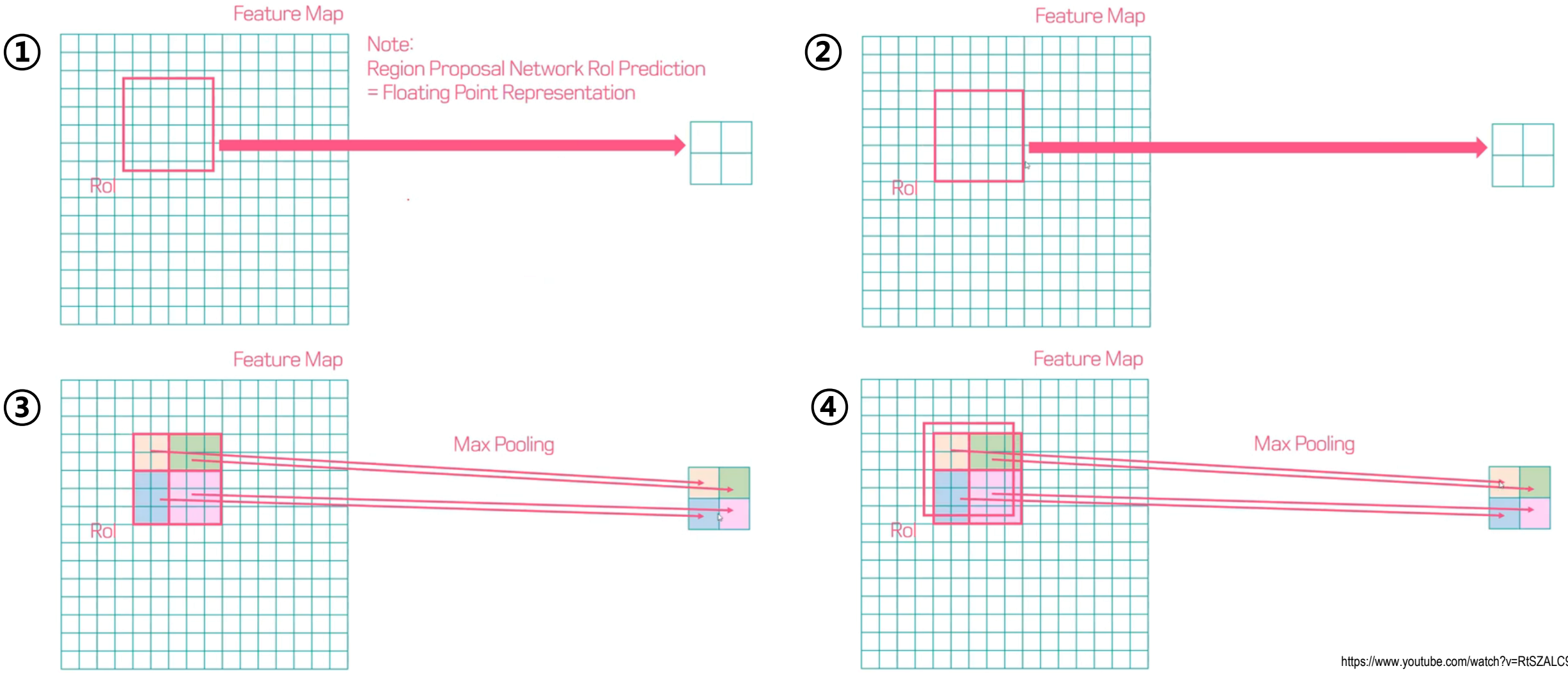
Head	Output	Loss
Classification head	Object category	Softmax CE
Bounding box regression head	Refined coordinates	Smooth-L1
Mask head	K binary masks (per class)	Pixel-wise BCE



https://herbwood.tistory.com/20

ROI Align (1/2)

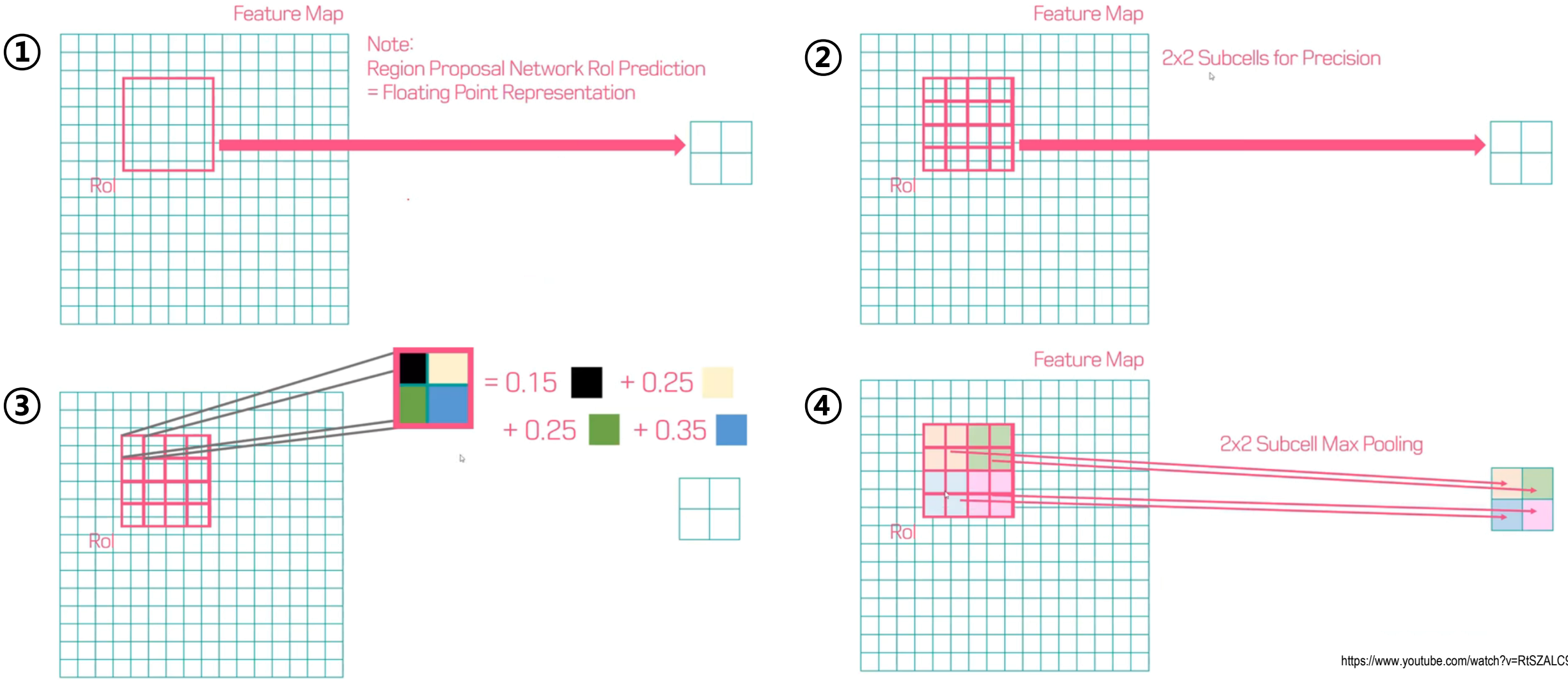
- **ROI Pooling**
  - 'Rounding' → Boundary misalignment → Poor mask



https://www.youtube.com/watch?v=RtSZALC9DIU

ROI Align (2/2)

- **ROI Align**
  - Goal : Extracting fixed-size region features **without misalignment** → Precise pooled feature
  - 2D 'Bilinear interpolation' → No quantization → Higher accuracy mask



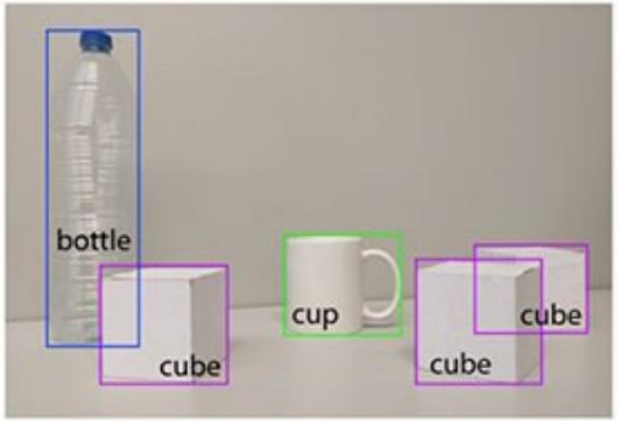
https://www.youtube.com/watch?v=RtSZALC9DIU

Mask head

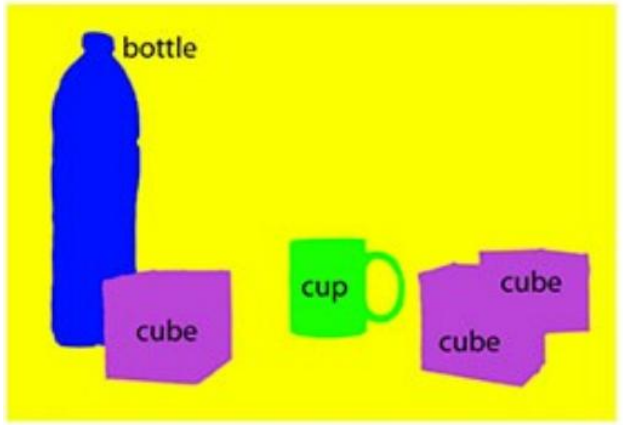
Mask R-CNN, ICCV 2017

- **Problem definition of the mask head**

Faster R-CNN



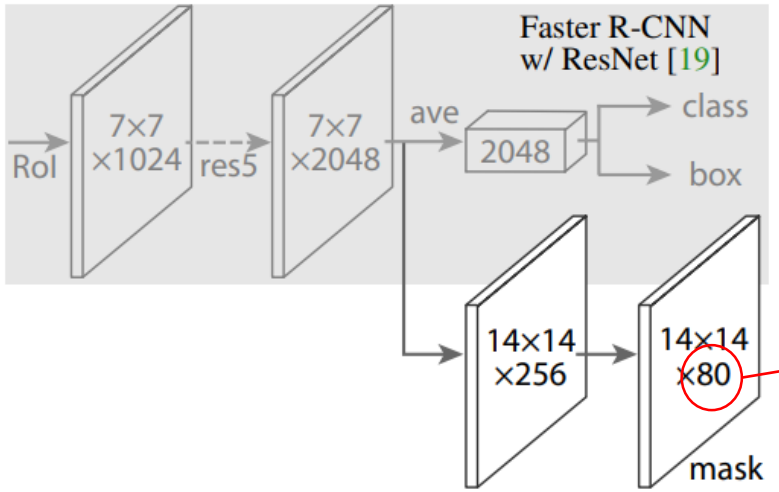
FCN





- Per pixel
  - Multi-class classification
    → Binary classification
  - Softmax
    → Sigmoid

Mask R-CNN



Binary mask for each class

Experiments (1/3)

- **Binary masks** is superior to the multinomial masks
- FCN keeps the equivariance property rather than MLP

	AP	AP_{50}	AP_{75}
softmax	24.8	44.1	25.1
sigmoid	**30.3**	**51.2**	**31.5**
	+5.5	*+7.1*	*+6.4*

(b) **Multinomial *vs*. Independent Masks** (ResNet-50-C4): *Decoupling* via per-class binary masks (sigmoid) gives large gains over multinomial masks (softmax).

	mask branch	AP	AP_{50}	AP_{75}
MLP	fc: 1024→1024→80·28^2	31.5	53.7	32.8
MLP	fc: 1024→1024→1024→80·28^2	31.5	54.0	32.6
FCN	conv: 256→256→256→256→256→80	**33.6**	**55.2**	**35.3**

(e) **Mask Branch** (ResNet-50-FPN): Fully convolutional networks (FCN) *vs*. multi-layer perceptrons (MLP, fully-connected) for mask prediction. FCNs improve results as they take advantage of explicitly encoding spatial layout.

Experiments (2/3)

- **RoI Align** improves AP and bounding box accuracy effectively

	align?	bilinear?	agg.	AP	AP_{50}	AP_{75}
RoIPool [12]			max	26.9	48.8	26.4
RoIWarp [10]		✓	max	27.2	49.2	27.1
		✓	ave	27.1	48.9	27.1
RoIAlign	✓	✓	max	**30.2**	**51.0**	**31.8**
	✓	✓	ave	**30.3**	**51.2**	**31.5**

(c) **RoIAlign** (ResNet-50-C4): Mask results with various RoI layers. Our RoIAlign layer improves AP by ~3 points and AP_{75} by ~5 points. Using proper alignment is the only factor that contributes to the large gap between RoI layers.

	AP	AP_{50}	AP_{75}	AP^{bb}	AP^{bb}_{50}	AP^{bb}_{75}
RoIPool	23.6	46.5	21.6	28.2	52.7	26.9
RoIAlign	**30.9**	**51.8**	**32.1**	**34.0**	**55.3**	**36.4**
	+7.3	*+ 5.3*	*+10.5*	*+5.8*	*+2.6*	*+9.5*

(d) **RoIAlign** (ResNet-50-**C5**, *stride 32*): Mask-level and box-level AP using *large-stride* features. Misalignments are more severe than with stride-16 features (Table 2c), resulting in big accuracy gaps.

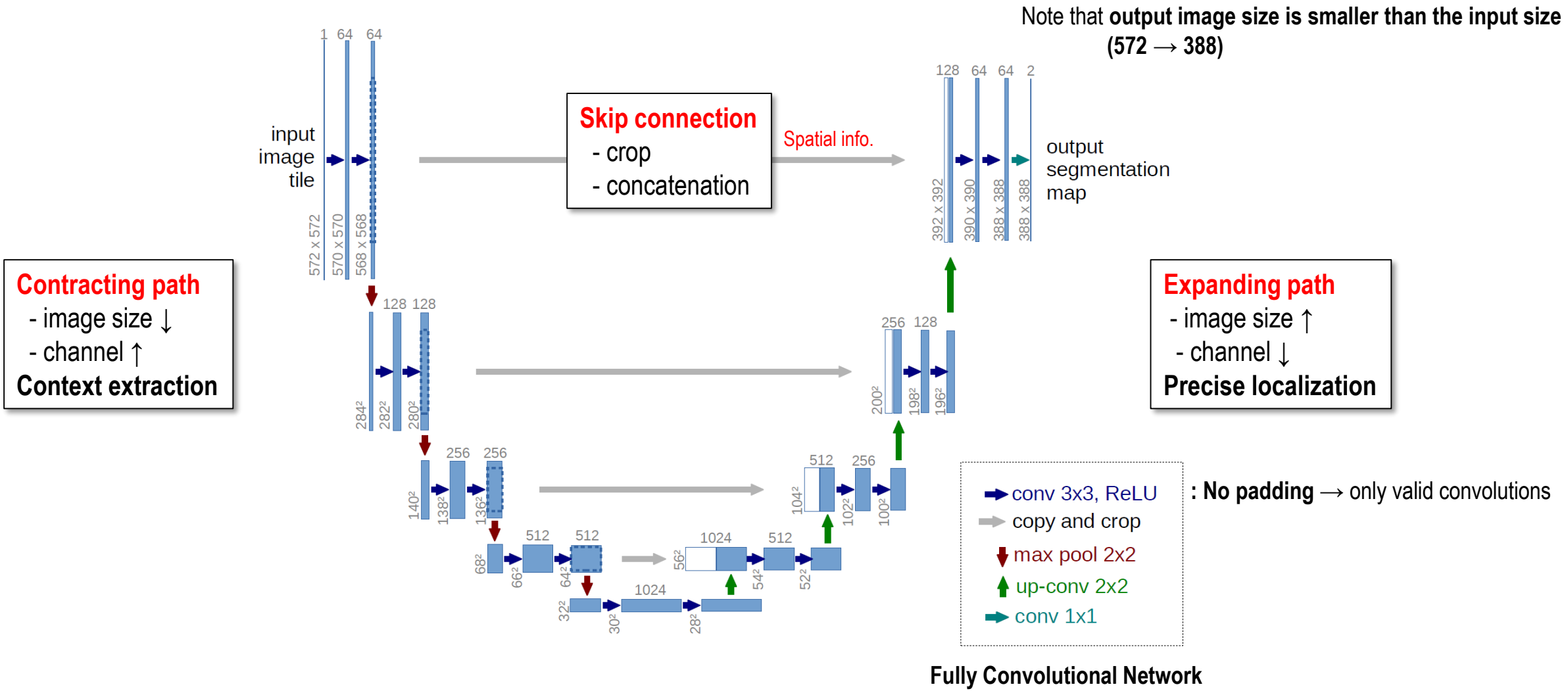
Experiments (3/3)



[U-Net]

U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI 2015

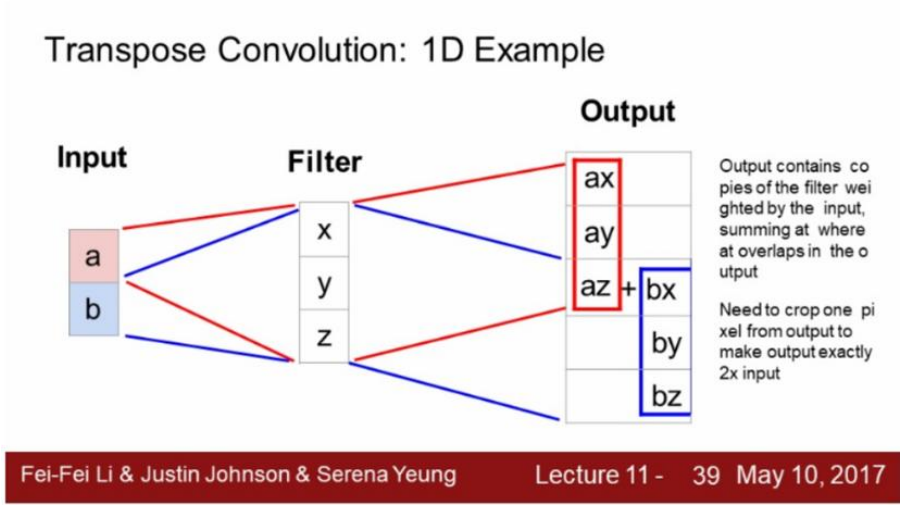
- Architecture



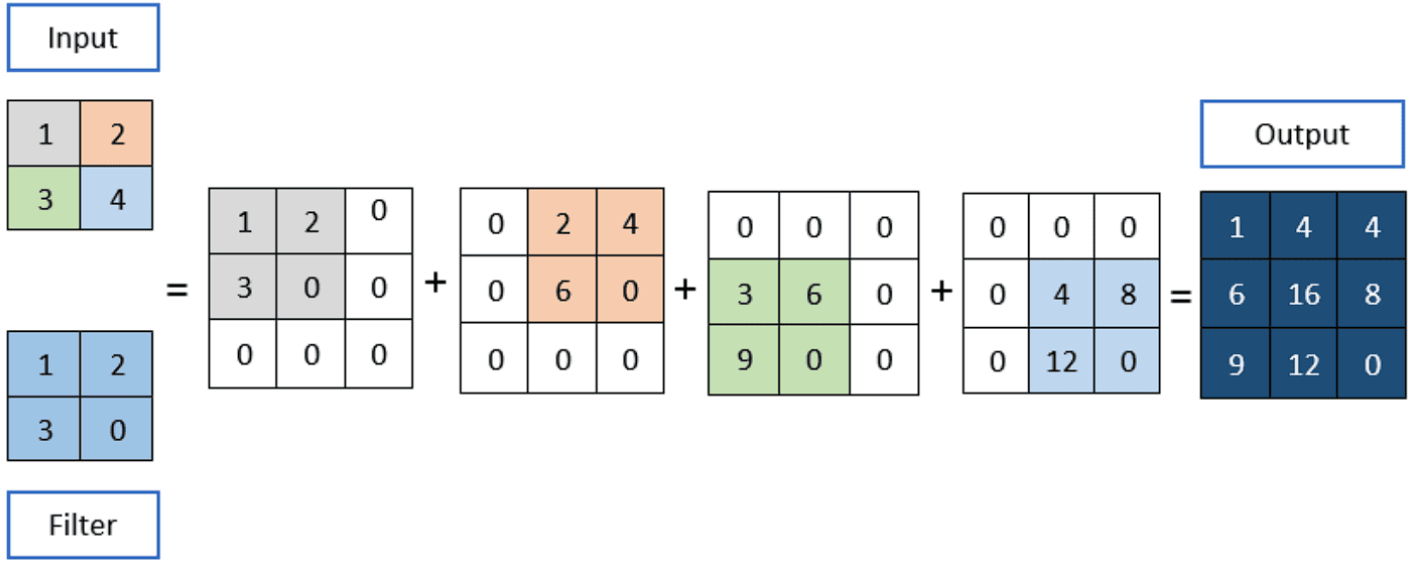
Transpose convolution (Up-sampling)

U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI 2015

1D Transpose convolution



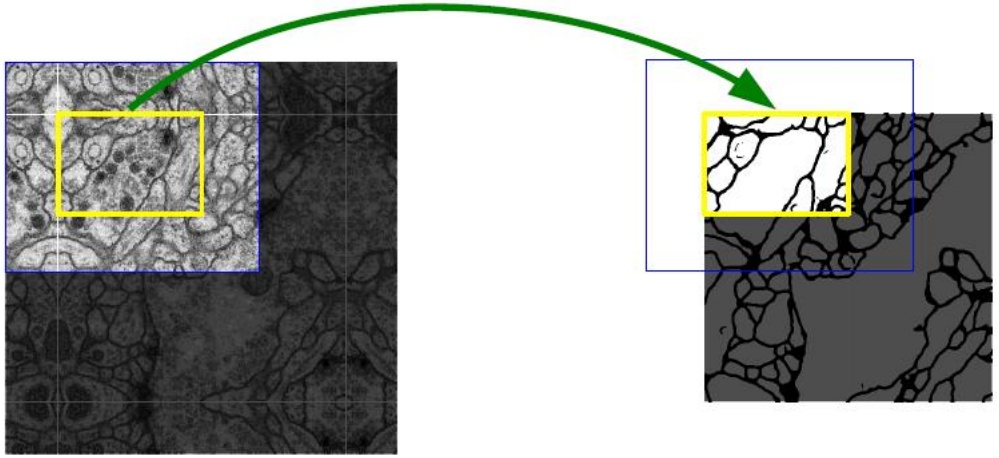
2D Transpose convolution



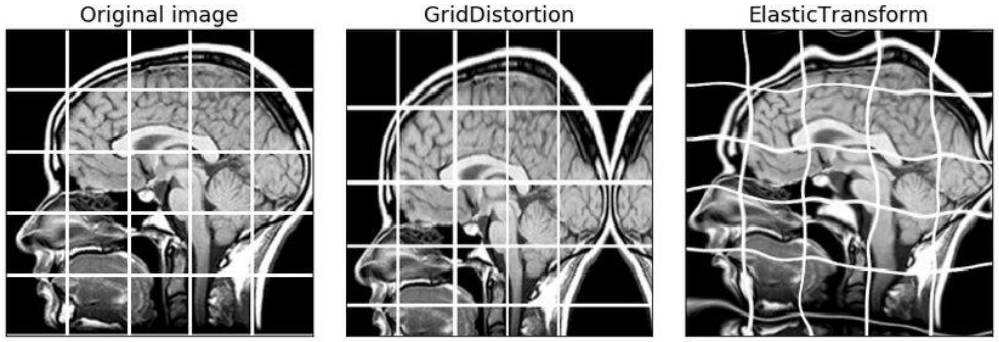
Training (1/2)

- **Overlap-tile strategy** for large images (especially high-resolution medical images)
  - Instead of resizing the whole image, it divides the image into tiles.
  - A tile is predicted using a larger overlapping input area with mirroring extrapolation
- **Elastic transformation**
  - A data augmentation to teach the network invariance to deformation

Overlap-tile strategy
i.e., Mirroring extrapolation



Data augmentation
e.g., Elastic transformation



High labeling cost → Small training set

Albumentations: fast and flexible image augmentations, 2020

Training (2/2)

- **Energy (i.e., loss)**

$$E = \sum_{\mathbf{x}\in\Omega} w(\mathbf{x}) \log(p_{\ell(\mathbf{x})}(\mathbf{x}))$$

$$w(\mathbf{x}) = \underbrace{w_c(\mathbf{x})}_{①} + \underbrace{w_0 \cdot \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right)}_{②}$$

① : To balance the class frequencies. i.e., the inverse frequency

② : To emphasis pixels on the boundary

where $w_c : \Omega \to \mathbb{R}$ is the weight map to balance the class frequencies, $d_1 : \Omega \to \mathbb{R}$ denotes the distance to the border of the nearest cell and $d_2 : \Omega \to \mathbb{R}$ the distance to the border of the second nearest cell. In our experiments we set $w_0 = 10$ and $\sigma \approx 5$ pixels.

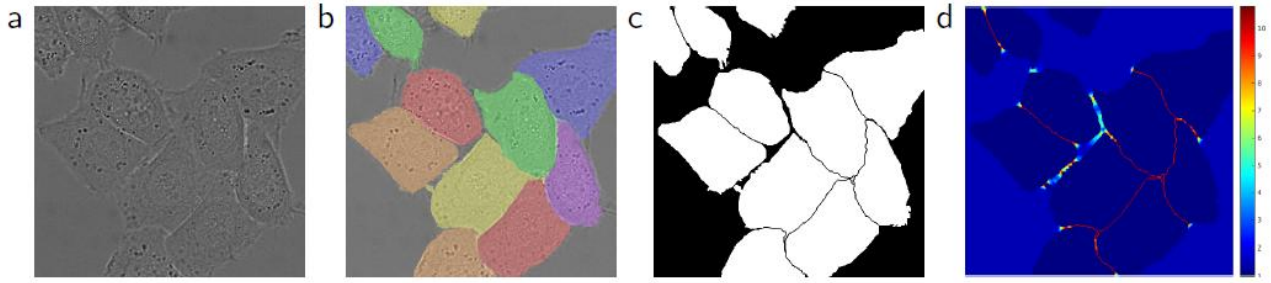


Fig. 3. HeLa cells on glass recorded with DIC (differential interference contrast) microscopy. (**a**) raw image. (**b**) overlay with ground truth segmentation. Different colors indicate different instances of the HeLa cells. (**c**) generated segmentation mask (white: foreground, black: background). (**d**) map with a pixel-wise loss weight to force the network to learn the border pixels.

Experiments

Table 2. Segmentation results (IOU) on the ISBI cell tracking challenge 2015.

Name	PhC-U373	DIC-HeLa
IMCB-SG (2014)	0.2669	0.2935
KTH-SE (2014)	0.7953	0.4607
HOUS-US (2014)	0.5323	-
second-best 2015	0.83	0.46
u-net (2015)	**0.9203**	**0.7756**

ISBI Cell Tracking Challenge 2015

Big jump !

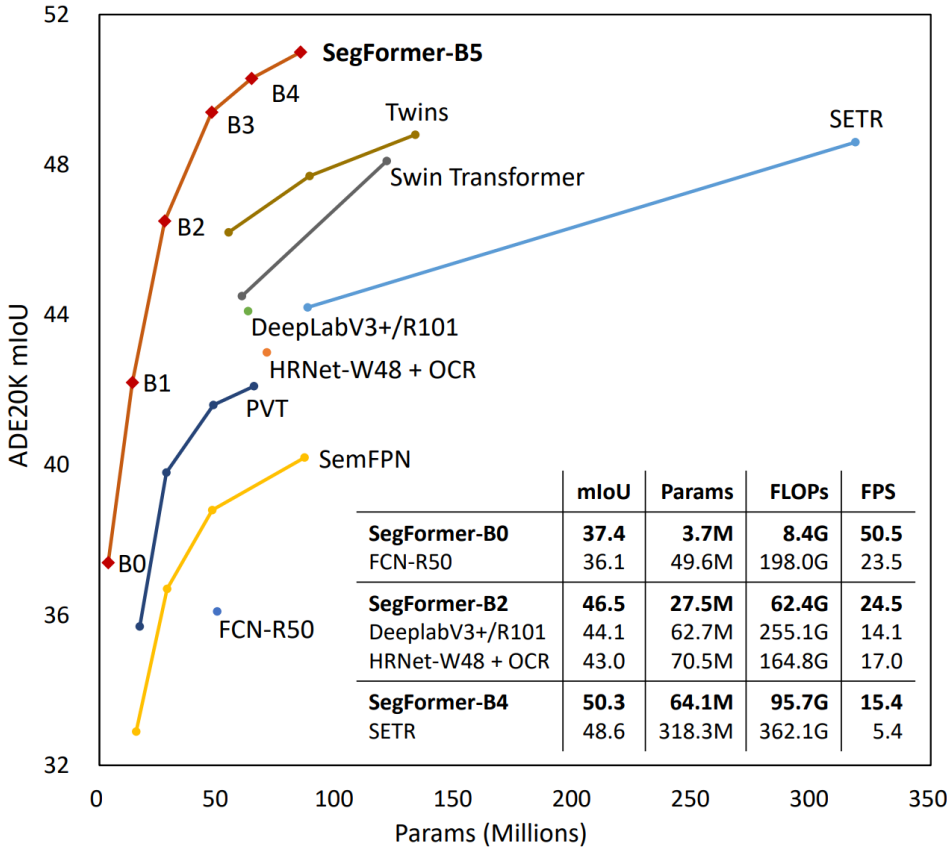
MICCAI 2015

— Medical Image Computing and Computer-Assisted Intervention

[SegFormer]

- **Motivation**
  - Limited Global Context in CNN-based models
    - CNNs mainly focus on local receptive fields
  - High computational cost for high-resolution features
    - Deep encoders lose spatial detail
    - Heavy decoders required to recover resolution
      → Slower inference, difficult for real-time applications
  - Transformer models not fully optimized for segmentation
    - Standard ViT uses single-scale features
    - Objects vary in size → scale ambiguity
    - Strong reliance on positional encoding

Achieve both high accuracy and high efficiency
by strengthening the encoder and simplifying the decoder !



	mIoU	Params	FLOPs	FPS
SegFormer-B0	**37.4**	**3.7M**	**8.4G**	**50.5**
FCN-R50	36.1	49.6M	198.0G	23.5
SegFormer-B2	**46.5**	**27.5M**	**62.4G**	**24.5**
DeeplabV3+/R101	44.1	62.7M	255.1G	14.1
HRNet-W48 + OCR	43.0	70.5M	164.8G	17.0
SegFormer-B4	**50.3**	**64.1M**	**95.7G**	**15.4**
SETR	48.6	318.3M	362.1G	5.4

Figure 1: **Performance *vs.* model efficiency on ADE20K.** All results are reported with single model and single-scale inference. SegFormer achieves a new state-of-the-art 51.0% mIoU while being significantly more efficient than previous methods.

Architecture

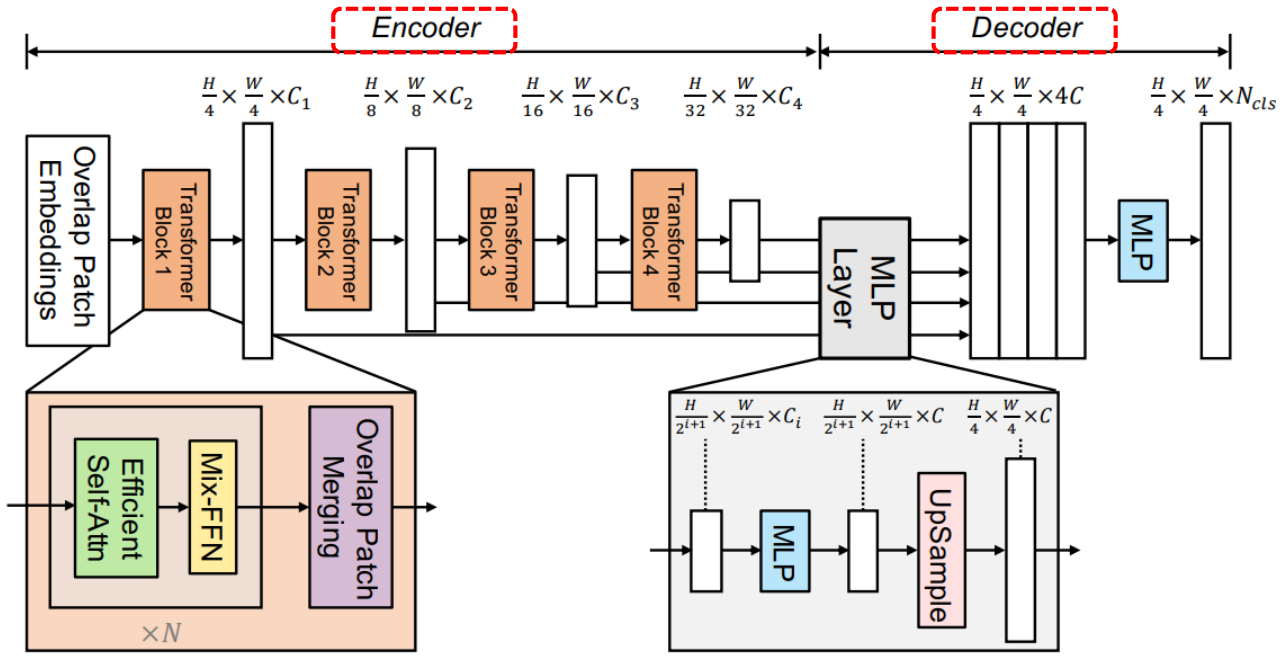
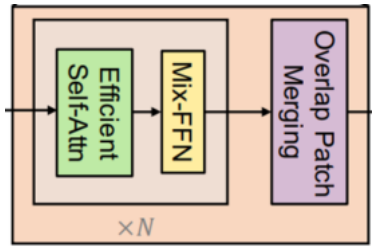


Figure 2: **The proposed SegFormer framework** consists of two main modules: A hierarchical Transformer encoder to extract coarse and fine features; and a lightweight All-MLP decoder to directly fuse these multi-level features and predict the semantic segmentation mask. "FFN" indicates feed-forward network.

	Summary
Hierarchical Transformer Encoder	Strong multi-scale features
Lightweight All-MLP Decoder	Fast and efficient segmentation
No Positional Encoding	Better generalization and simpler design

Efficient self-attention

SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers, 2021



- **Standard self-attention**
  - Causing quadratic complexity because every token attends to every other token
- **Efficient self-attention**
  - Reducing token interactions through token compression
  - Fewer tokens at deeper stages thanks to hierarchical structure
  - Maintaining attention globally with lower resolution representations → Not true global attention

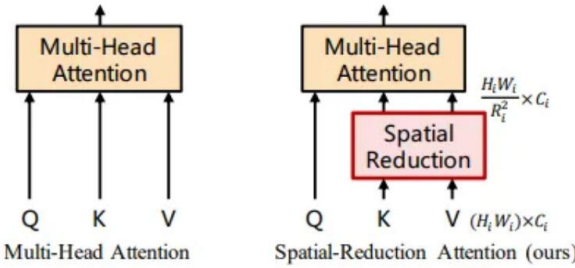
$$\text{Attention}(Q, K, V) = \text{Softmax}(\frac{QK^{\top}}{\sqrt{d_{head}}})V. \quad \Rightarrow \quad K : (N, C) \quad \Rightarrow \quad O(N^2)$$

N : # of patches
C : feature dimension

$$\hat{K} = \text{Reshape}(\frac{N}{R}, C \cdot R)(K)$$
$$K = \text{Linear}(C \cdot R, C)(\hat{K}), \quad \Rightarrow \quad K : (N, C) \rightarrow K : (N/R, C) \quad \Rightarrow \quad O(\frac{N^2}{R})$$

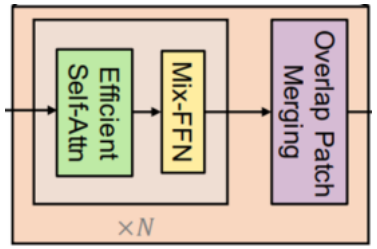
Q: (N, C)

R : reduction ratio : [64, 16, 4, 1]



https://lcyking.tistory.com/entry/%EB%85%BC%EB%AC%B8%EB%A6%AC%EB%B7%B0-SegFormer-Simple-and-Efficient-Design-for-SemanticSegmentation-with-Transformers

Mix-FFN

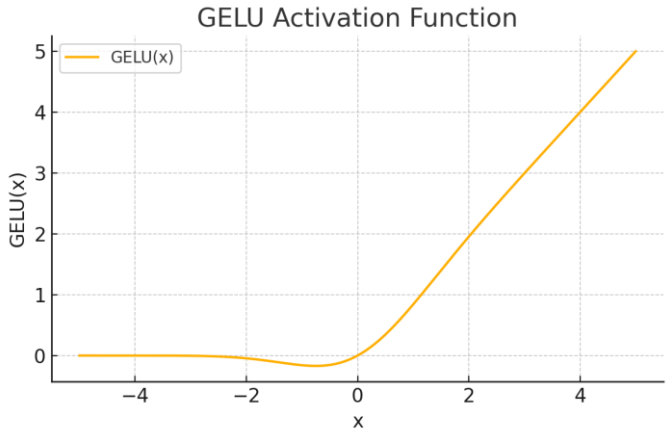


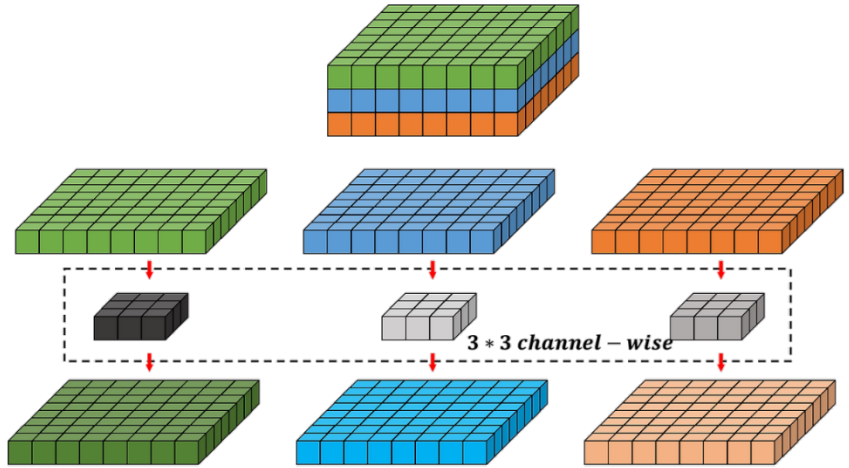
- **Positional embedding removal**
  - Zero-padding for convolution provides the network with location information

How much position information do convolutional neural networks encode?, ICLR 2020

- **Depth-wise convolution**
  - Spatial context
  - Efficiency
  - Channel preservation

$$\mathbf{x}_{out} = \text{MLP}(\text{GELU}(\text{Conv}_{3\times3}(\text{MLP}(\mathbf{x}_{in})))) + \mathbf{x}_{in}$$

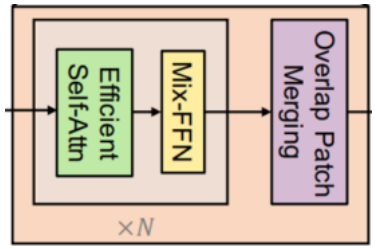




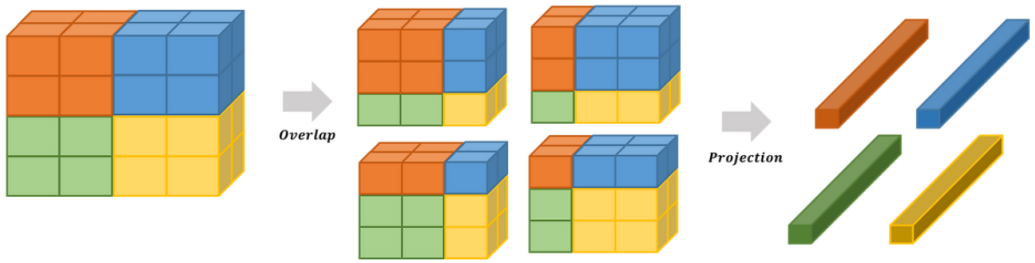
https://lcyking.tistory.com/entry/%EB%85%BC%EB%AC%B8%EB%A6%AC%EB%B7%B0-SegFormer-Simple-and-Efficient-Design-for-SemanticSegmentation-with-Transformers

Overlap patch merging

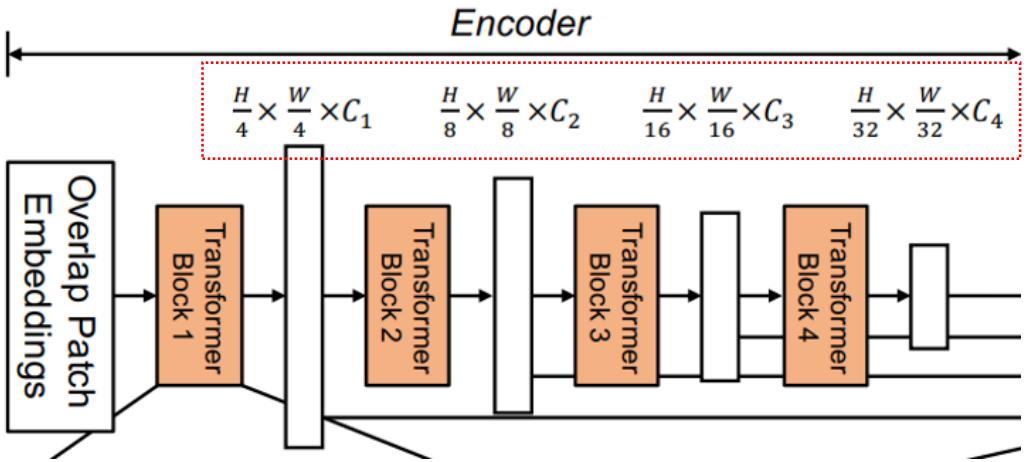
SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers, 2021



- A convolution-like operation is used (**kernel size > stride)**
- This creates patches that share pixels with neighboring patches
  - Preserves local spatial continuity across patch boundaries
  - Reduces boundary artifacts
  - Maintains finer details during downsampling



Block	Kernel	Stride	Padding
1	7	4	3
2, 3, 4	3	2	1



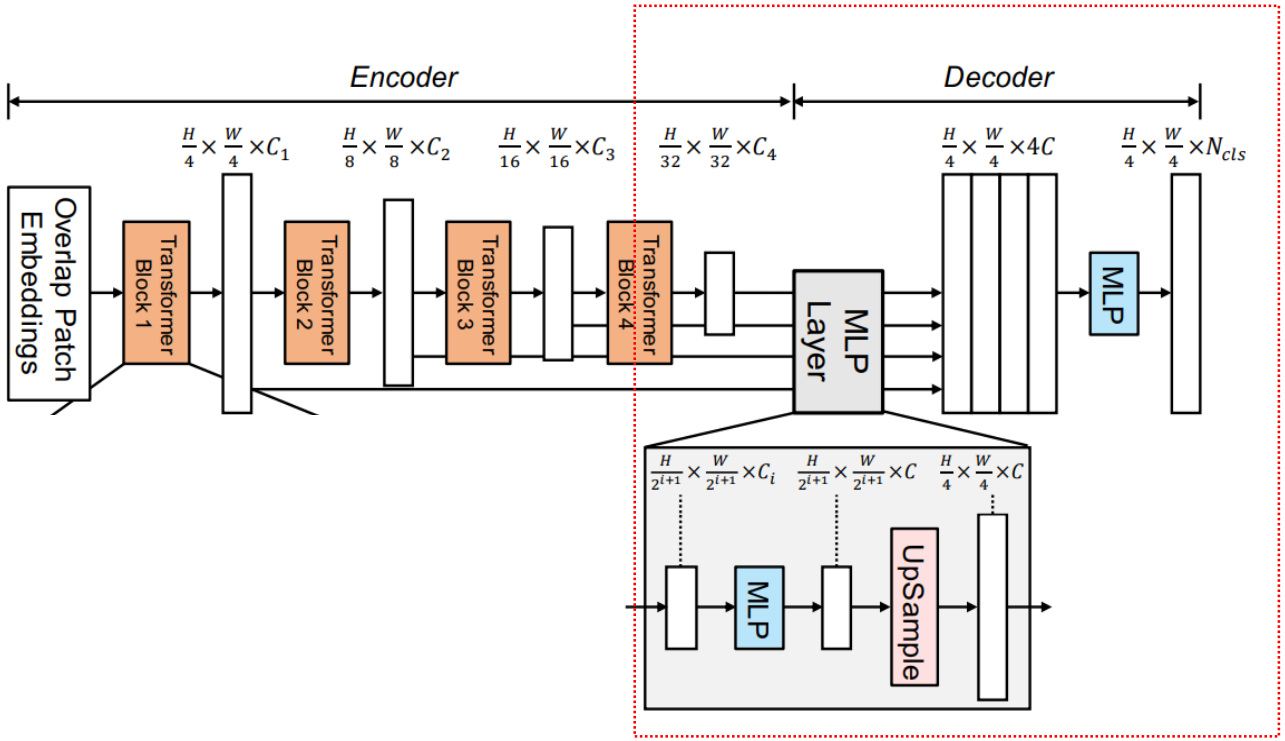
https://lcyking.tistory.com/entry/%EB%85%BC%EB%AC%B8%EB%A6%AC%EB%B7%B0-SegFormer-Simple-and-Efficient-Design-for-SemanticSegmentation-with-Transformers

MLP decoder

SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers, 2021

- **Fusing multi-scale features**
  - Conducted based on only MLPs
- Achieving **efficient and fast segmentation** without heavy convolutional structures

$$\hat{F}_i = \text{Linear}(C_i, C)(F_i), \forall i$$
$$\hat{F}_i = \text{Upsample}(\frac{W}{4} \times \frac{W}{4})(\hat{F}_i), \forall i$$
$$F = \text{Linear}(4C, C)(\text{Concat}(\hat{F}_i)), \forall i$$
$$M = \text{Linear}(C, N_{cls})(F),$$



Experiments (1/2)

Effective Receptive Field

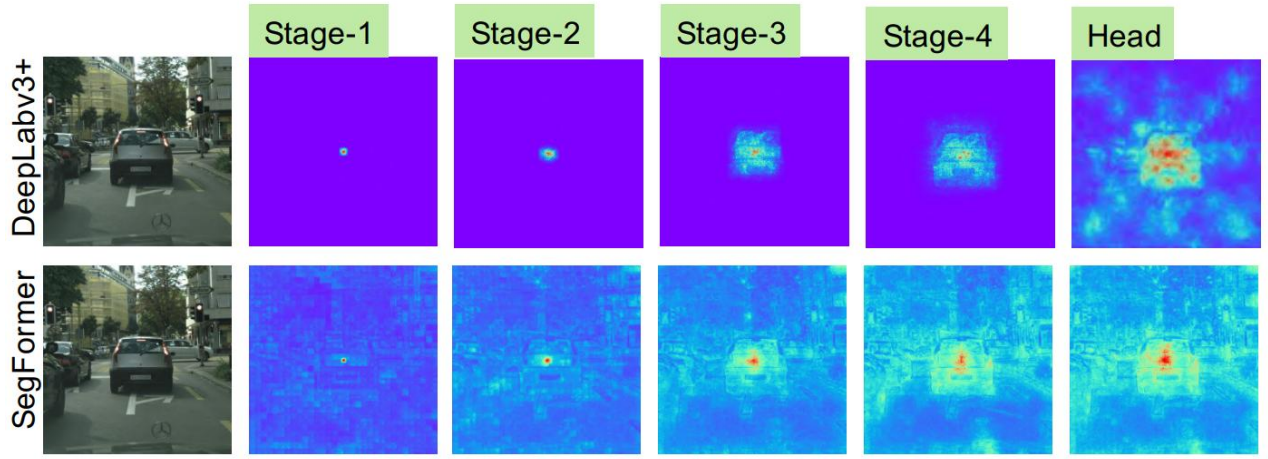


Figure 6: **Effective Receptive Field on Cityscapes.** ERFs of the four stages and the decoder heads of both architectures are visualized.

Effect of MLP dimension (C)

(b) Accuracy as a function of the MLP dimension C in the decoder on ADE20K.

C	Flops ↓	Params ↓	mIoU ↑
256	25.7	24.7	44.9
512	39.8	25.8	45.0
768	62.4	27.5	45.4
1024	93.6	29.6	45.2
2048	304.4	43.4	45.6

Effect of Mix-FFN

(c) Mix-FFN vs. positional encoding (PE) for different test resolution on Cityscapes.

Inf Res	Enc Type	mIoU ↑
768×768	PE	77.3
1024×2048	PE	74.0
768×768	Mix-FFN	80.5
1024×2048	Mix-FFN	79.8

Big jump !

Experiments (2/2)

SegFormer
= Hierarchical Transformer + MLP Decoder
→ Simple + Accurate + Fast !

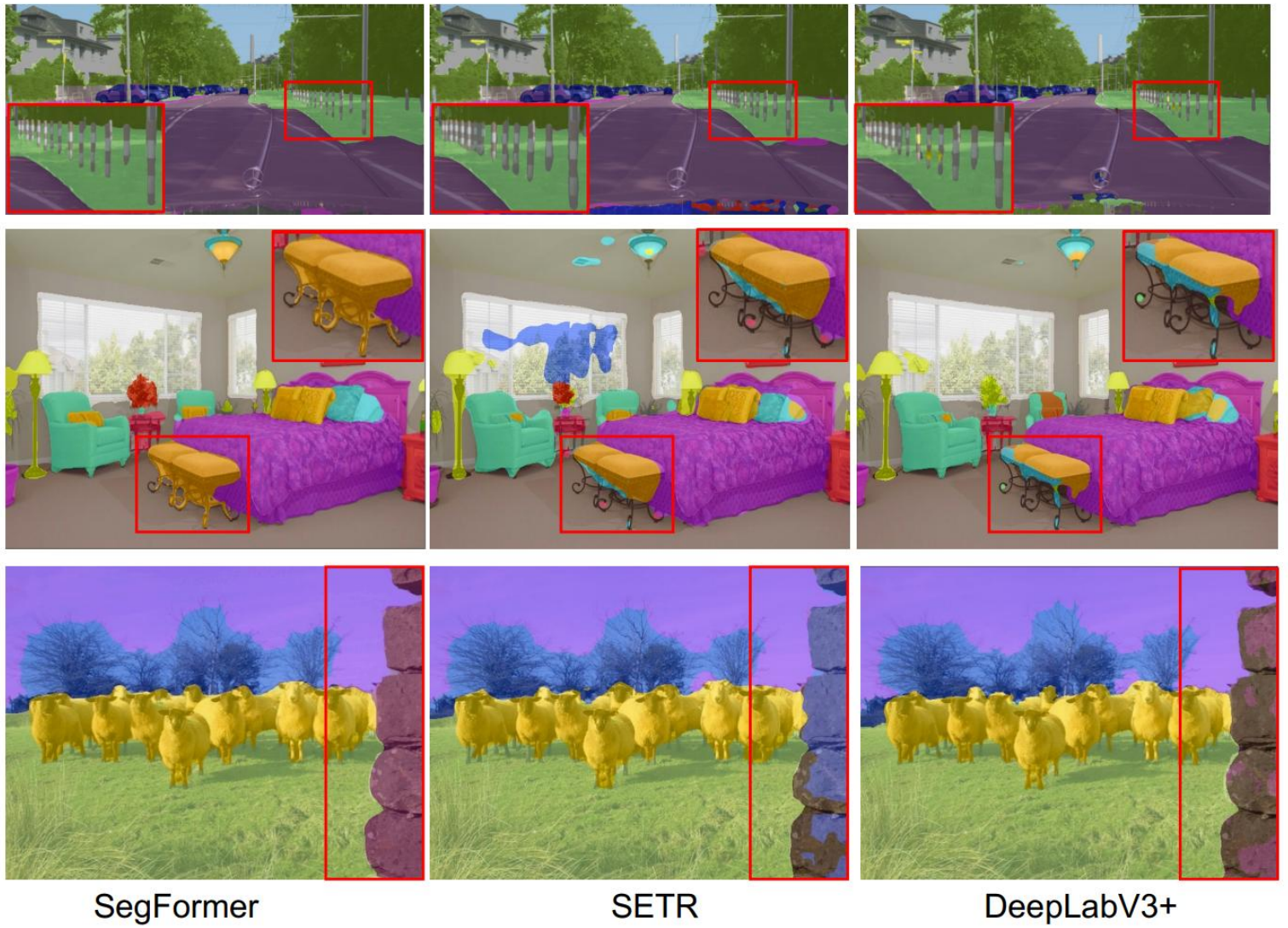


Figure 5: **Qualitative results on Cityscapes, ADE20K and COCO-Stuff.** First row: Cityscapes. Second row: ADE20K. Third row: COCO-Stuff. Zoom in for best view.

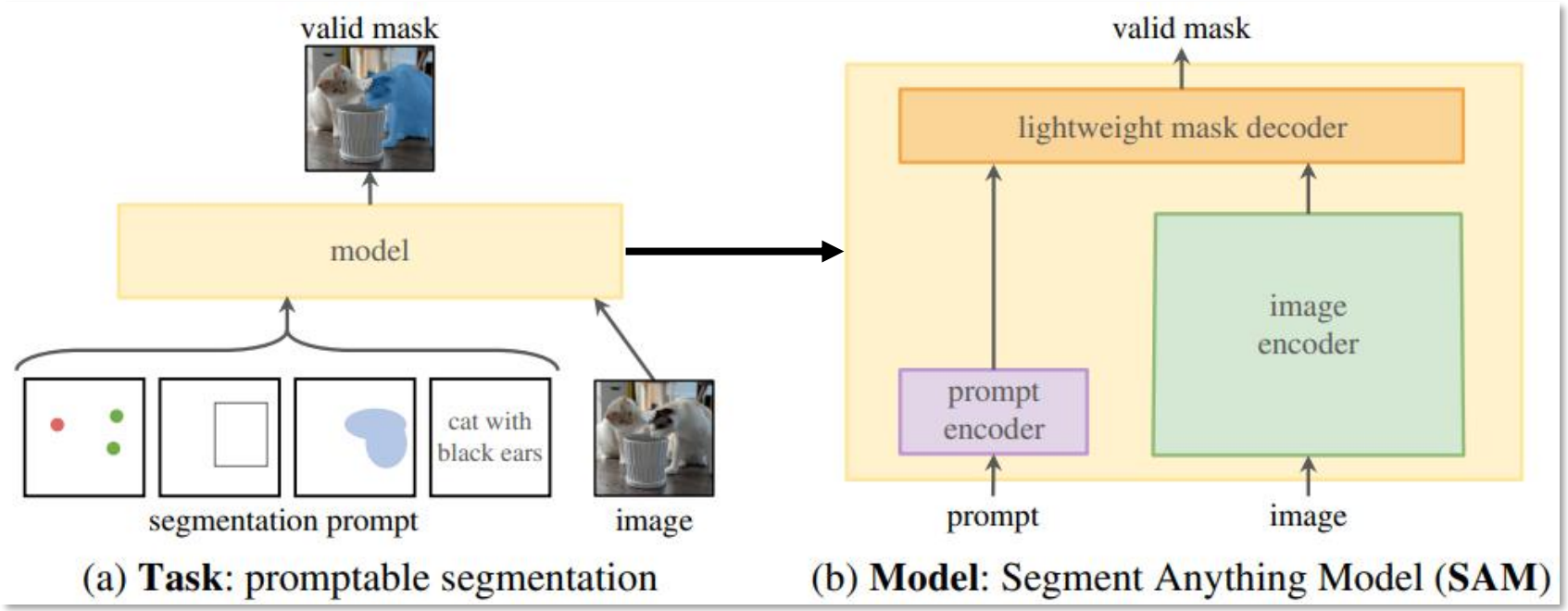
[SAM]

- **Goal**
  - Segment any object, even without retraining → VFM (Vision Foundation Model) for general purposes

- Zero-shot segmentation
- Edge detection
- Object proposal
- Instance segmentation
- Text-based segmentation

- **Task → "Promptable segmentation"**
  - Points
  - Bounding box
  - Text phrase
  - Free form mask editing

- **Model**
  - Image encoder
  - Prompt encoder
  - Mask decoder



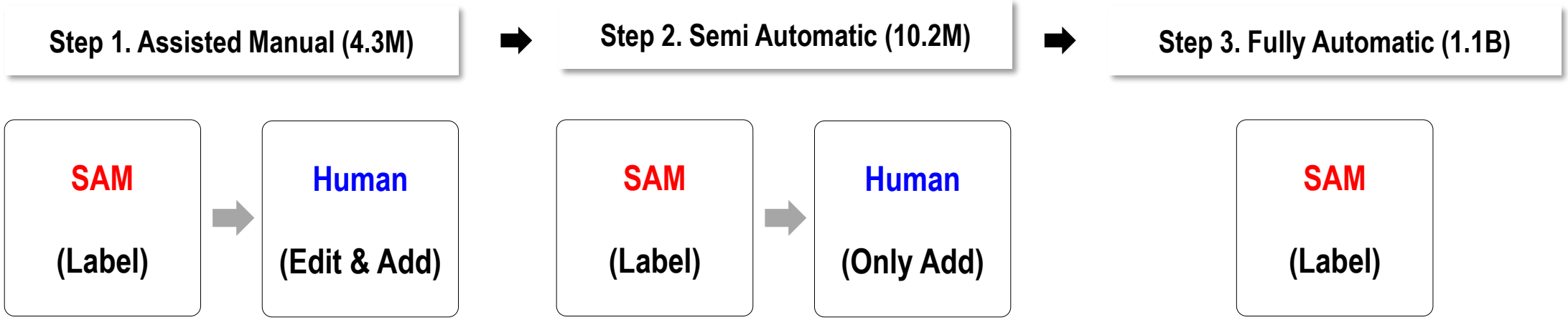
Dataset & Labeling

- **Data set**

Dataset	# of images	# of masks
LVIS v1	120K	1.5M
MS COCO	120K	900K
ADE20	28K	700K
Open Images	1M	2.7M
SA-1B (SAM)	11M	1.1B

X 400

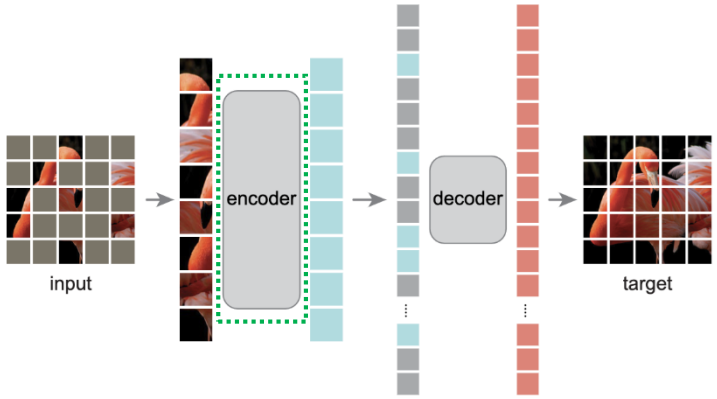
- **Labeling**



Architecture

- **Image Encoder**
  - Vision Transformer trained with MAE (Masked Auto Encoder)
- **Prompt Encoder**
  - Mask → Convolution then pixel-wise sum
  - Points / Box → Positional embedding

    Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains, 2020
  - Text → Text encoder of CLIP
- **Mask Decoder**
  - Lightweight transformer
  - Multiple mask candidates + IoU score



Masked Auto Encoder

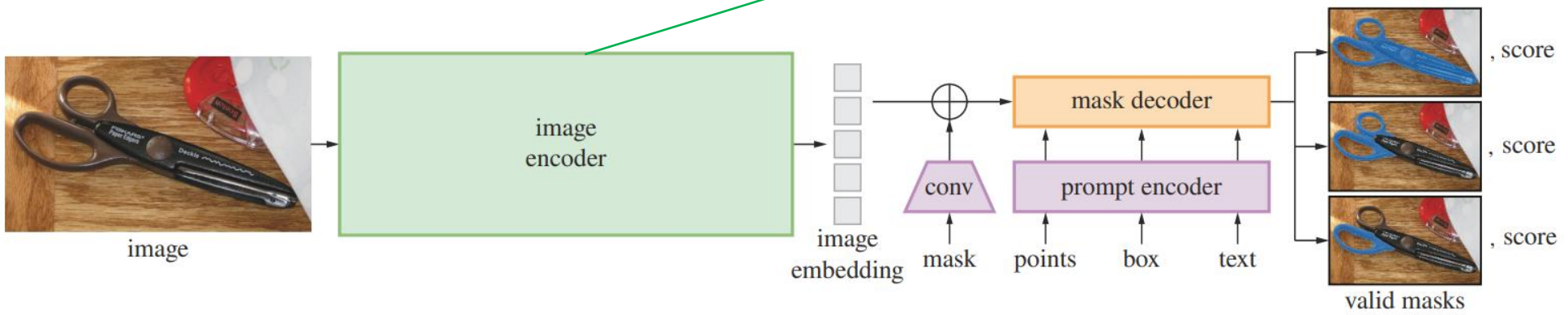
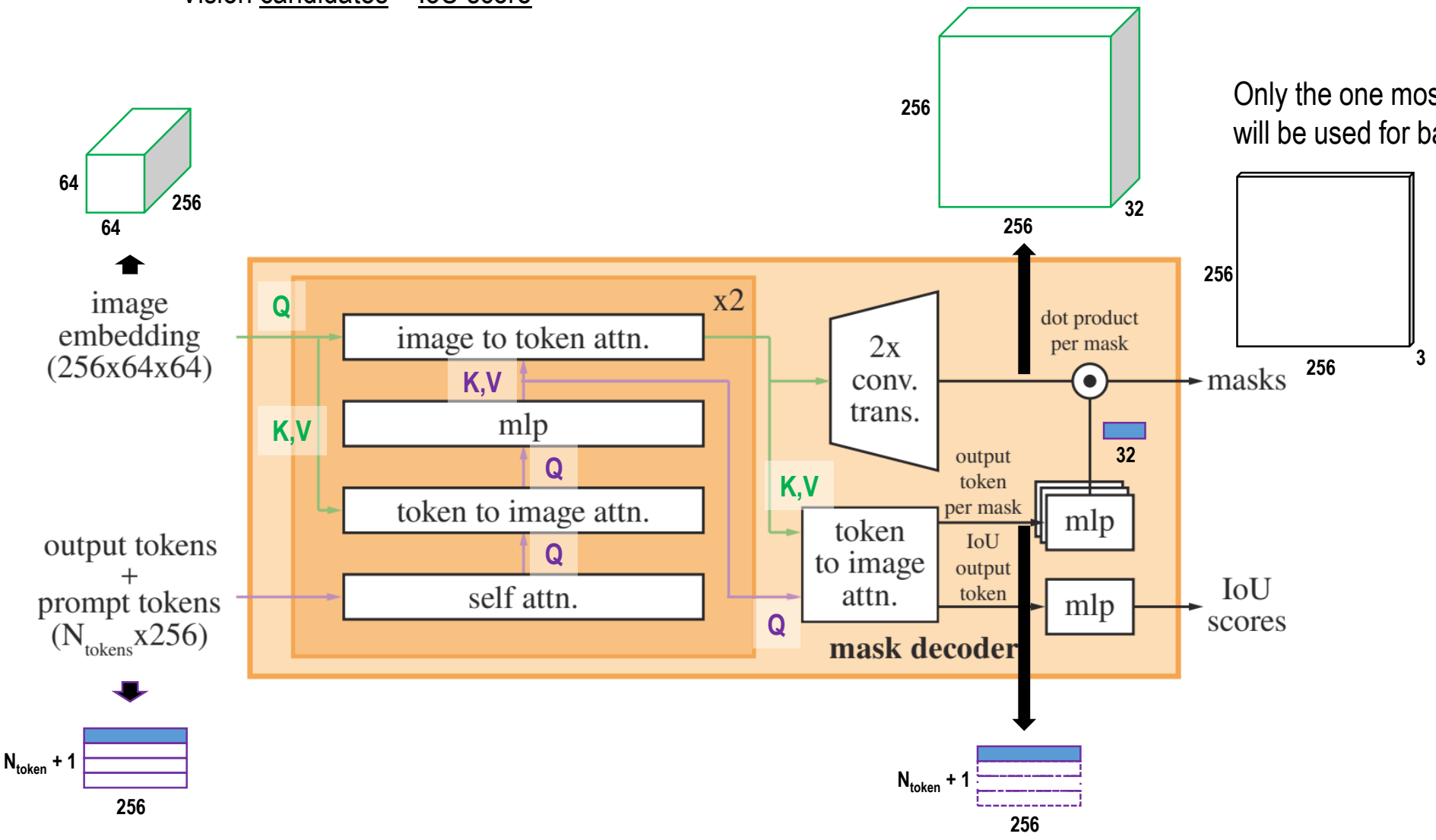


Figure 4: Segment Anything Model (SAM) overview. A heavyweight image encoder outputs an image embedding that can then be efficiently queried by a variety of input prompts to produce object masks at amortized real-time speed. For ambiguous prompts corresponding to more than one object, SAM can output multiple valid masks and associated confidence scores.

Mask decoder

- **Cross attention between image embedding and prompt embedding**
  - Vision candidates + IoU score



Only the one most similar to the ground truth will be used for backpropagation

Ambiguity

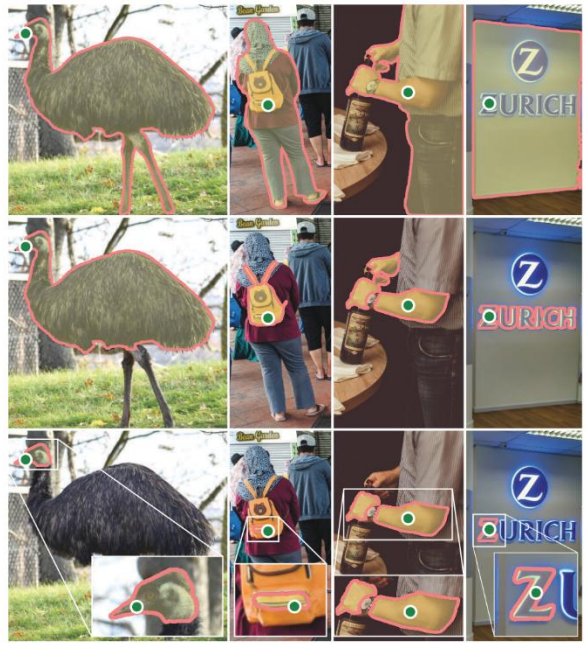
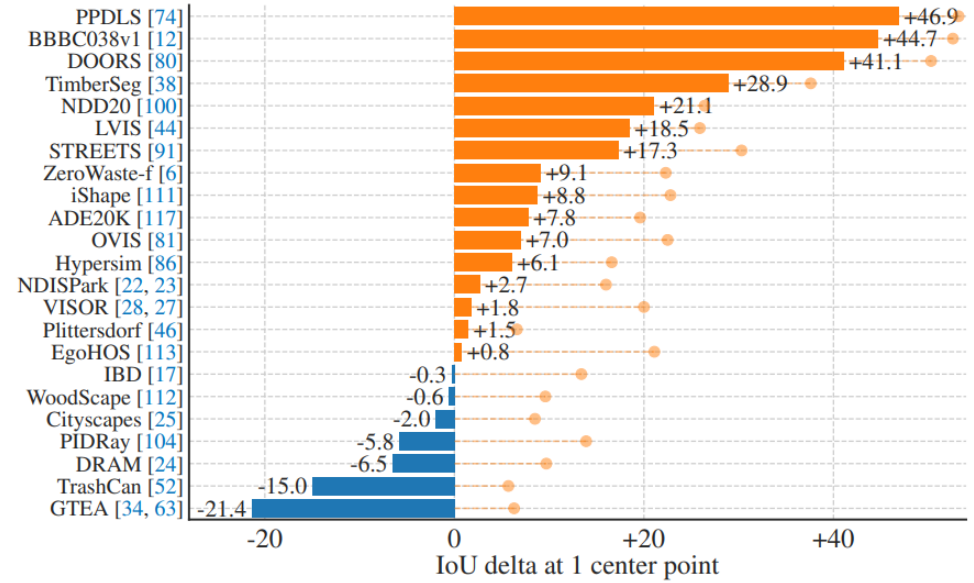


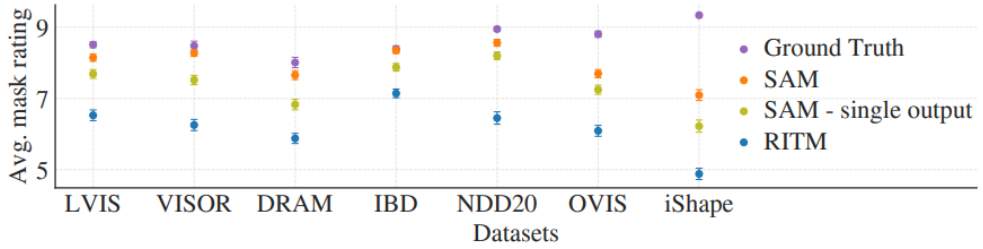
Figure 3: Each column shows 3 valid masks generated by SAM from a single ambiguous point prompt (green circle).

Zero-shot experiments (1/3)

Single point segmentation



(a) SAM *vs*. RITM [92] on 23 datasets



(b) Mask quality ratings by human annotators

Edge detection

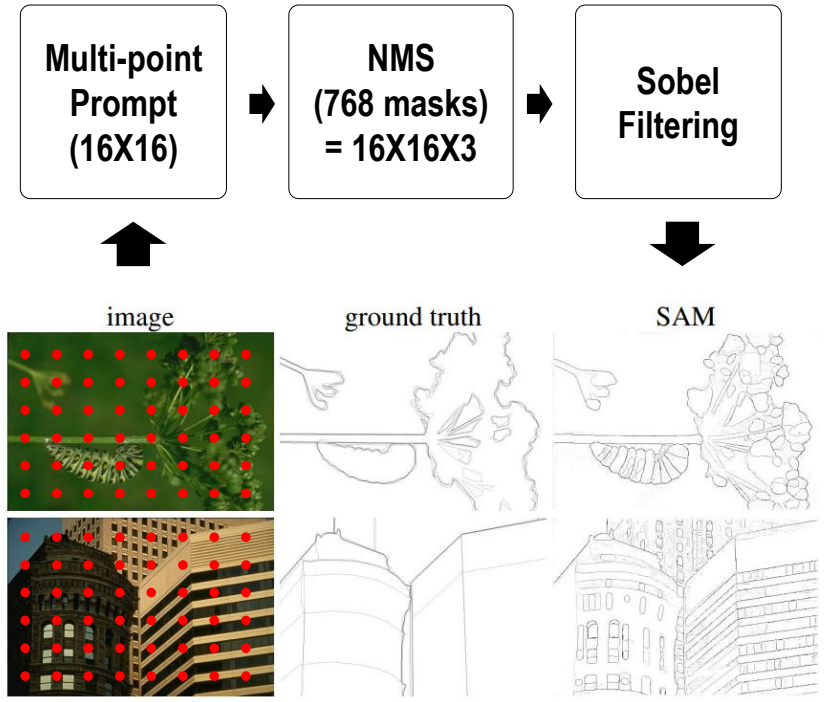


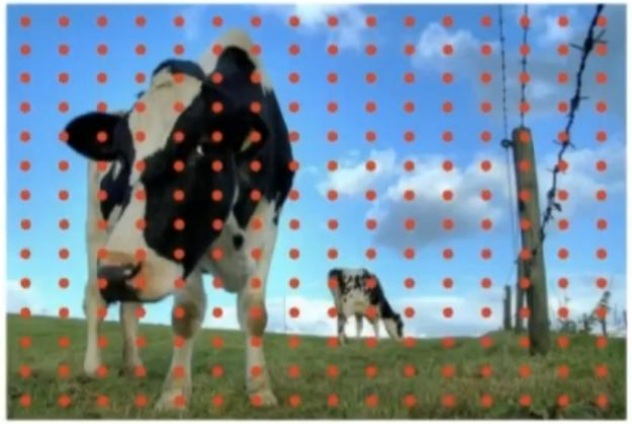
Figure 10: Zero-shot edge prediction on BSDS500. SAM was not trained to predict edge maps nor did it have access to BSDS images or annotations during training.

Zero-shot experiments (2/3)

Object proposal

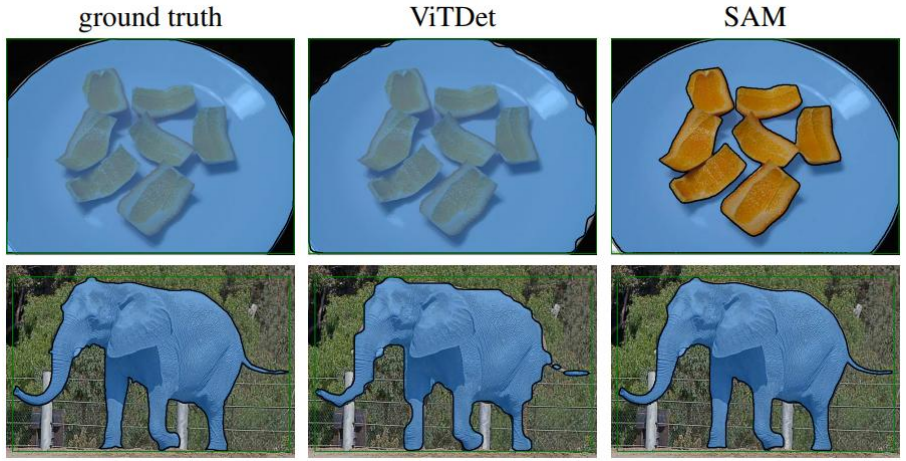
method	mask AR@1000 all	small	med.	large	freq.	com.	rare
ViTDet-H [62]	63.0	51.7	80.8	87.0	63.1	63.3	58.3
zero-shot transfer methods:							
SAM – single out.	54.9	42.8	76.7	74.4	54.7	59.8	62.0
SAM	59.3	45.5	81.6	86.9	59.1	63.9	65.8

Table 4: Object proposal generation on LVIS v1. SAM is applied zero-shot, *i.e.* it was not trained for object proposal generation nor did it access LVIS images or annotations.

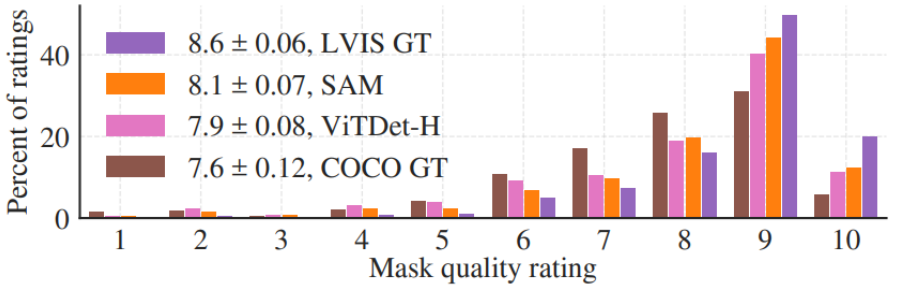


1000 masks

Instance segmentation



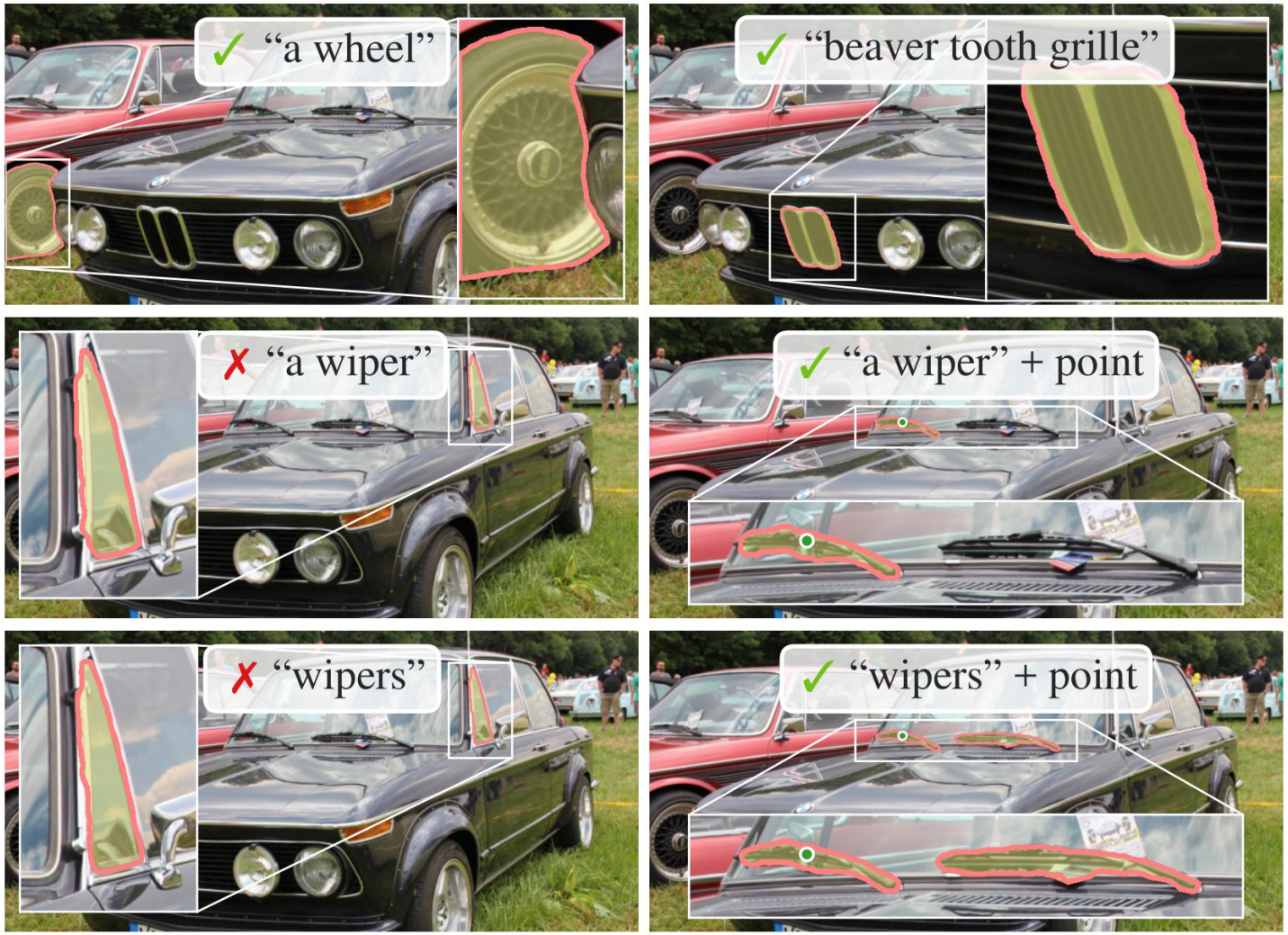
method	COCO [66] AP	AP^S	AP^M	AP^L	LVIS v1 [44] AP	AP^S	AP^M	AP^L
ViTDet-H [62]	51.0	32.0	54.3	68.9	46.6	35.0	58.0	66.3
zero-shot transfer methods (segmentation module only):								
SAM	46.5	30.8	51.0	61.7	44.7	32.5	57.6	65.5



Zero-shot experiments (3/3)

Text to mask

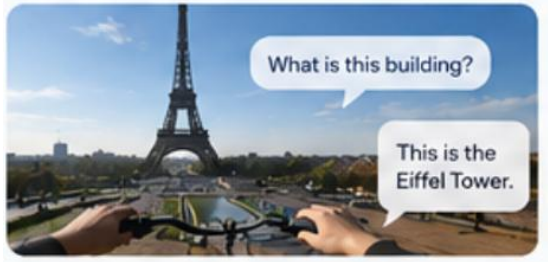
- CLIP image encoder
- CLIP text encoder



Summary

- **A universal mask engine**
- **Annotation tool** → dramatically reduce labeling labor
- **Base module** for:
  - Grounded SAM (text + segmentation)
  - Open-Vocabulary Segmentation
  - Robotics grasping / navigation (zero-shot)
- **Evolution**





SAM is a platform, not just a model !

SAM3 GitHub

	SAM (2023)	SAM 2 (2024)	SAM 3 (2025)
Supported media	Single images	Images + Video frames	Images + Video (unified)
Prompt types (official)	Visual prompts (points / boxes / masks) Text	points / boxes / masks	Visual prompts + Text (noun-phrase) + Image exemplar (concept prompt)
Output behavior	Single object mask per prompt	Single object mask per prompt (per frame) with temporal tracking	All instances of given concept → segmentation masks + detection + tracking
Use case emphasis	Interactive segmentation, annotation, zero-shot mask generation	Video segmentation / tracking + image segmentation	Open-vocabulary segmentation / detection / tracking, video + image, concept-wide mask generation
Main limitation	No video support; needs manual visual prompt for each target	Still needs prompt per target; prompt still visual only	Complex prompts supported but limited to simple noun-phrases; relational reasoning needs external modules